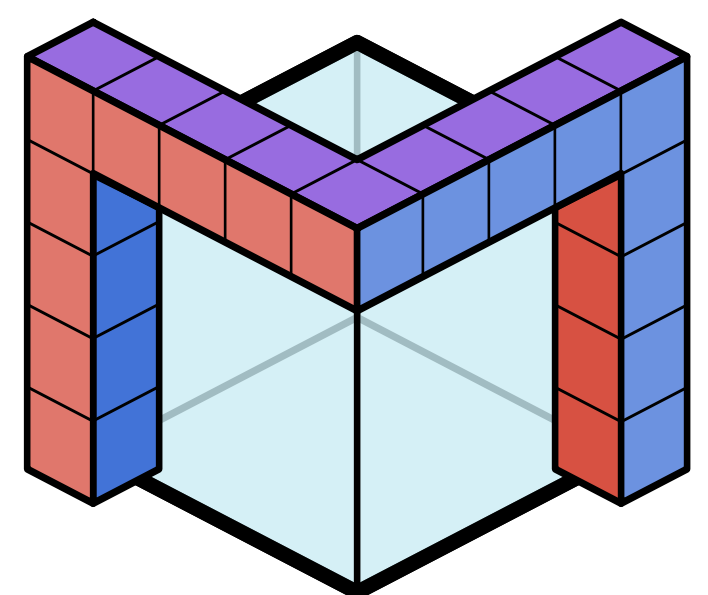
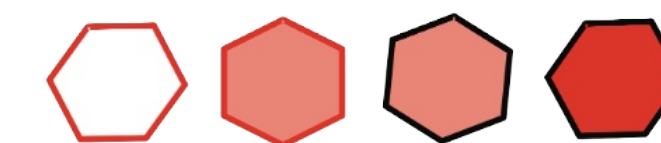




metatensor

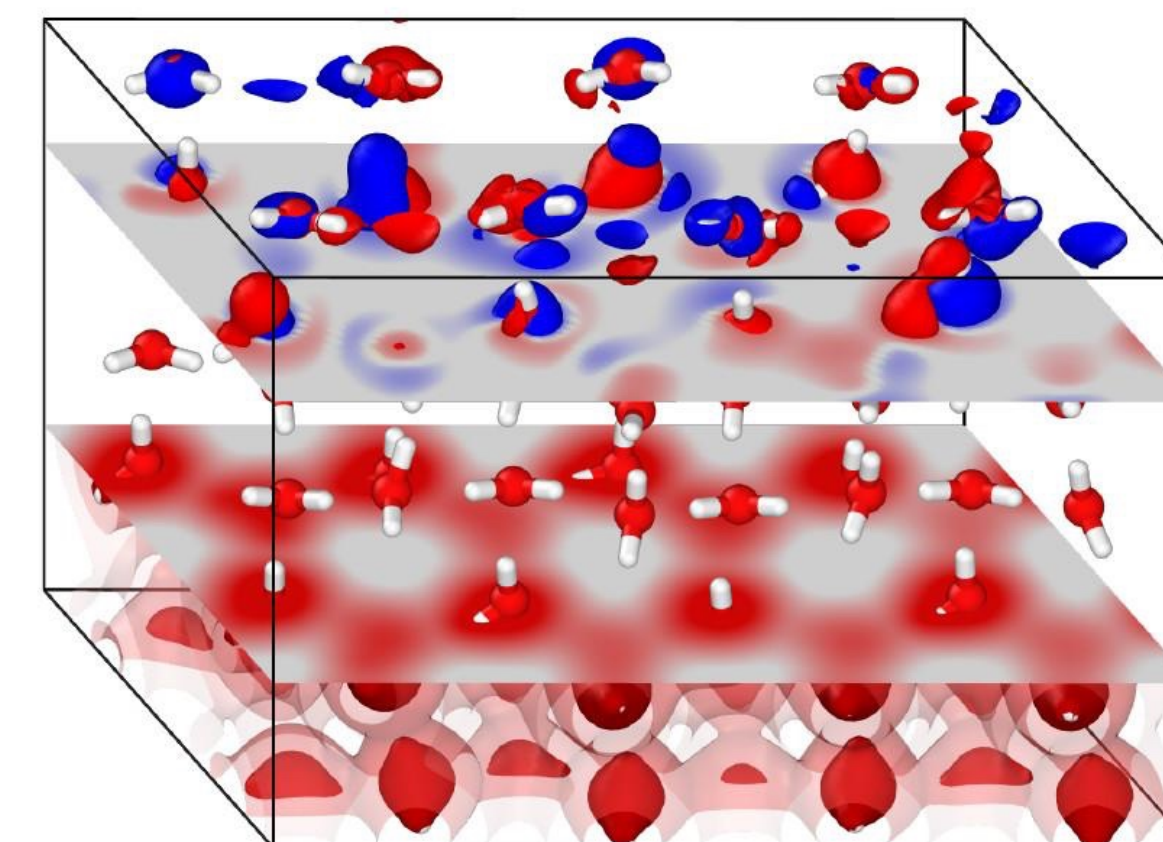
MARVEL



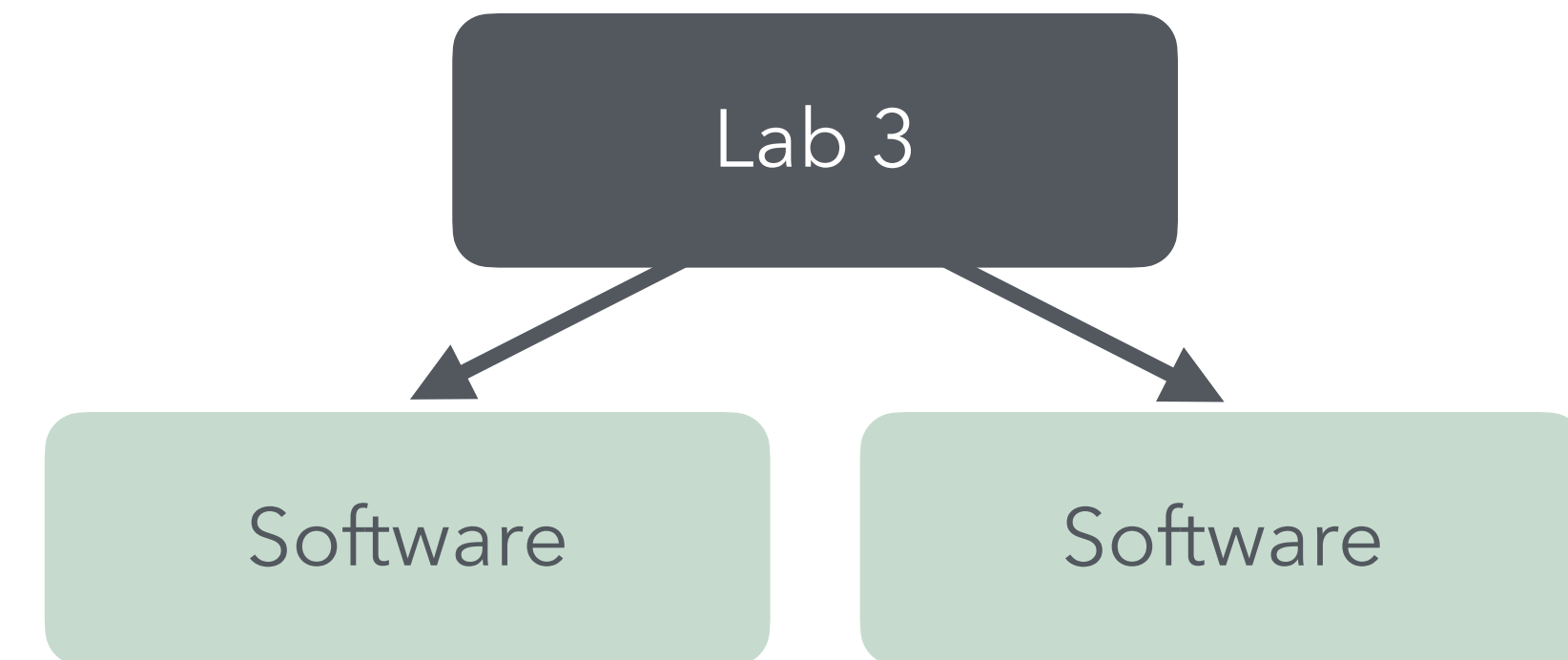
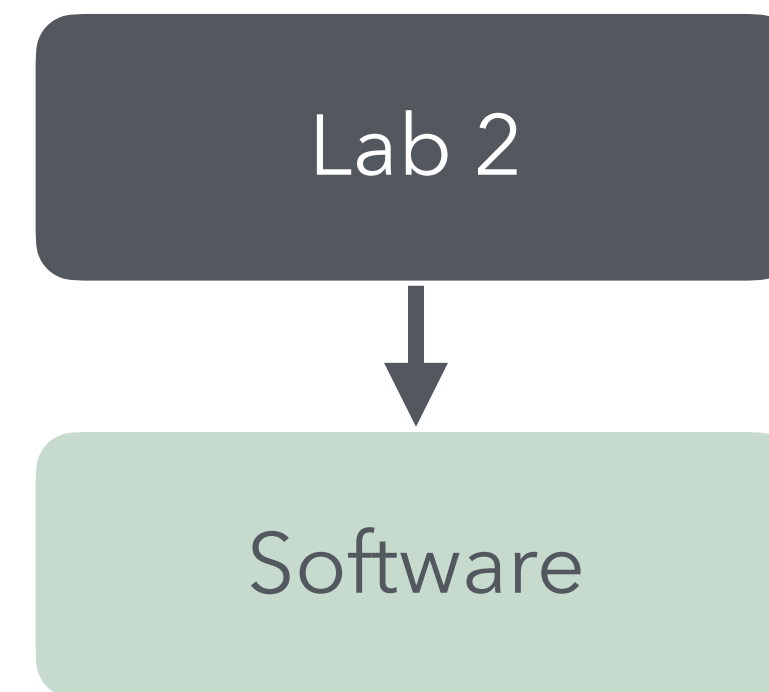
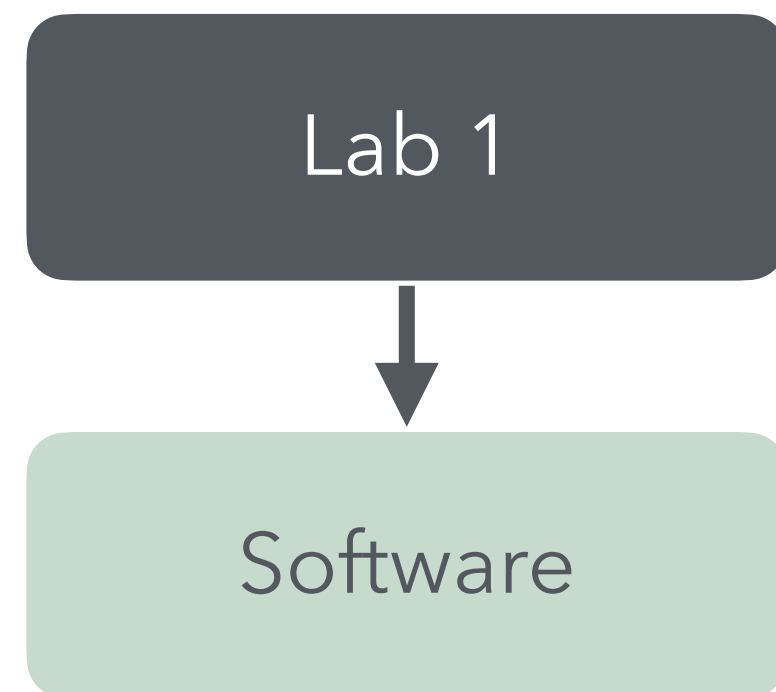
NATIONAL CENTRE OF COMPETENCE IN RESEARCH

Metatensor and metatomic

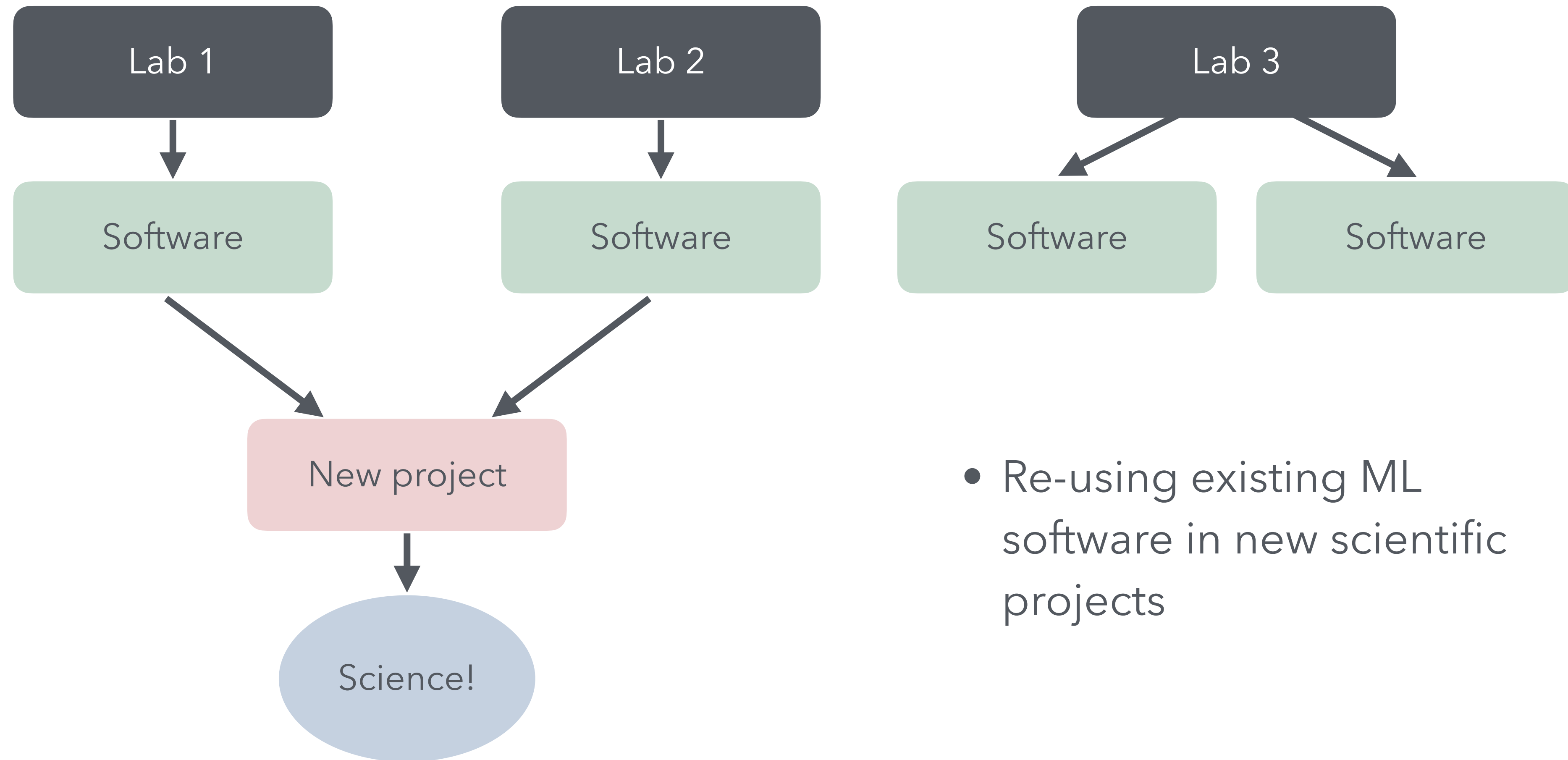
Machine learning models you can use everywhere for everything



The state of things

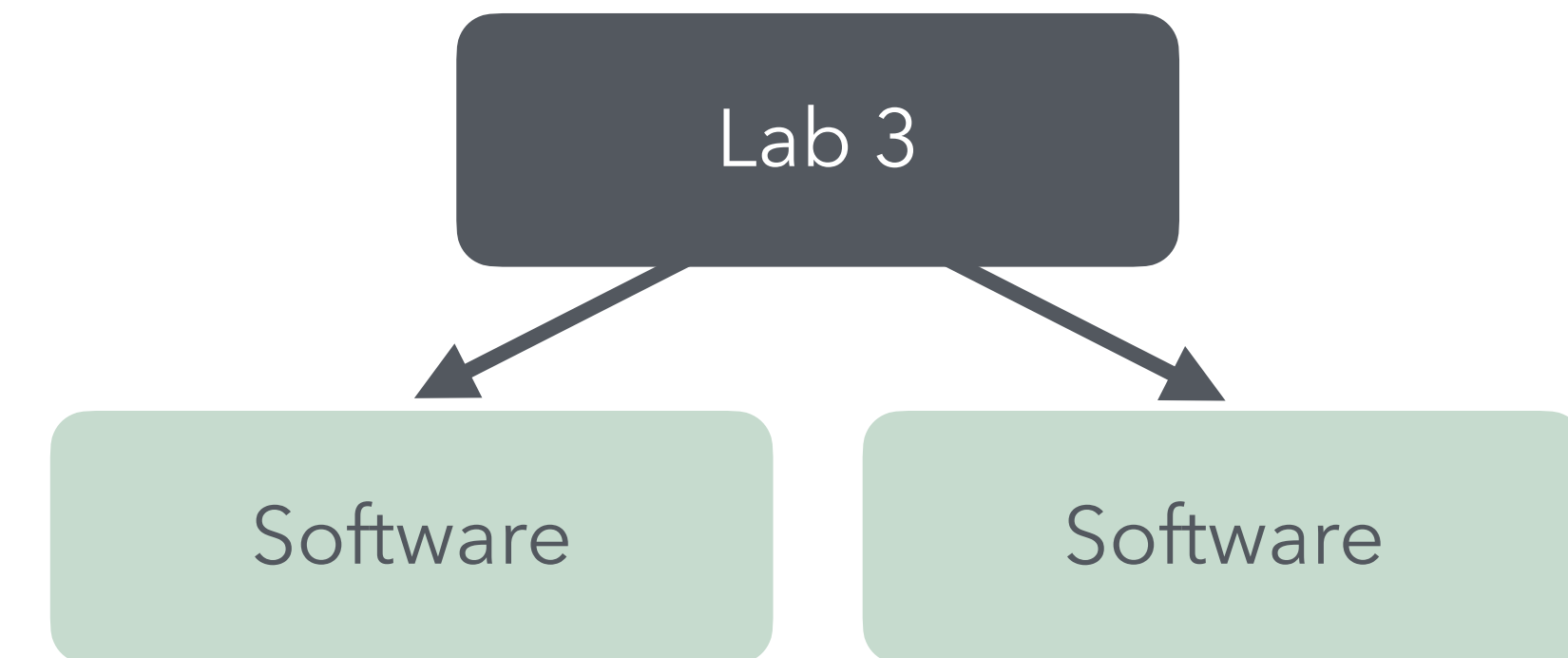
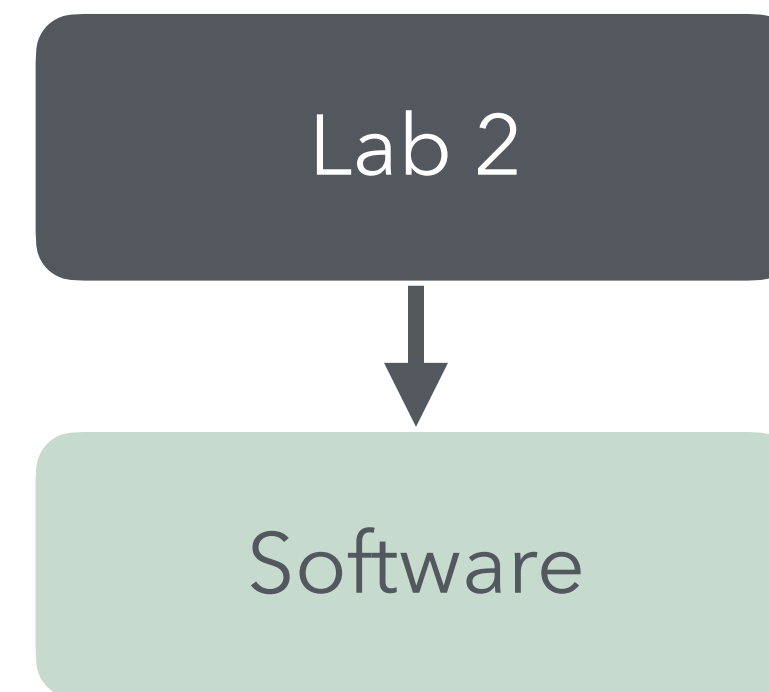
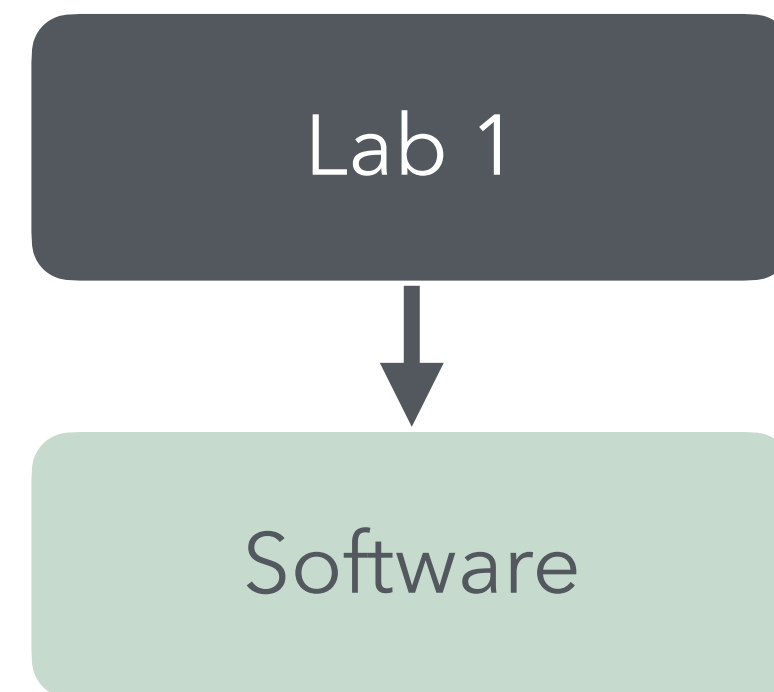


The state of things



- Re-using existing ML software in new scientific projects

The state of things



Existing code

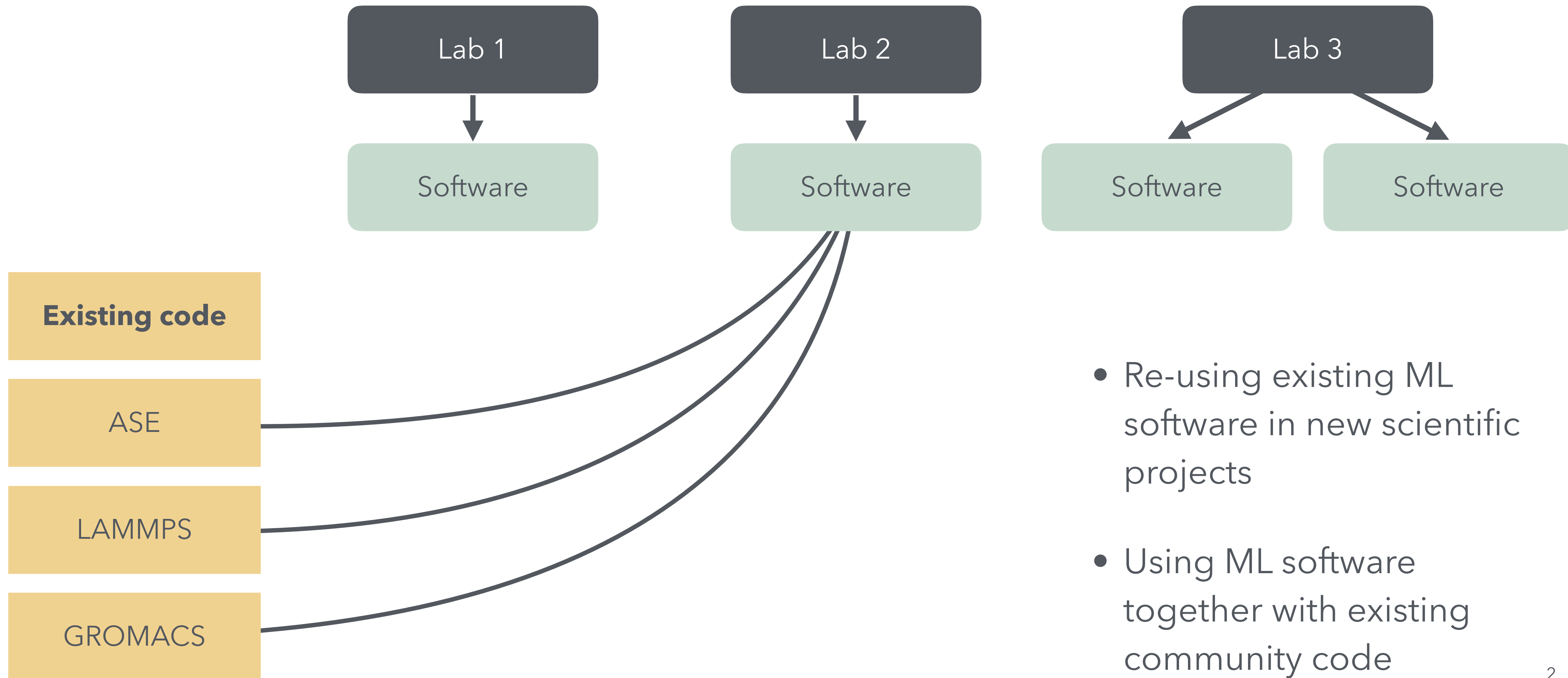
ASE

LAMMPS

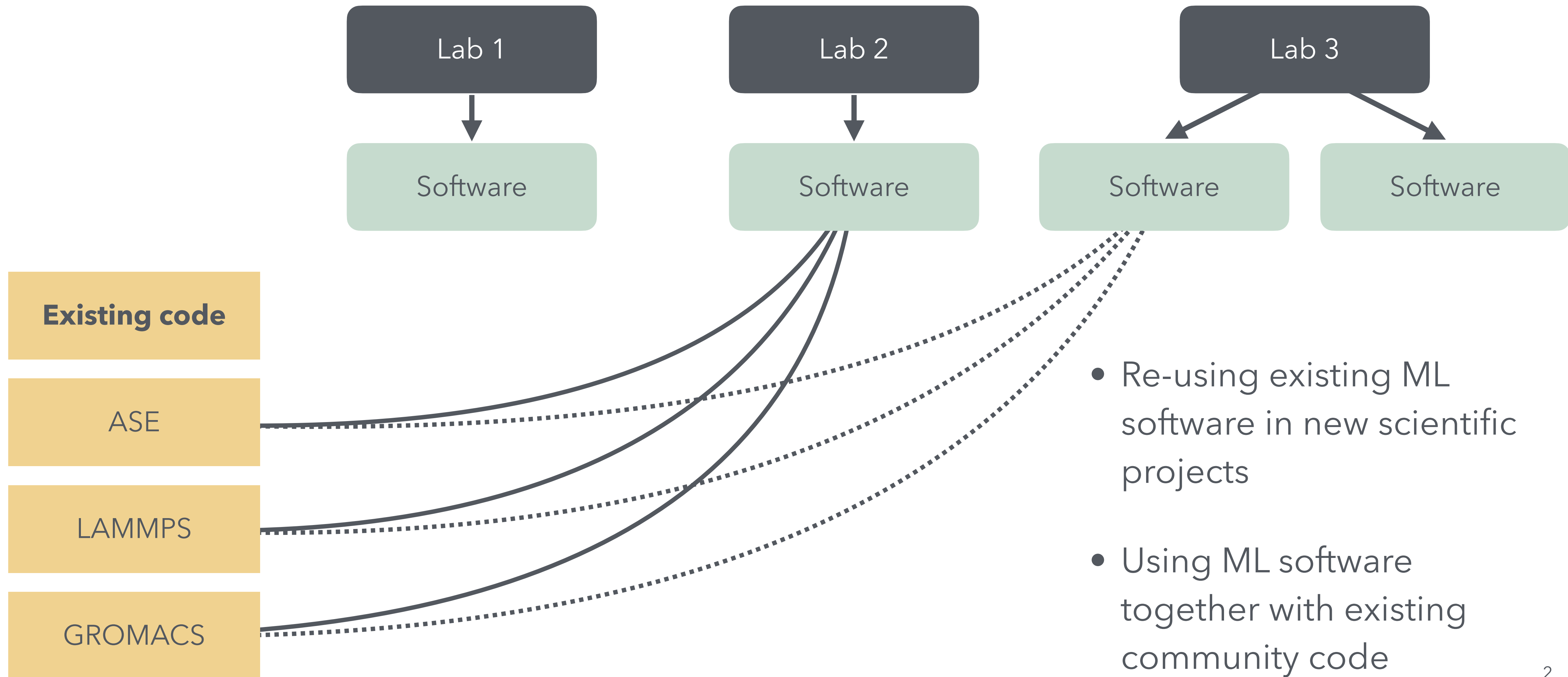
GROMACS

- Re-using existing ML software in new scientific projects
- Using ML software together with existing community code

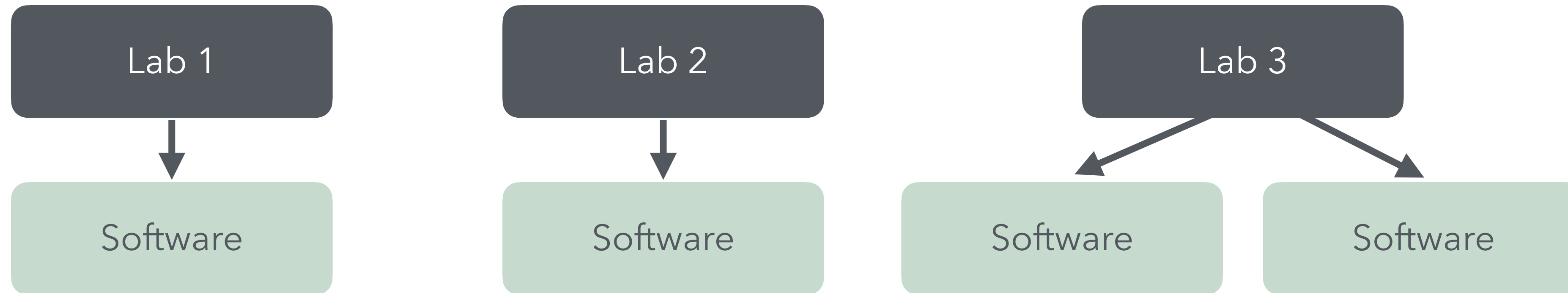
The state of things



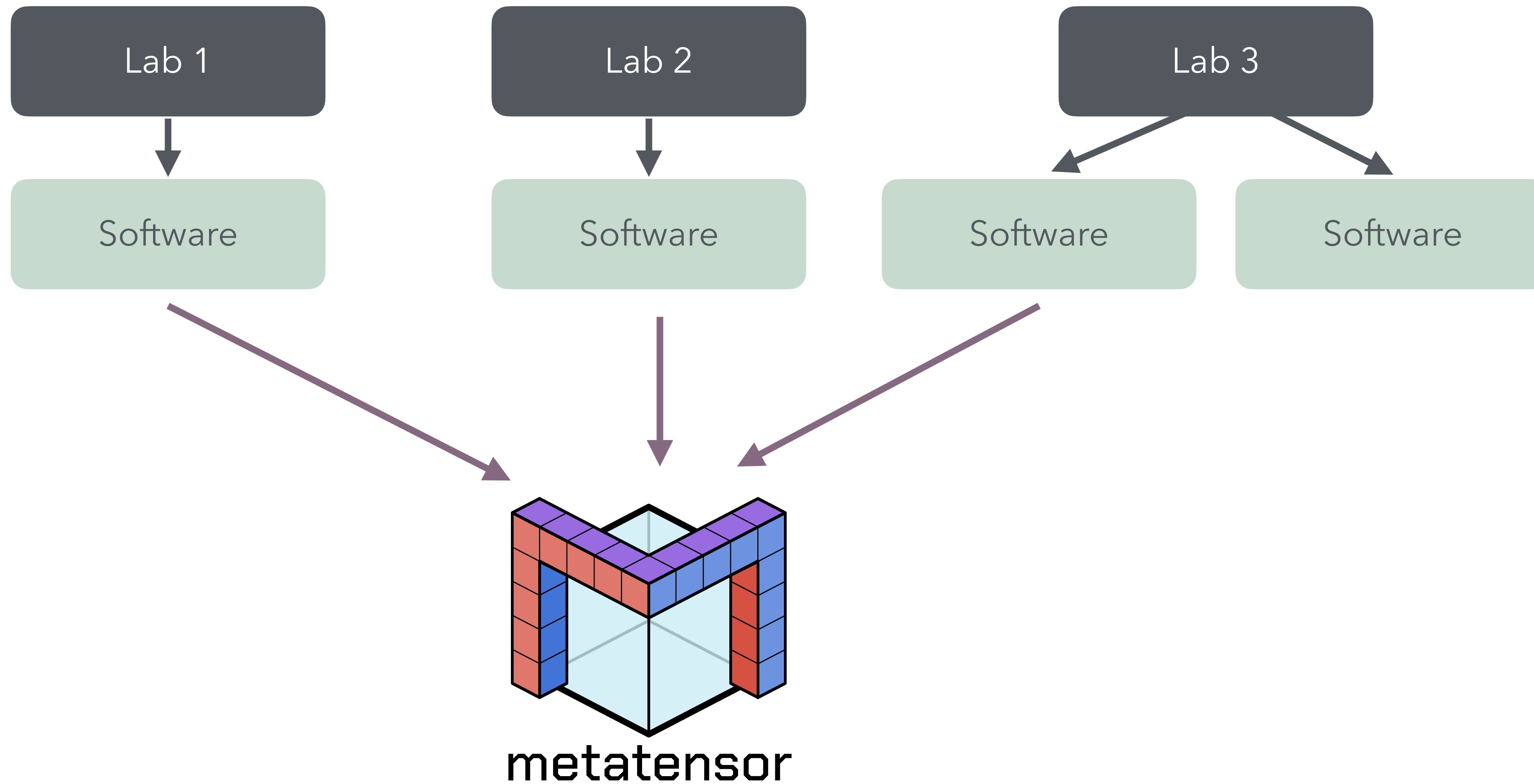
The state of things



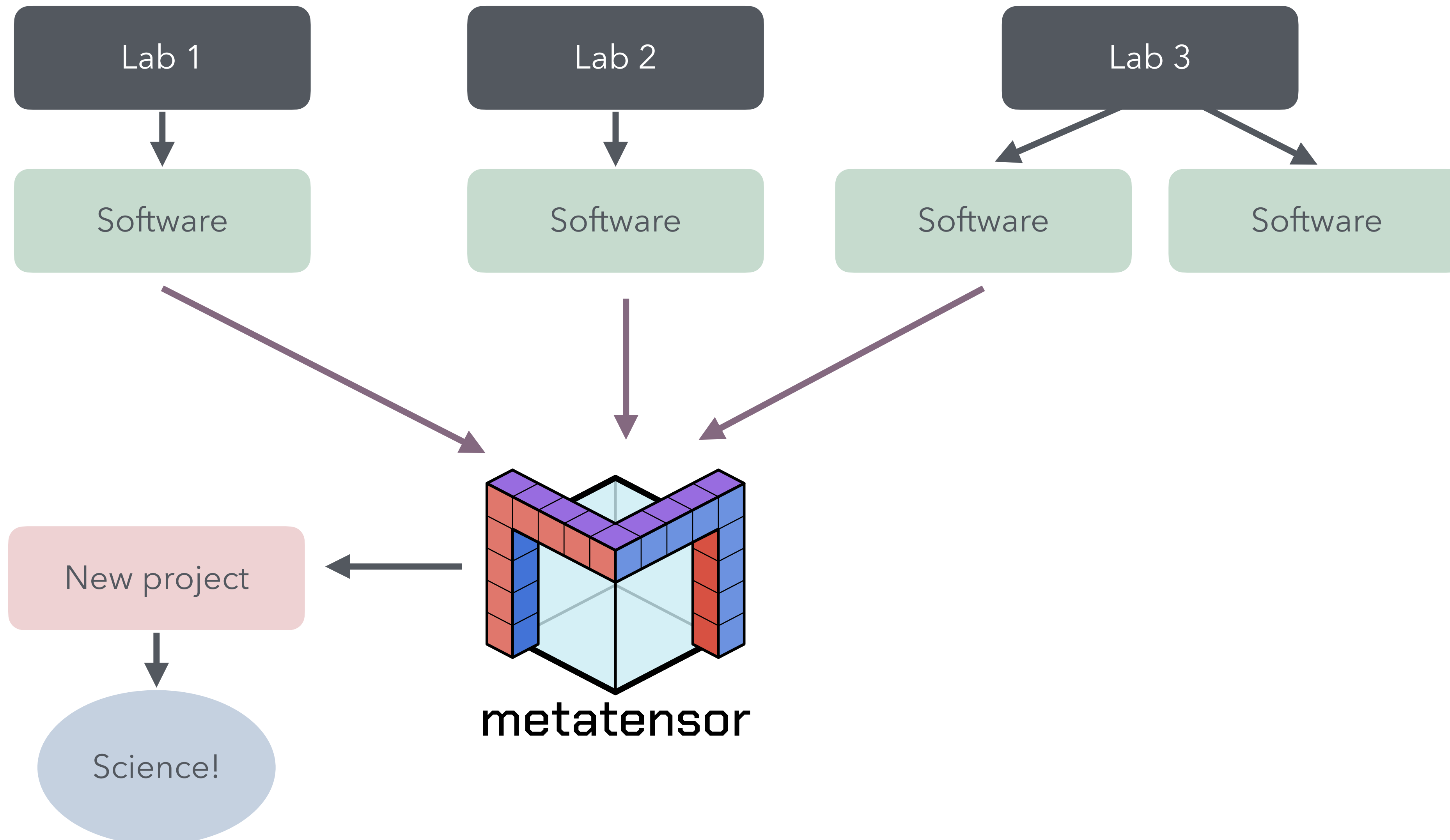
Creating convergence points



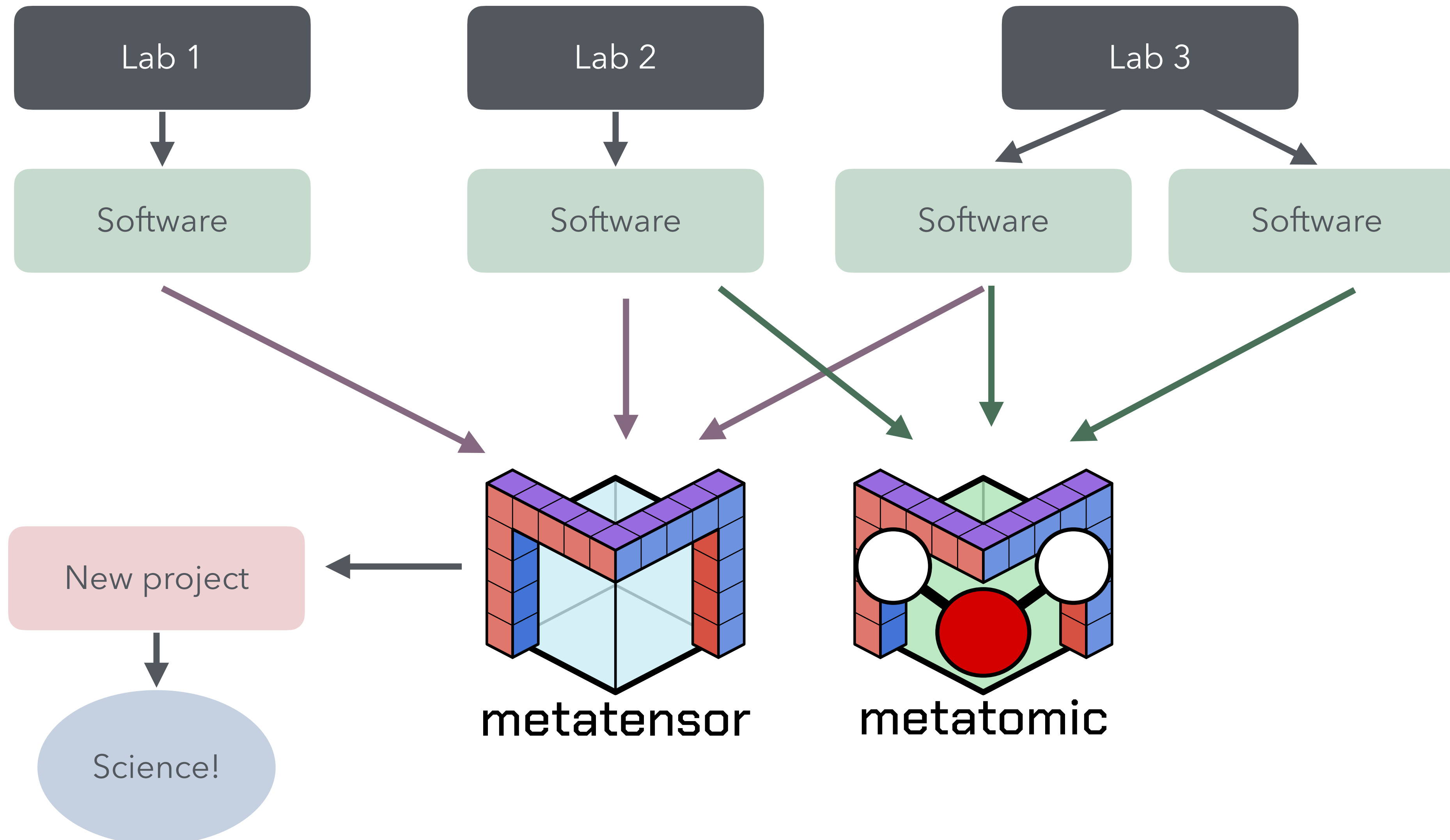
Creating convergence points



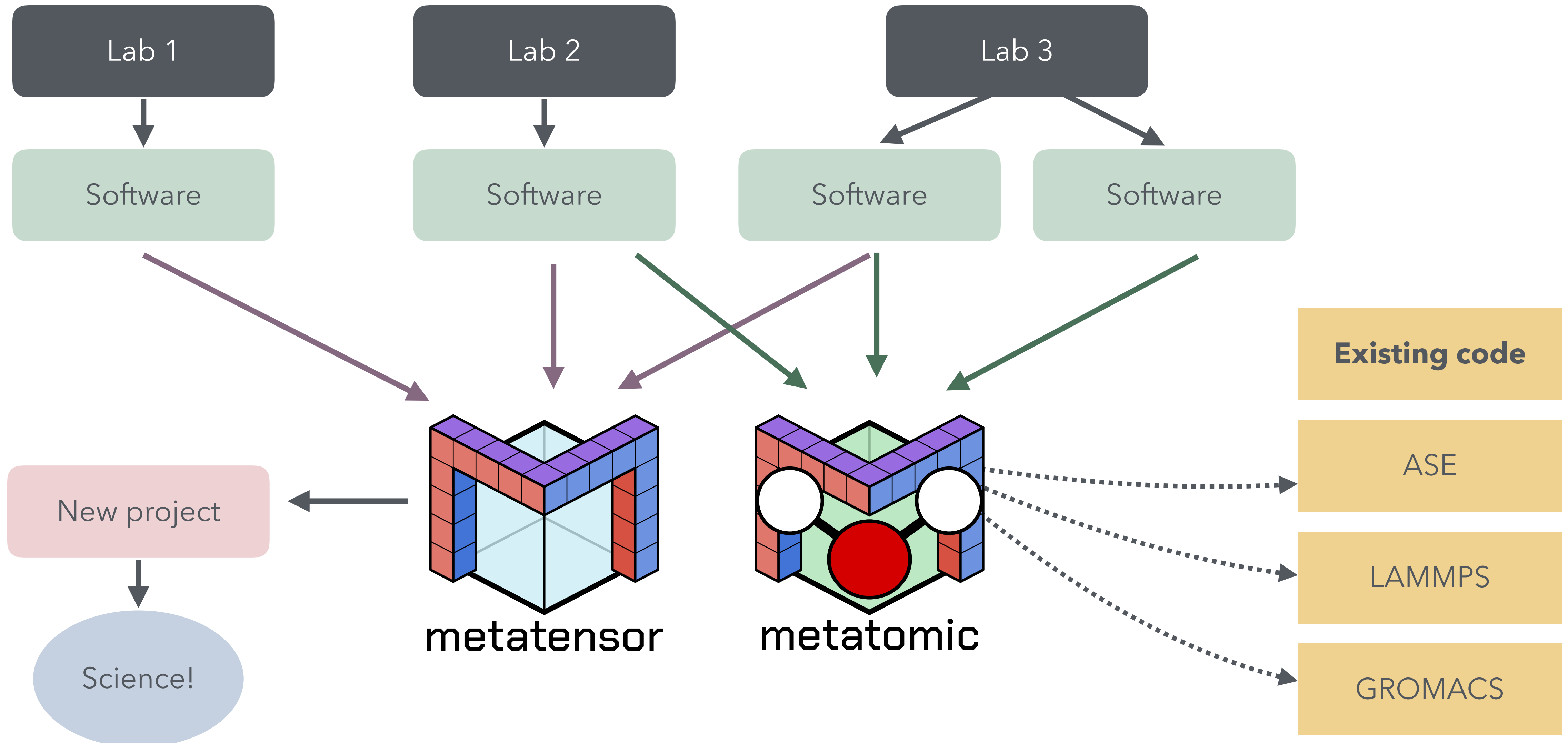
Creating convergence points

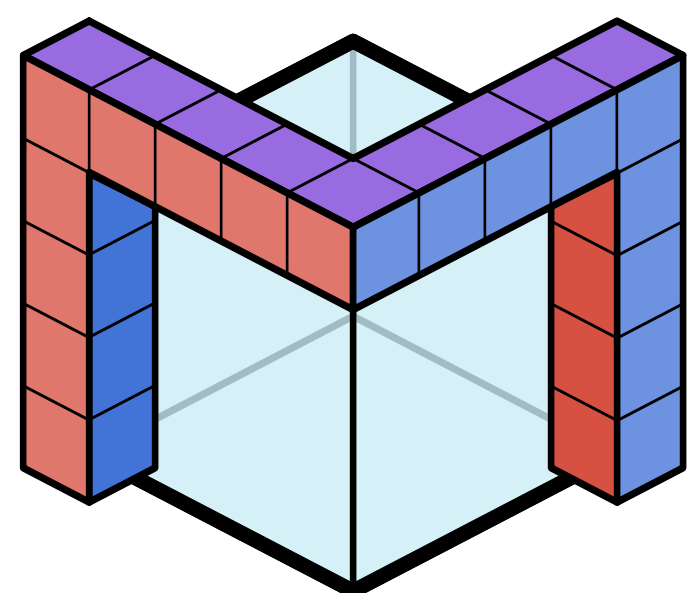


Creating convergence points



Creating convergence points





metatensor

mts-operations

mts-learn

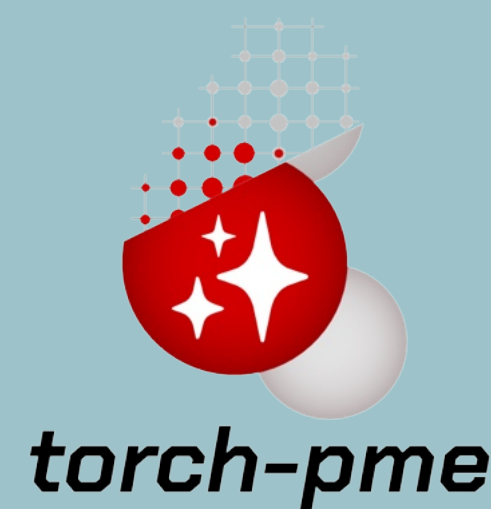
SALTED

featonic

AniSOAP

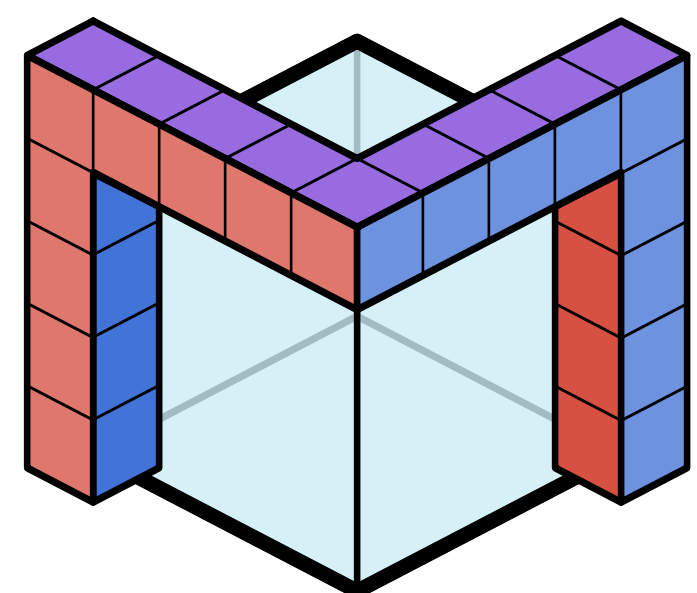
vesin

spex

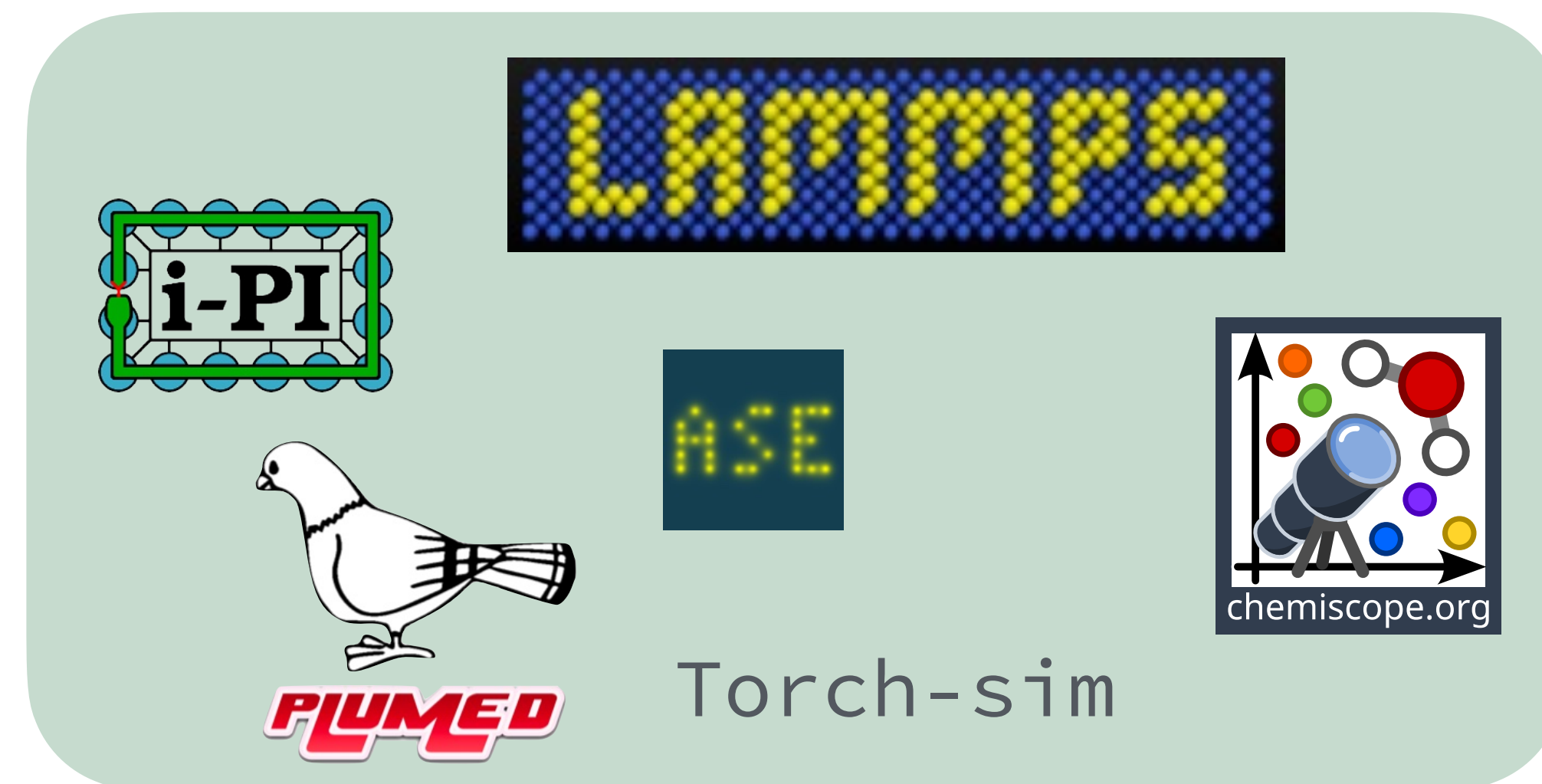


torch-pme

Q-Stack

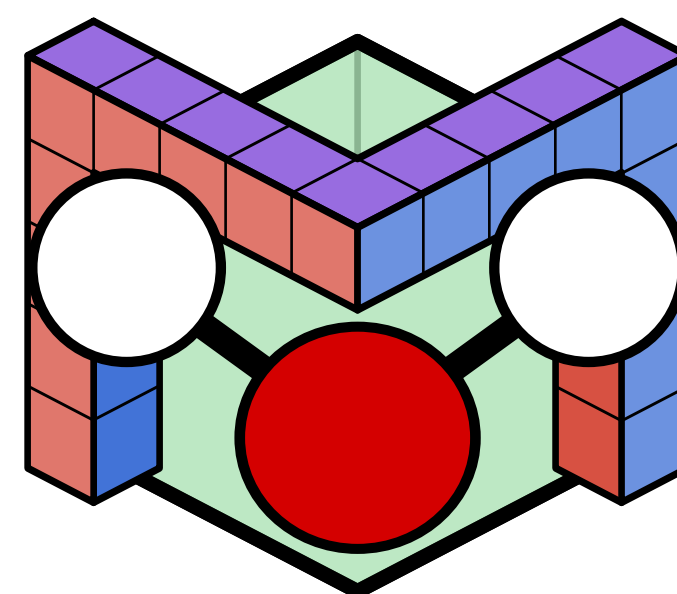


metatensor



mts-operations

mts-learn



metatomic

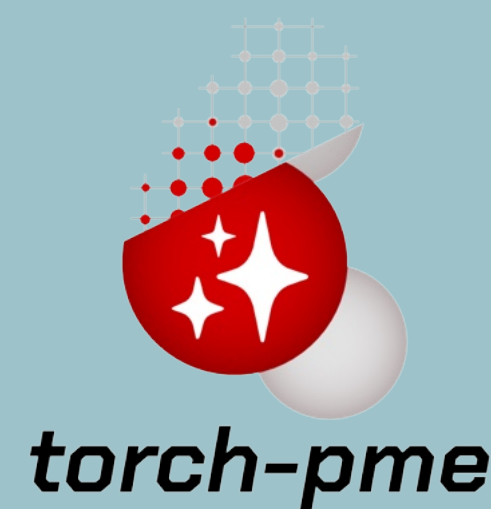
AniSOAP

SALTED

featomic

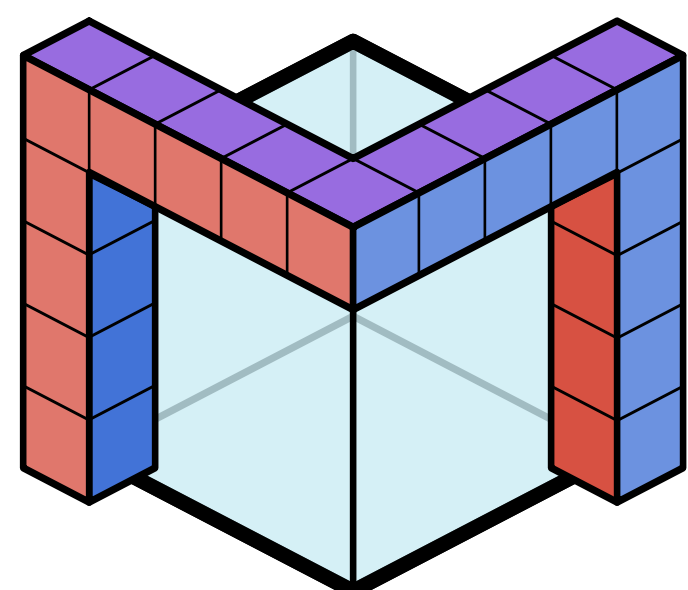
vesin

spex

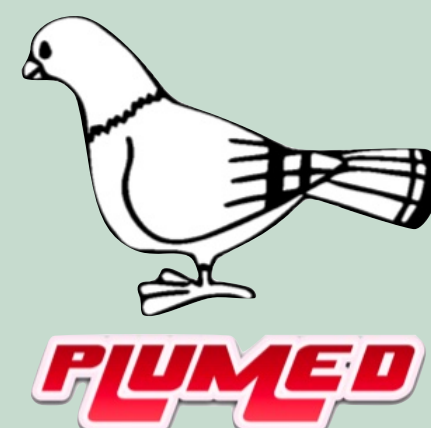
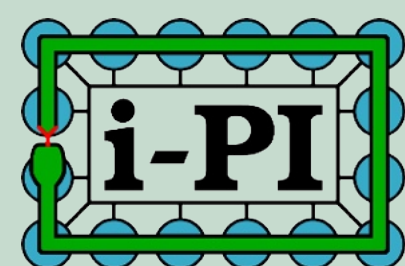


torch-pme

Q-Stack



metatensor



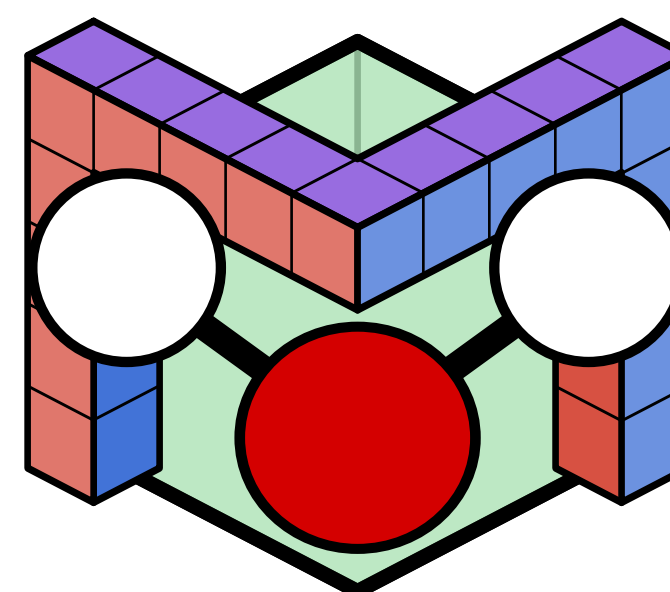
Torch-sim

ShiftML

PET-MAD

mts-operations

mts-learn



metatomic

SOAP-BPNN

LLPR

PET

GAP

NanoPET

SALTED

featomic

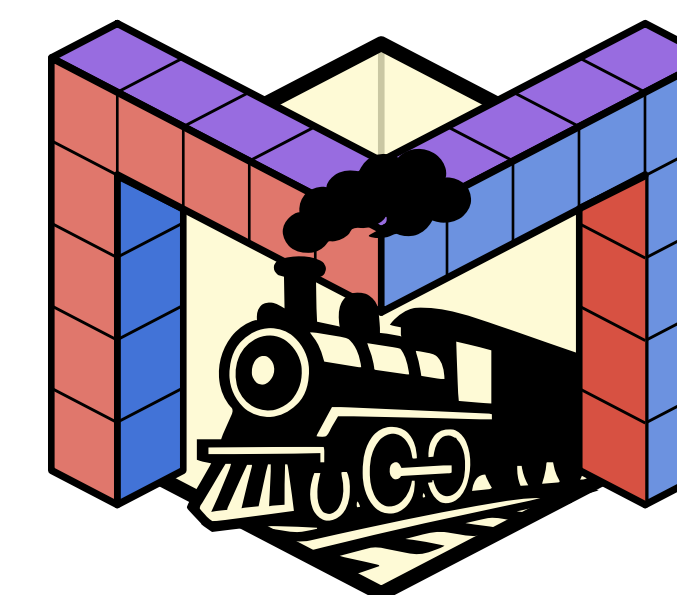
AniSOAP

vesin

spex

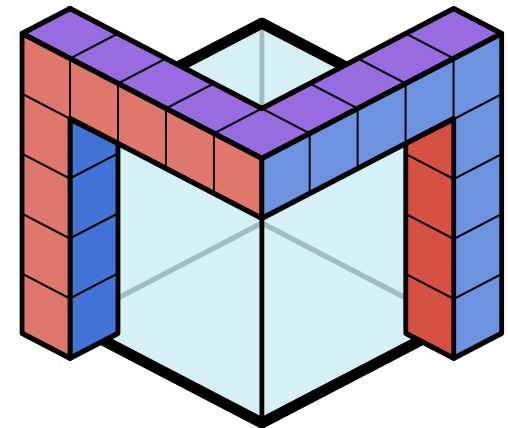


Q-Stack



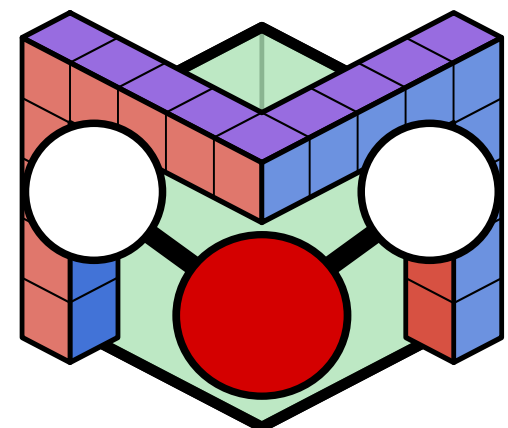
metatrain

Different packages for different users



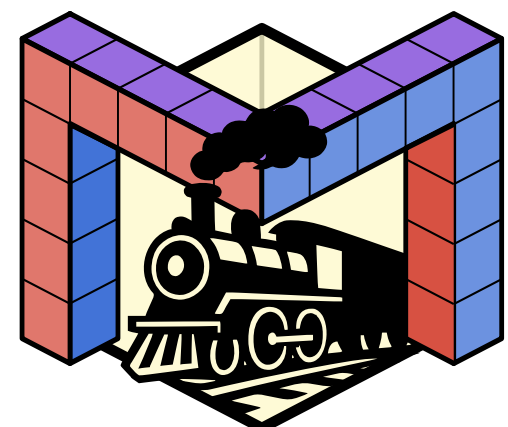
metatensor

Write low-level code, define new custom tools for machine learning



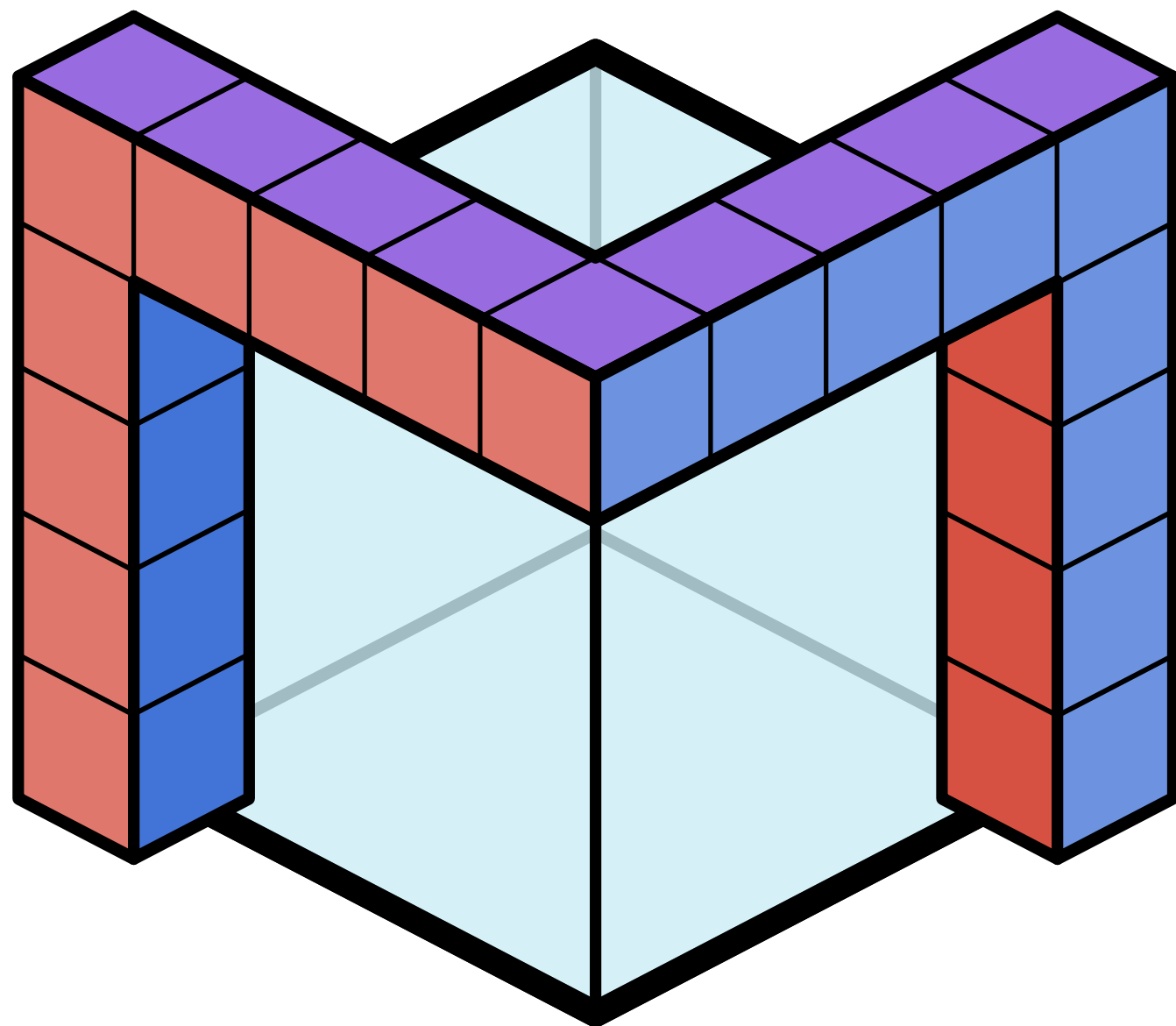
metatomic

Define new ML architectures, new tools specifically for atomistic simulations



metatrain

Train ML models without writing any code, access all advanced features from metatensor and metatomic



Metatensor: sharing data

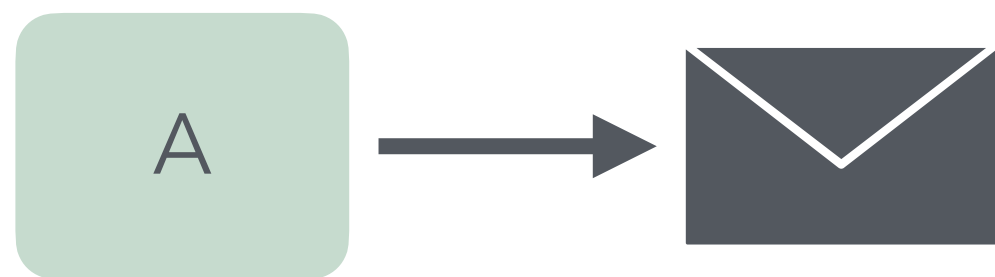
What are we trying to do & why?

Exchange data across libraries without them knowing about each other

What are we trying to do & why?

Exchange data across libraries without them knowing about each other

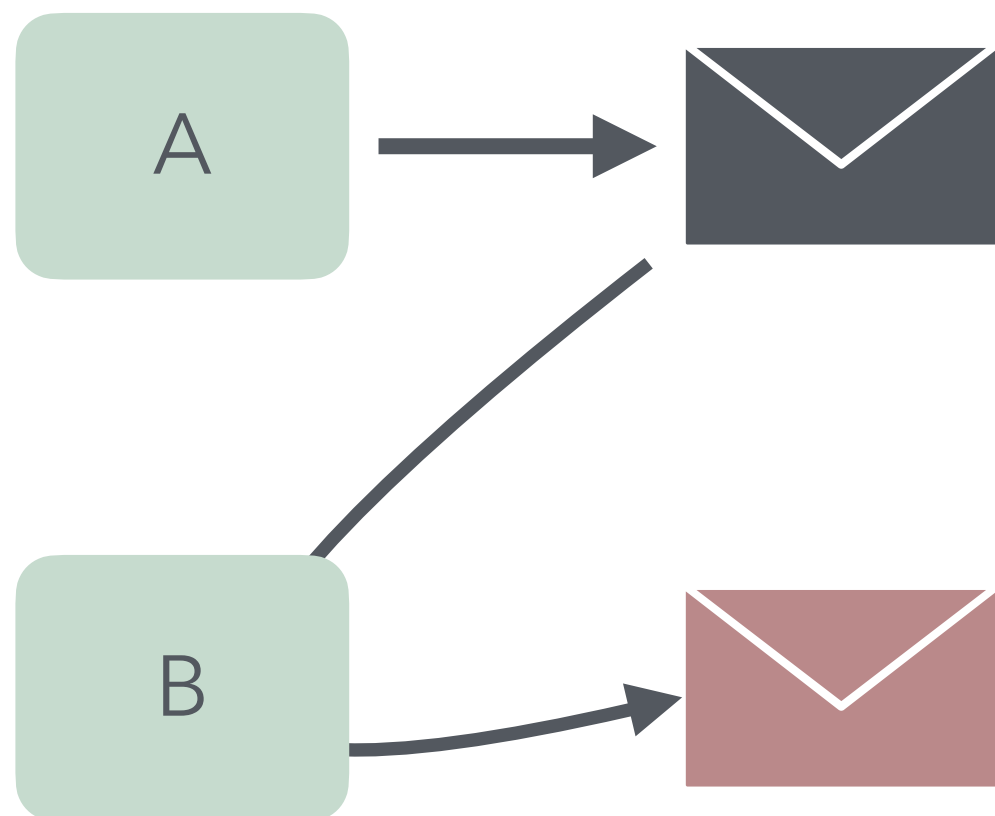
- Store **metadata** together with the data, making the container **self-describing**



What are we trying to do & why?

Exchange data across libraries without them knowing about each other

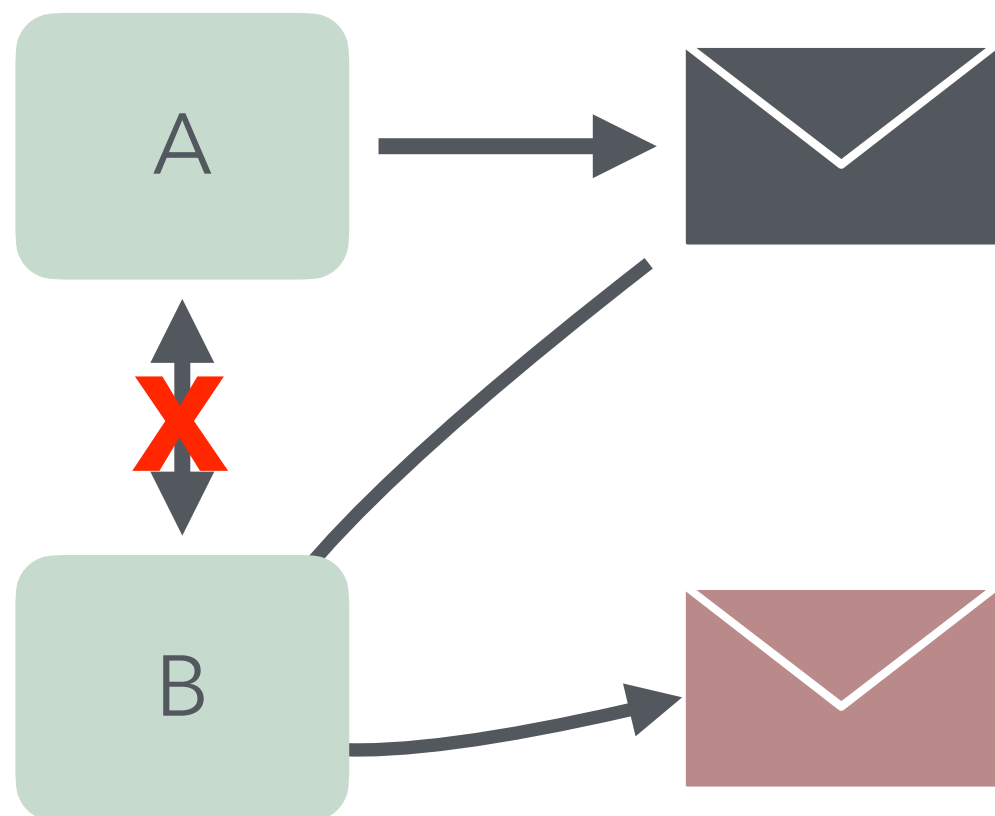
- Store **metadata** together with the data, making the container **self-describing**



What are we trying to do & why?

Exchange data across libraries without them knowing about each other

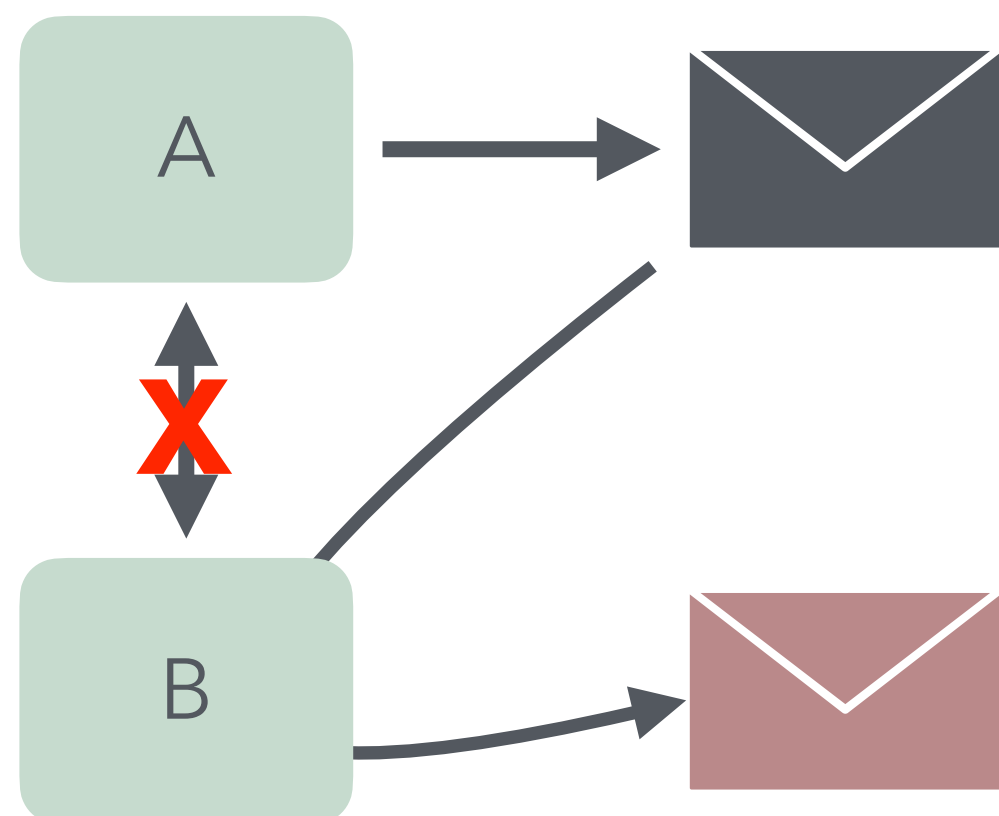
- Store **metadata** together with the data, making the container **self-describing**



What are we trying to do & why?

Exchange data across libraries without them knowing about each other

- Store **metadata** together with the data, making the container **self-describing**
- Handle **gradients** together with the data (ML representations and physical properties)

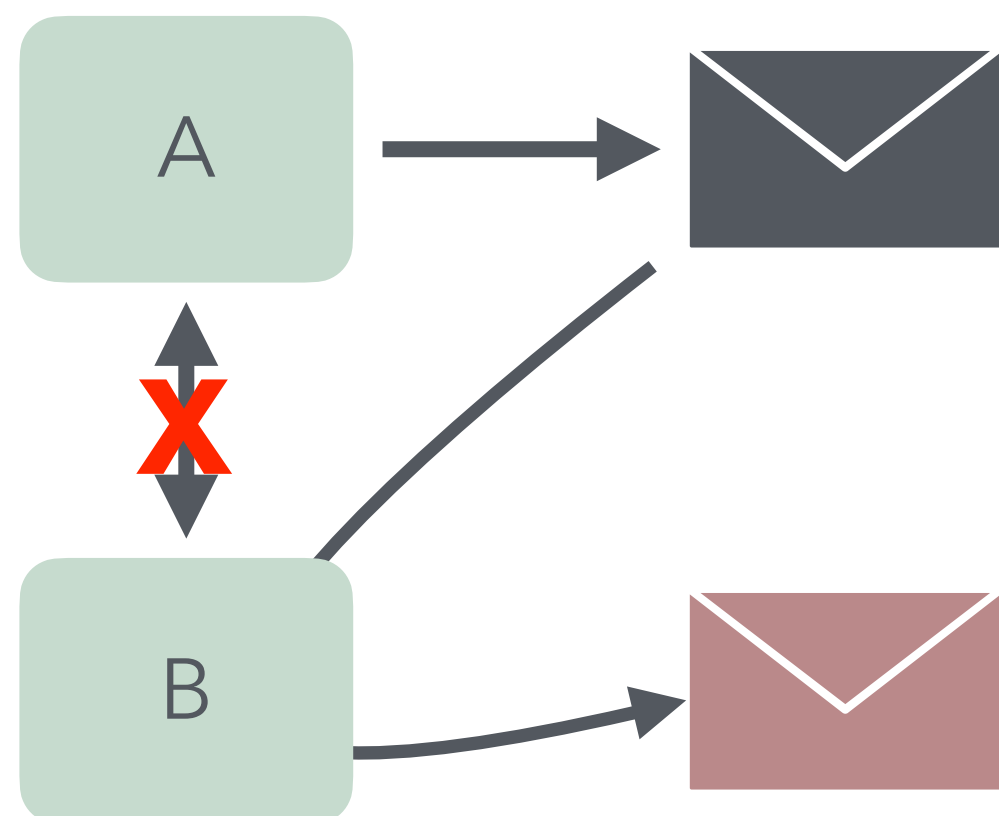


$$\vec{f}_i = - \vec{\nabla}_{R_i} E$$
$$\sigma_{ij} = - \nabla_{H_{ij}} E$$

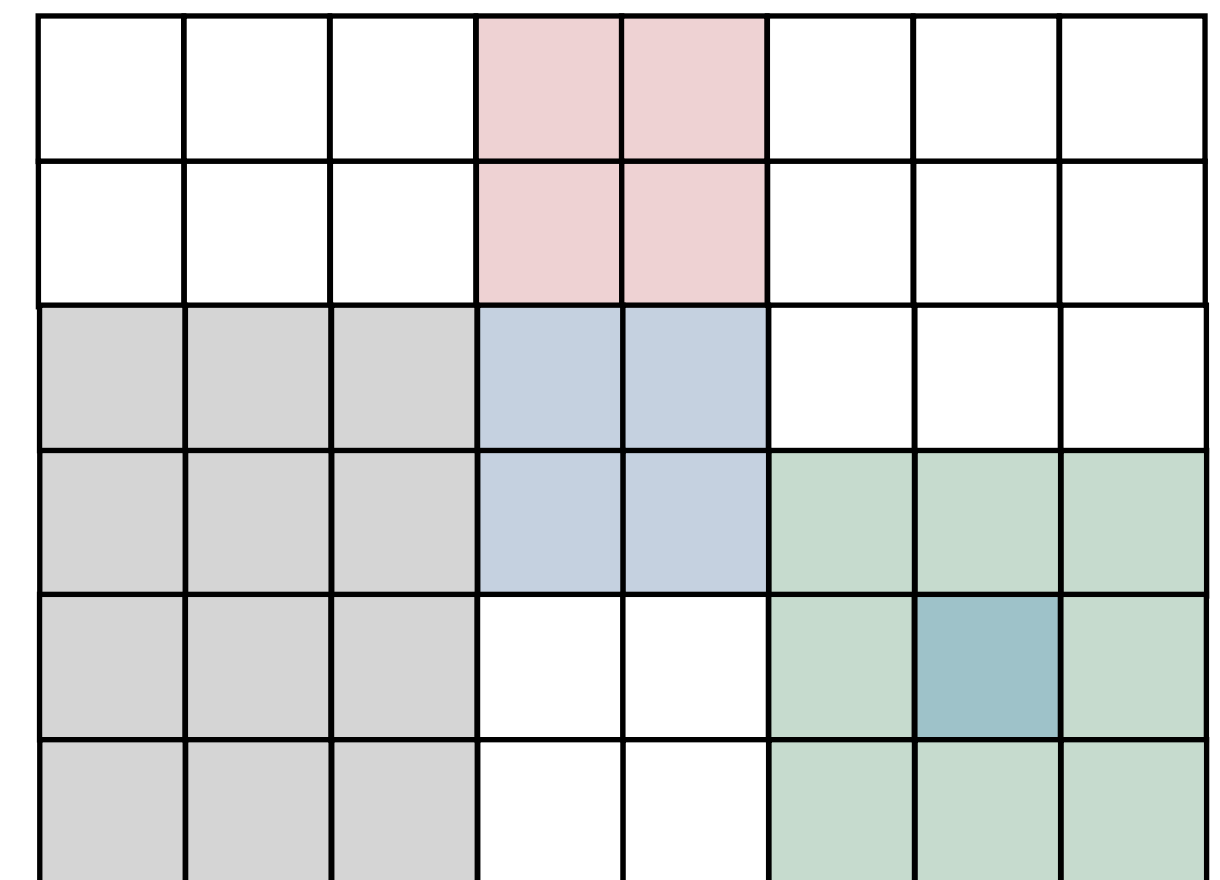
What are we trying to do & why?

Exchange data across libraries without them knowing about each other

- Store **metadata** together with the data, making the container **self-describing**
- Handle **gradients** together with the data (ML representations and physical properties)
- Exploit natural data **sparsity** to reduce memory usage



$$\begin{aligned} E \\ \vec{f}_i &= - \vec{\nabla}_{R_i} E \\ \sigma_{ij} &= - \nabla_{H_{ij}} E \end{aligned}$$



The data format

Data and Metadata

Gradients

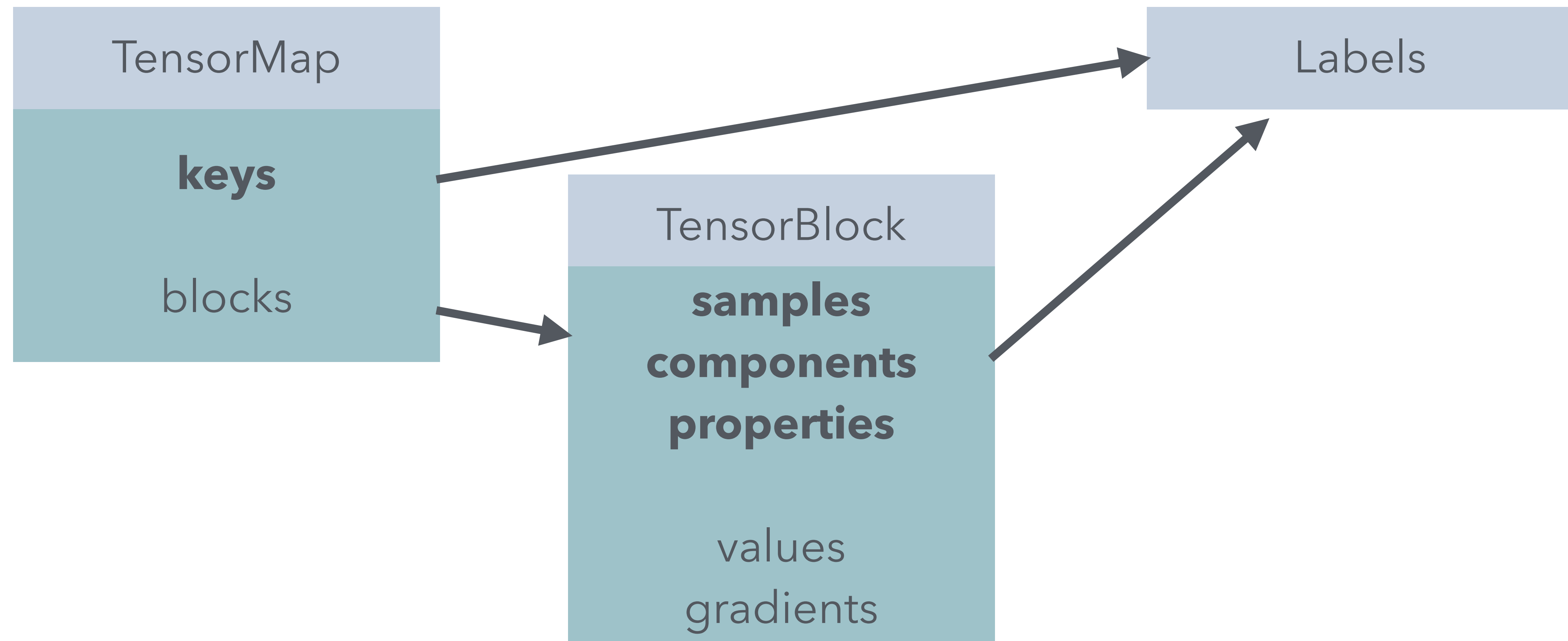
Sparsity

The data format

Data and Metadata

Gradients

Sparsity

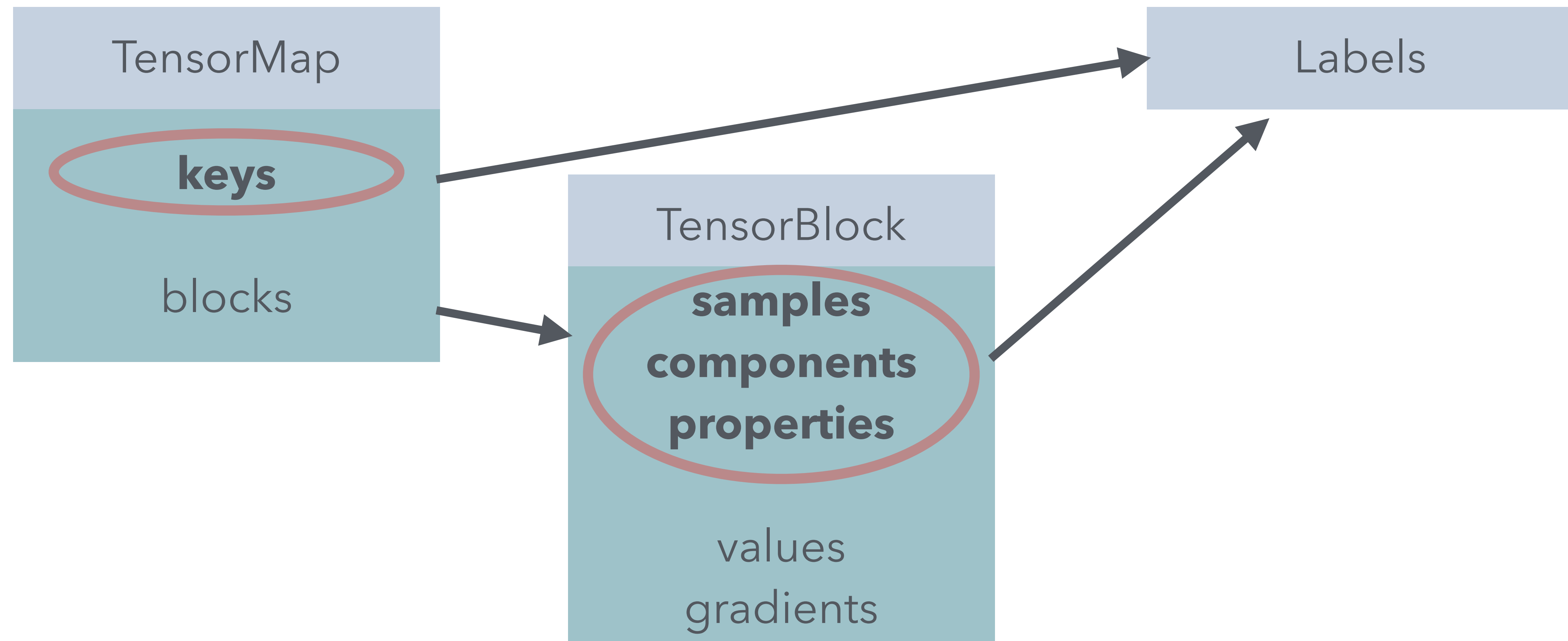


The data format

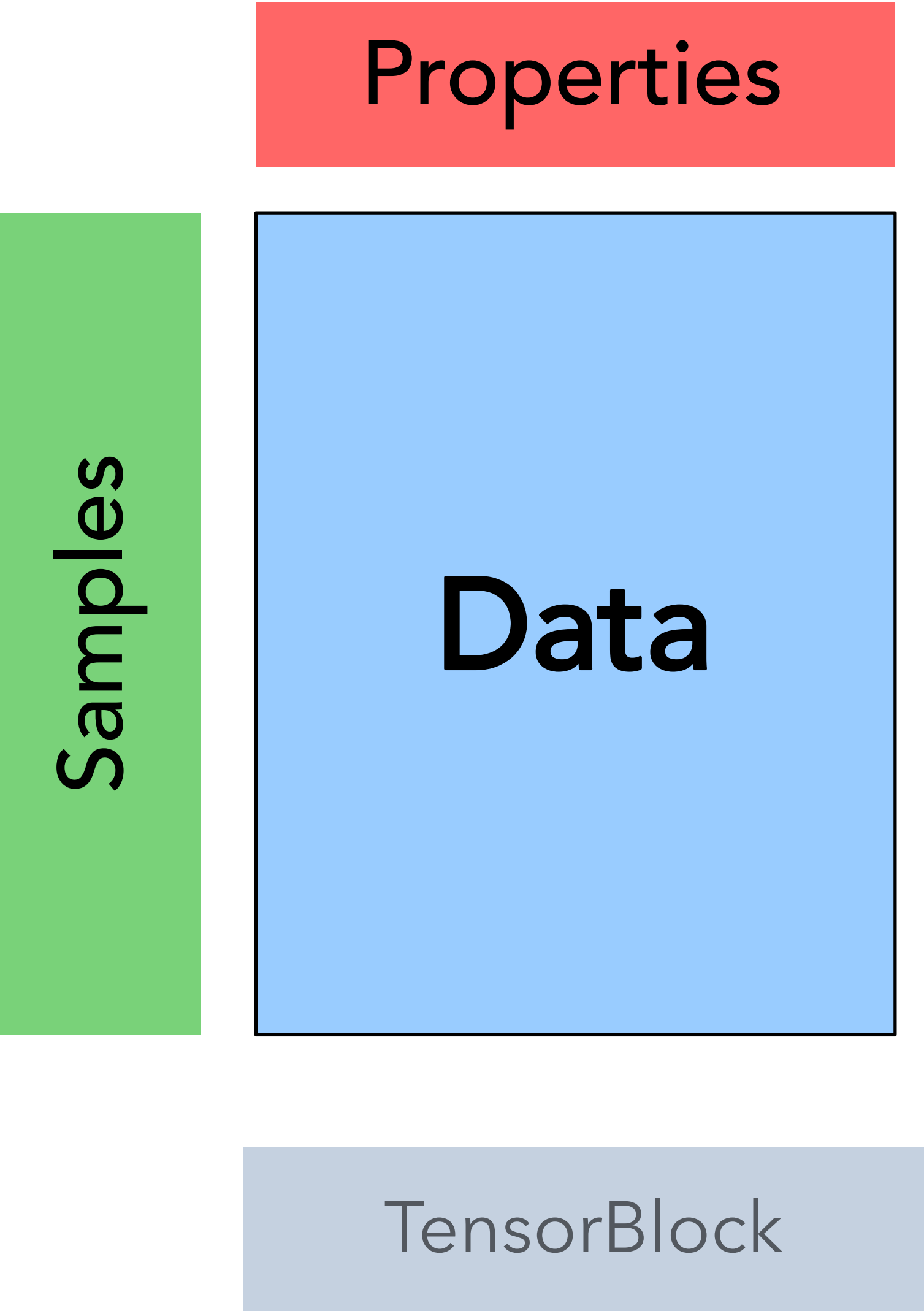
Data and Metadata

Gradients

Sparsity



Blocks and labels



system	atom
0	0
0	1
0	2
1	0
1	1
1	2
1	3
2	0
2	1
2	2

type_1	n_1	type_2	n_2
1	0	1	0
1	1	1	1
6	0	6	0
6	1	6	3
1	2	1	1
1	3	1	2
6	2	6	2
6	3	6	3

Labels

Metadata and sparsity

system	atom
0	0
0	1
0	2
1	0
1	1
1	2
1	3
2	0
2	1
2	2

```
block.values[0, :] == block[system=0, atom=0, :]
```

```
block.values[5, :] == block[system=1, atom=2, :]
```

Metadata and sparsity

system	atom
0	0
0	1
0	2
1	0
1	1
1	2
1	3
2	0
2	1
2	2

`block.values[0, :] == block[system=0, atom=0, :]`

`block.values[5, :] == block[system=1, atom=2, :]`

`block.values[:, 3] == block[:, type_1=6, n_1=1, type_2=6, n_2=3]`

type_1	n_1	type_2	n_2
1	0	1	0
1	1	1	1
6	0	6	0
6	1	6	3
1	2	1	1
1	3	1	2
6	2	6	2
6	3	6	3

Metadata and sparsity

system	atom
0	0
0	1
0	2
1	0
1	1
1	2
1	3
2	0
2	1
2	2

```
block.values[0, :] == block[system=0, atom=0, :]
```

```
block.values[5, :] == block[system=1, atom=2, :]
```

```
block.values[:, 3] == block[:, type_1=6, n_1=1, type_2=6, n_2=3]
```

```
block[system=1, atom=0, :] = xx.yy
```

```
block[system=0, atom=1, :] = 0
```

```
block[system=2, atom=0, :] = 0
```

```
block[:, type_1=8, ...] = 0
```

type_1	n_1	type_2	n_2
1	0	1	0
1	1	1	1
6	0	6	0
6	1	6	3
1	2	1	1
1	3	1	2
6	2	6	2
6	3	6	3

system	atom
0	0
0	2
1	0
1	1
2	1
2	2

Metadata and sparsity

system	atom
0	0
0	1
0	2
1	0
1	1
1	2
1	3
2	0
2	1
2	2

```
block.values[0, :] == block[system=0, atom=0, :]
```

```
block.values[5, :] == block[system=1, atom=2, :]
```

```
block.values[:, 3] == block[:, type_1=6, n_1=1, type_2=6, n_2=3]
```

```
block[system=1, atom=0, :] = xx.yy
```

```
block[system=0, atom=1, :] = 0
```

```
block[system=2, atom=0, :] = 0
```

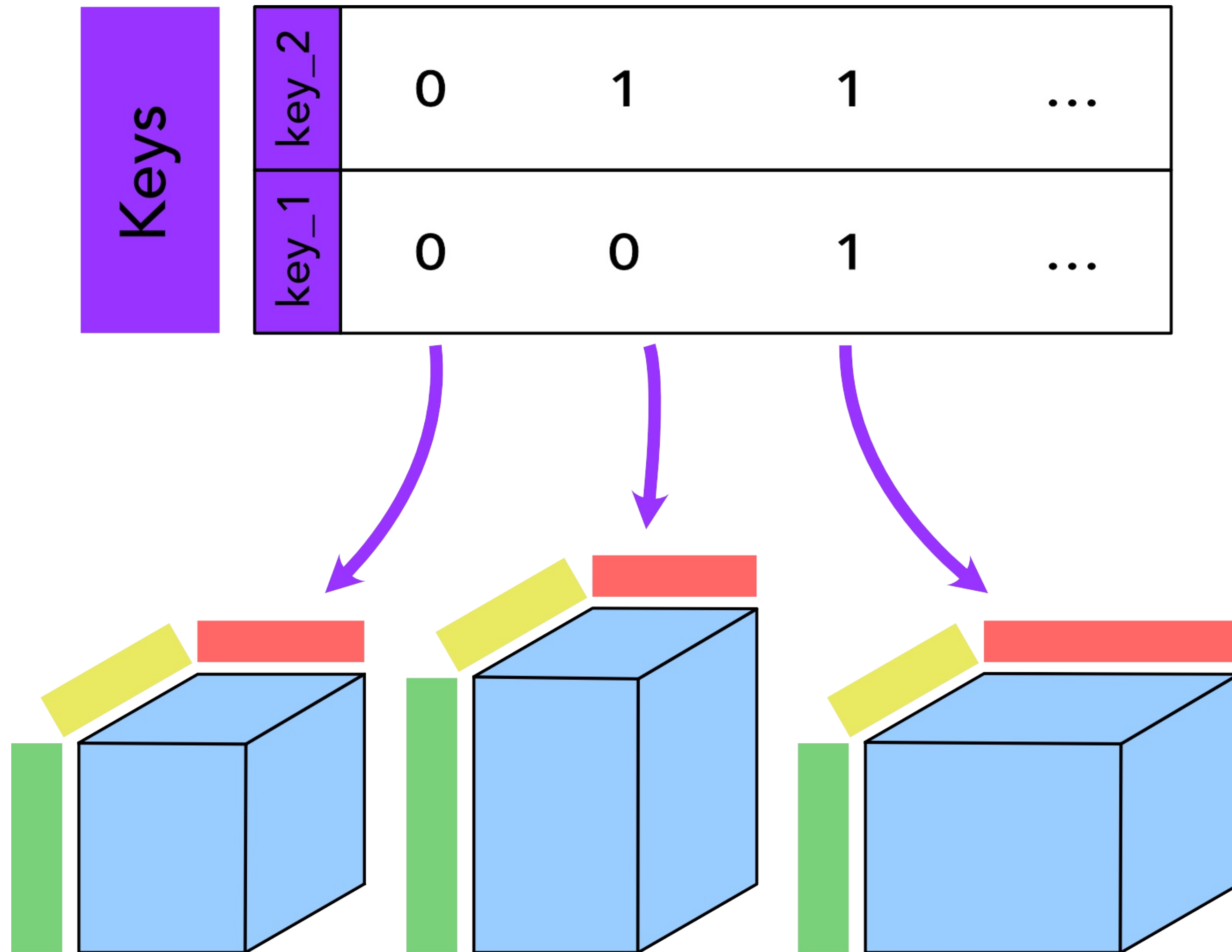
```
block[:, type_1=8, ...] = 0
```

type_1	n_1	type_2	n_2
1	0	1	0
1	1	1	1
6	0	6	0
6	1	6	3
1	2	1	1
1	3	1	2
6	2	6	2
6	3	6	3

system	atom
0	0
0	2
1	0
1	1
2	1
2	2

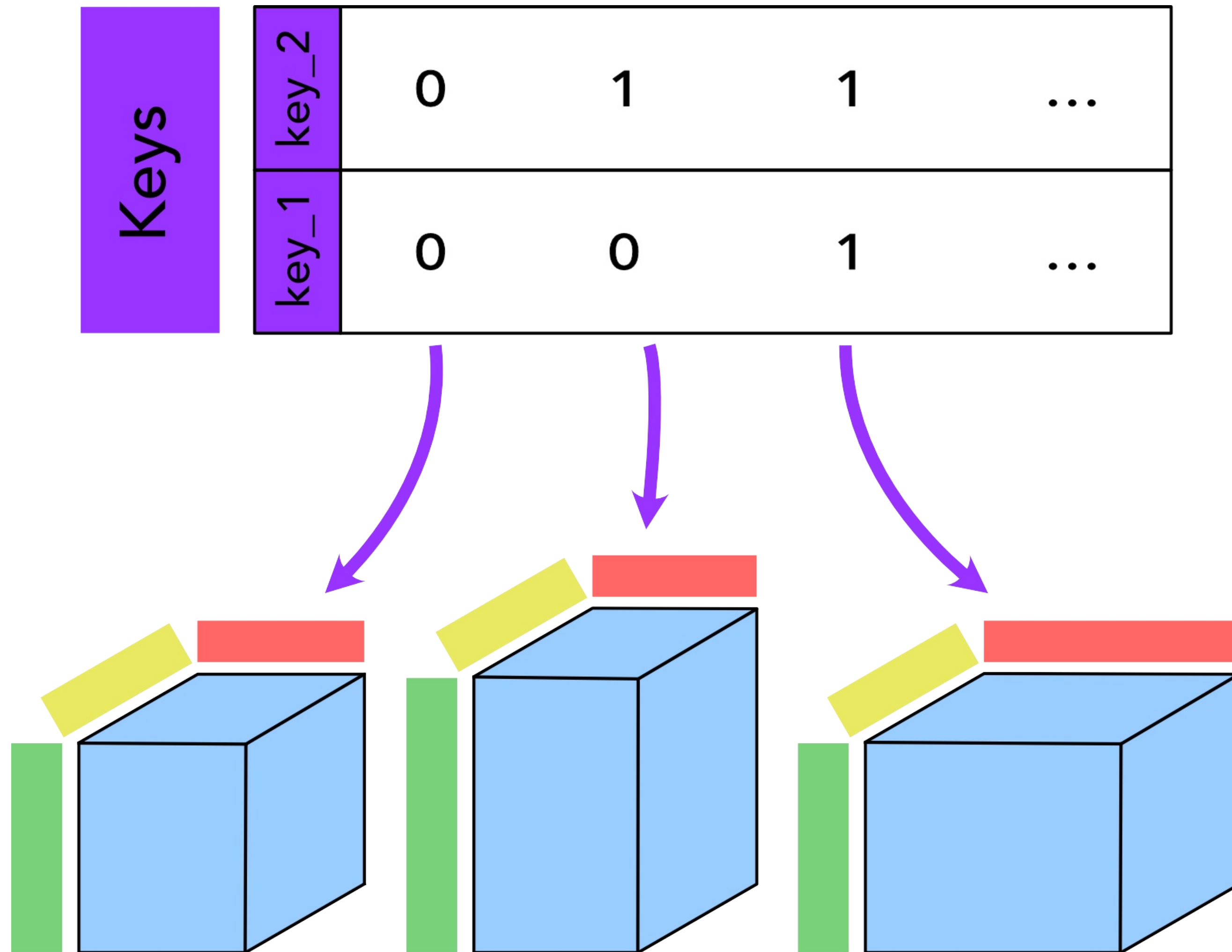
Missing entries are equal to zero

TensorMap



TensorMap

TensorMap

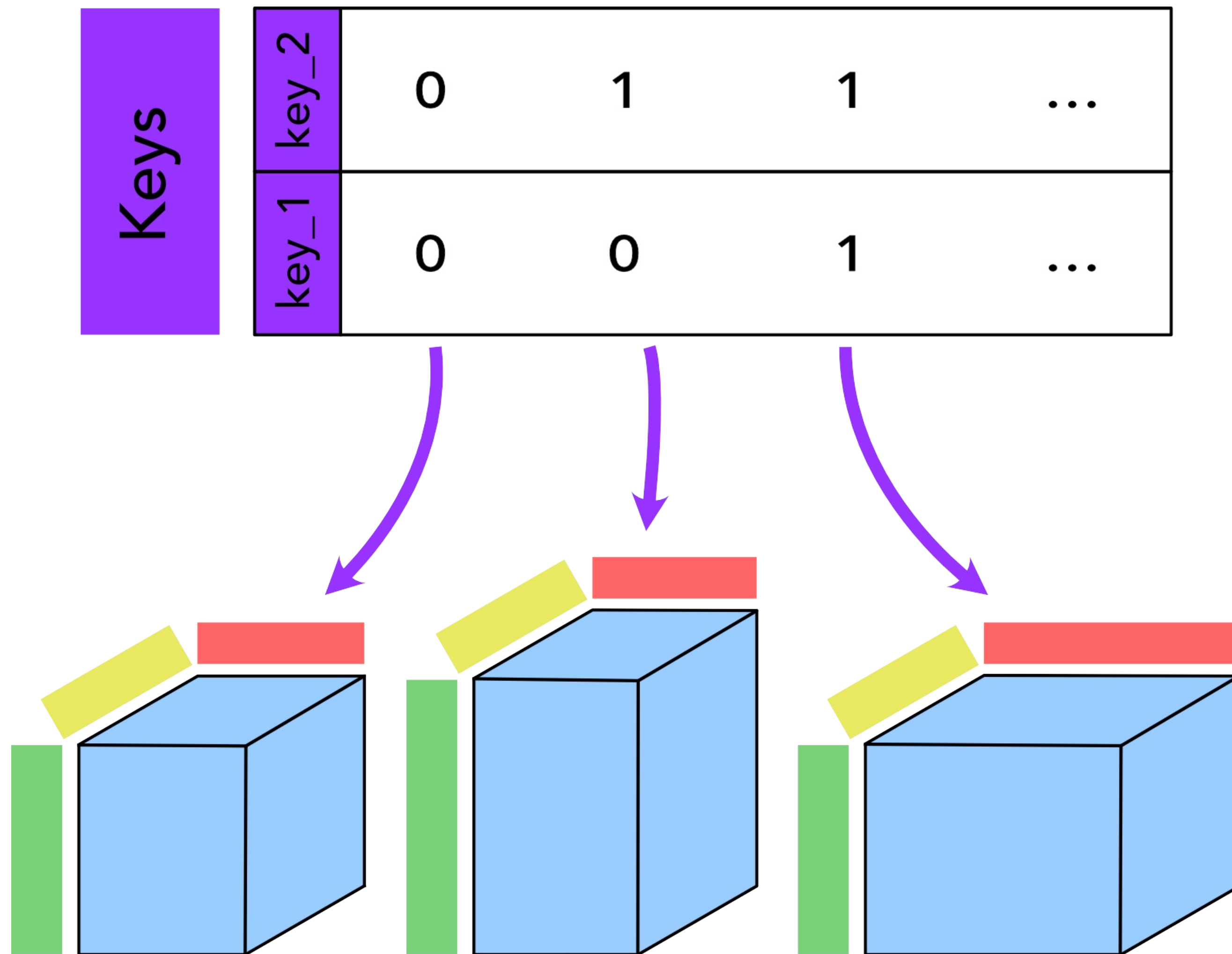


Missing keys/block for block-sparse structure

$\text{tensor}[\text{key_1}=2, \text{key_2}=0, \dots] = 0$

TensorMap

TensorMap



Use separate keys/blocks when different blocks have different data shape, e.g. angular momentum

$\text{tensor}[L=2, :, M=-4, :] = \emptyset$

Missing keys/block for block-sparse structure

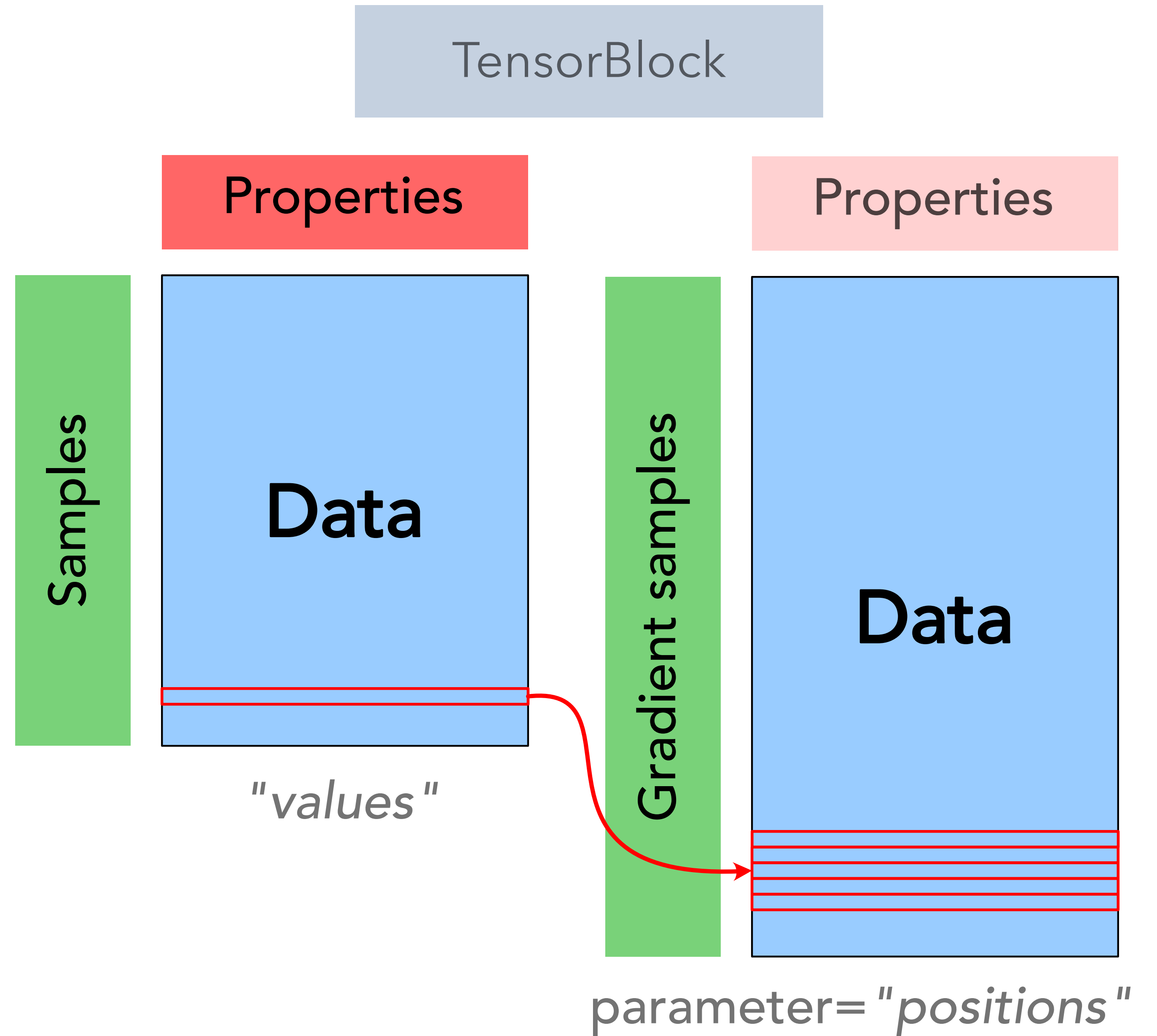
$\text{tensor}[\text{key}_1=2, \text{key}_2=0, \dots] = 0$

TensorMap

Gradients

Each **block** can have (multiple) "gradient" blocks attached to them, including gradient of gradients.

Special gradient samples structure to know which row is the gradient of **what**, with respect to **what**.

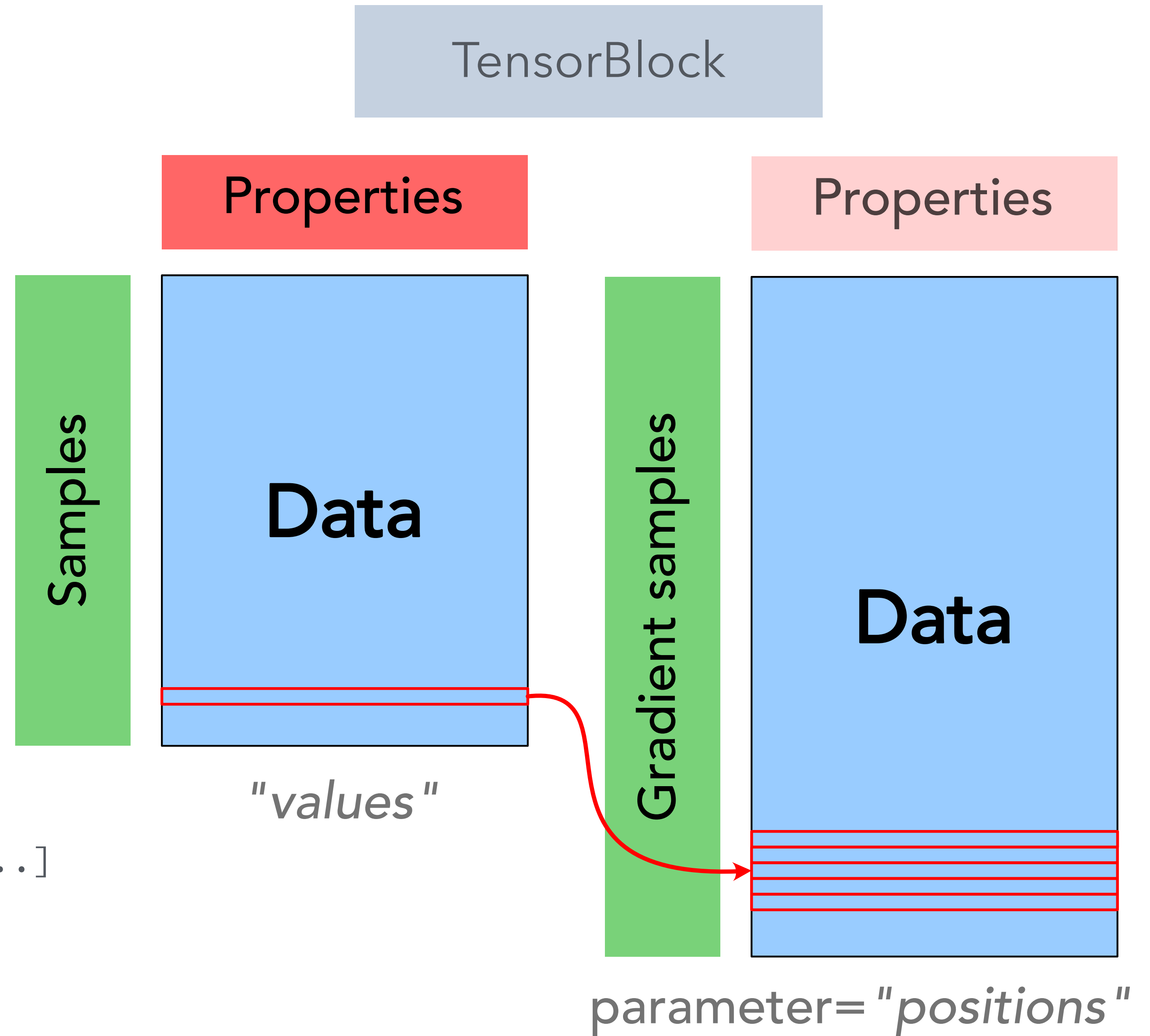


Gradients

Each **block** can have (multiple) "gradient" blocks attached to them, including gradient of gradients.

Special gradient samples structure to know which row is the gradient of **what**, with respect to **what**.

```
gradient[sample=33, system=0, atom=3, ...]
```



Gradients

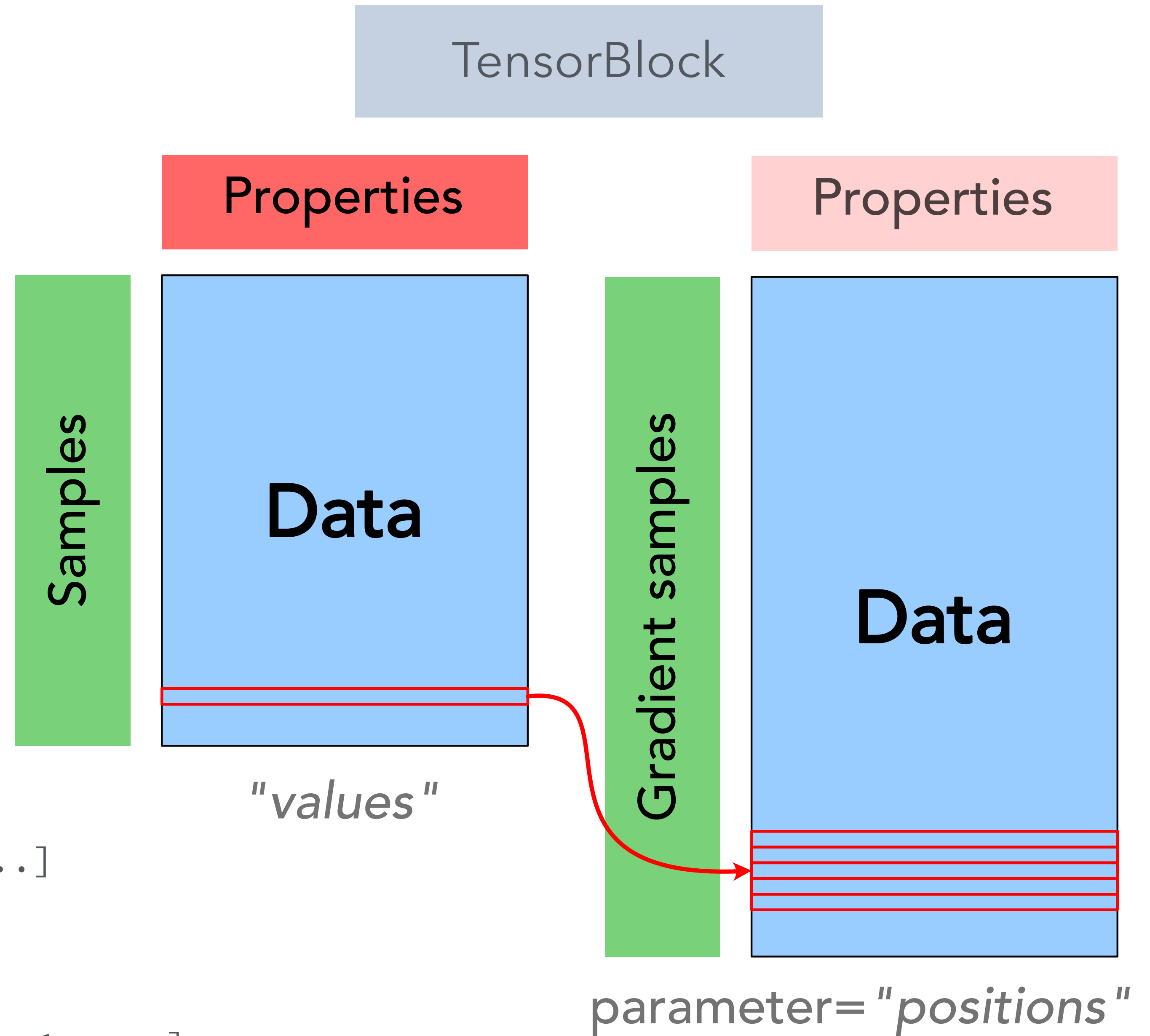
Each **block** can have (multiple) "gradient" blocks attached to them, including gradient of gradients.

Special gradient samples structure to know which row is the gradient of **what**, with respect to **what**.

```
gradient[sample=33, system=0, atom=3, ...]
```



```
block[33] = block[system=0, atom=1, ...]
```



Gradients

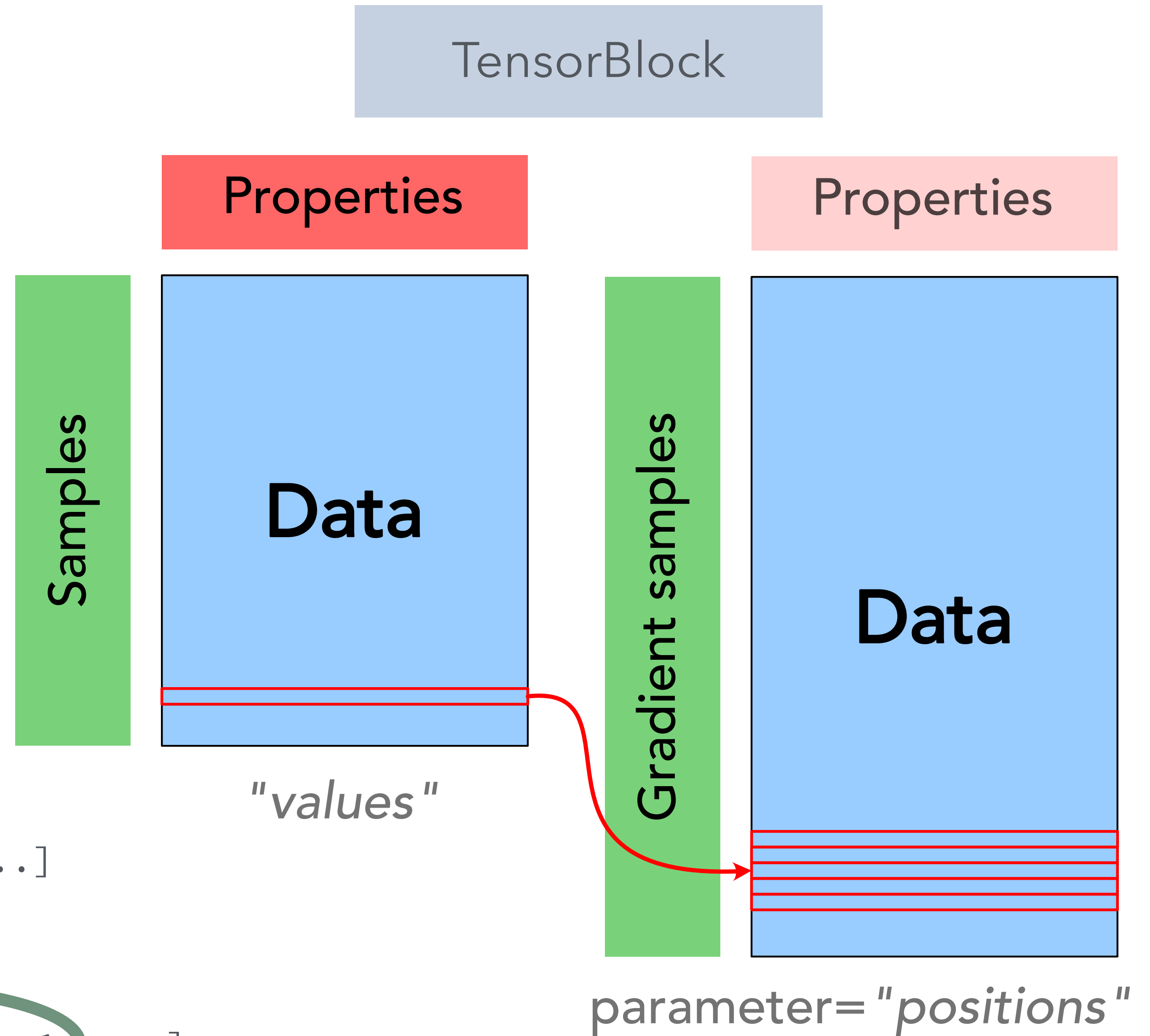
Each **block** can have (multiple) "gradient" blocks attached to them, including gradient of gradients.

Special gradient samples structure to know which row is the gradient of **what**, with respect to **what**.

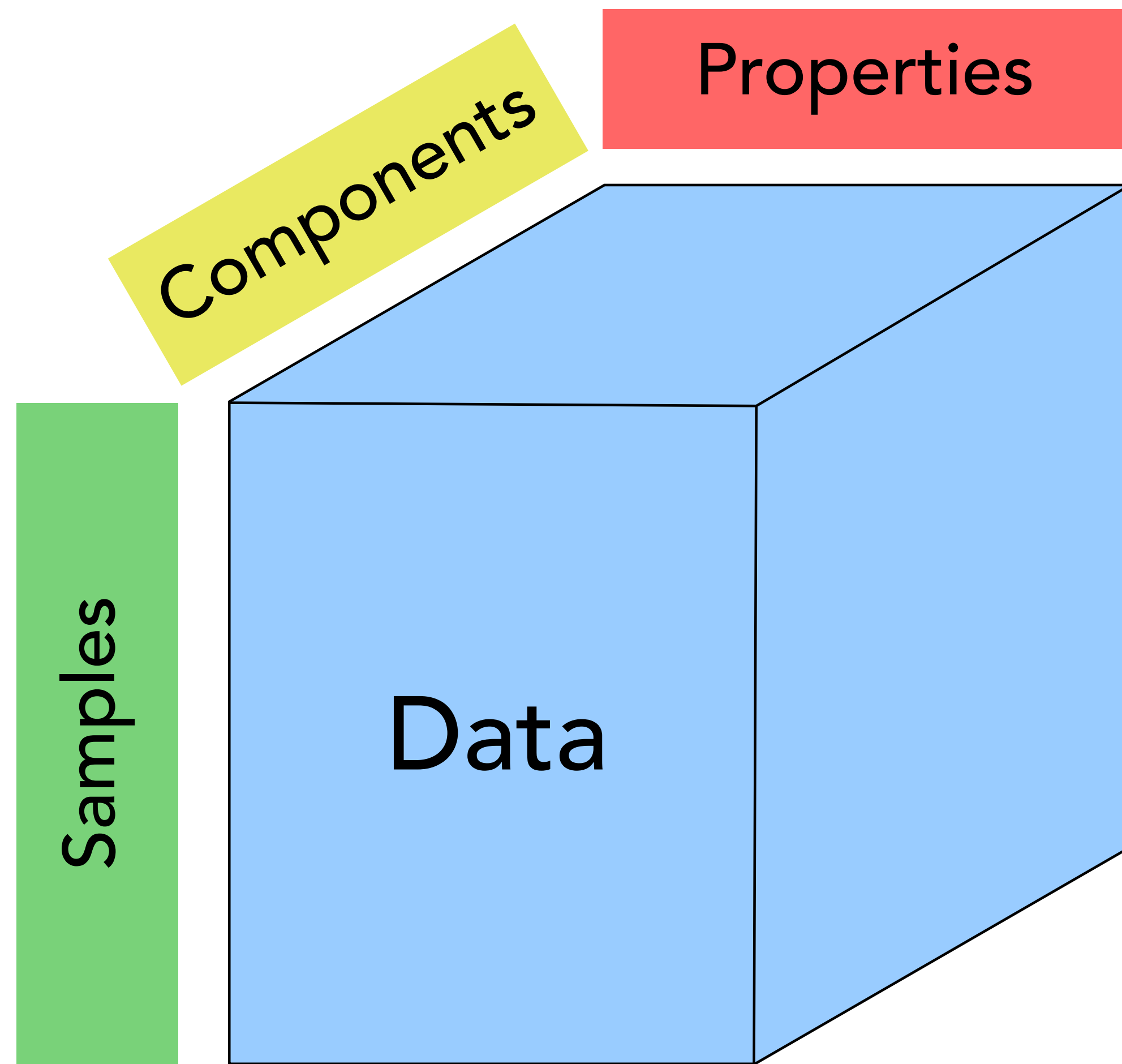
`gradient[sample=33, system=0, atom=3, ...]`



`block[33] = block[system=0, atom=1, ...]`



Tensorial data



When the data is a bunch of tensors, we don't want the ability to have *missing* values.

These dimensions of the data are stored in **components** labels.

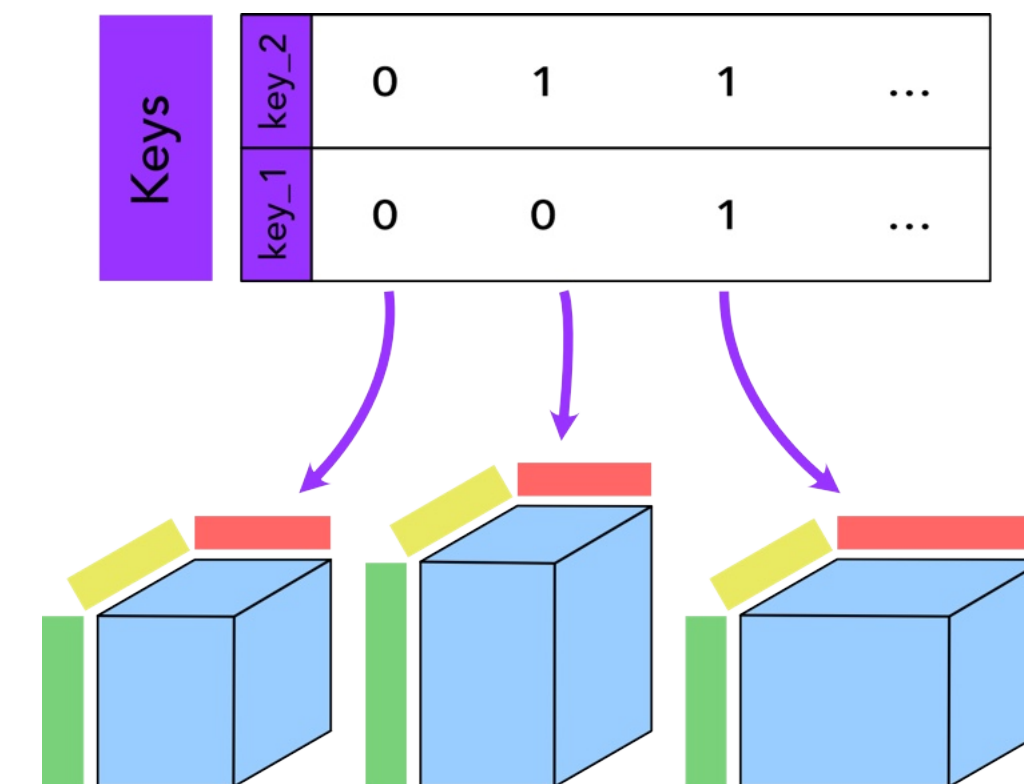
energy: no components

dipole: xyz component

Young modulus: [xyz_1, xyz_2] components

Components can also be use for spherical tensors: $\mu \in [-\lambda, \lambda]$

The whole thing



system	atom
0	0
0	1
0	2
1	0
1	1

type_1	n_1	type_2	n_2
1	0	1	0
1	1	1	1
6	0	6	0
6	1	6	3
1	2	1	1
1	3	1	2

tensor[center=8, lambda=1, system=1, atom=3, mu=-1, type_1=1, n_2=5]

keys

samples

components

properties

High-dimensional, self-describing sparse tensor

A couple more examples

energy[system, atom, energy]

"positions" gradient[sample, system, atom, xyz, energy]

"strain" gradient[sample, xyz_1, xyz_2, energy]

A couple more examples

energy[system, atom, energy]

"positions" gradient[sample, system, atom, xyz, energy]

"strain" gradient[sample, xyz_1, xyz_2, energy]

soap[center_type, neigh_1_type, neigh_2_type, system, atom, n1, n2]

A couple more examples

energy[system, atom, energy]

"positions" gradient[sample, system, atom, xyz, energy]

"strain" gradient[sample, xyz_1, xyz_2, energy]

soap[center_type, neigh_1_type, neigh_2_type, system, atom, n1, n2]

electron_density[lambda, sigma, atom_type, system, atom, mu, n]

A couple more examples

energy[system, atom, energy]

"positions" gradient[sample, system, atom, xyz, energy]

"strain" gradient[sample, xyz_1, xyz_2, energy]

soap[center_type, neigh_1_type, neigh_2_type, system, atom, n1, n2]

electron_density[lambda, sigma, atom_type, system, atom, mu, n]

random_thing[eta, chi, kappa, m, n, xyz, abc, i, j, k]

Sharing data across ecosystems

- People work with different programming languages and ecosystems



PyTorch



NumPy



julia

- They need to be able to share data with each other!

Sharing data across ecosystems

- People work with different programming languages and ecosystems



PyTorch



NumPy



julia

- They need to be able to share data with each other!

Python

C++

TorchScript

Rust

Python

C++

TorchScript

Rust

Sharing data across ecosystems

- People work with different programming languages and ecosystems



PyTorch

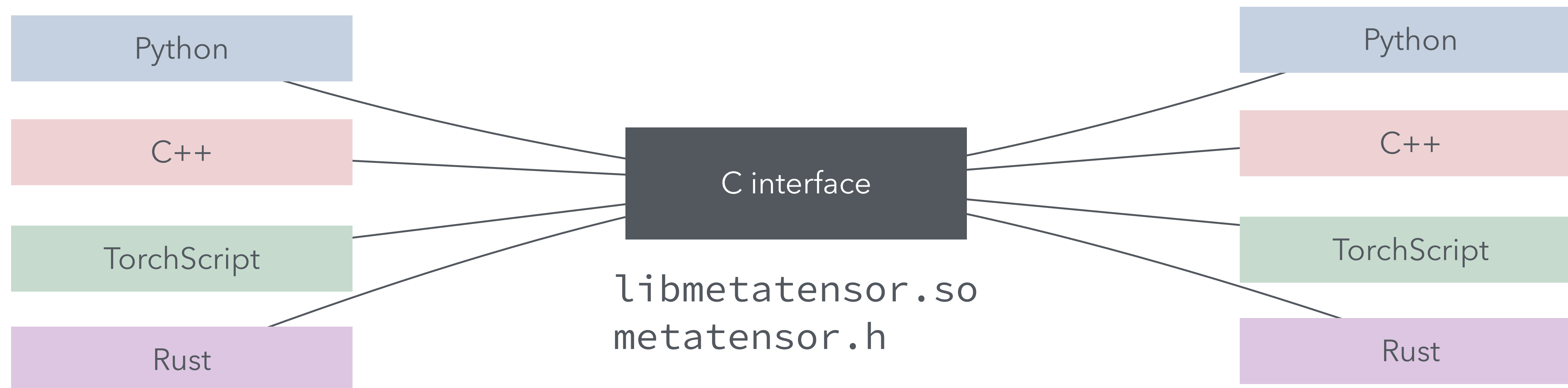


NumPy



julia

- They need to be able to share data with each other!



Sharing data across ecosystems

- People work with different programming languages and ecosystems



PyTorch

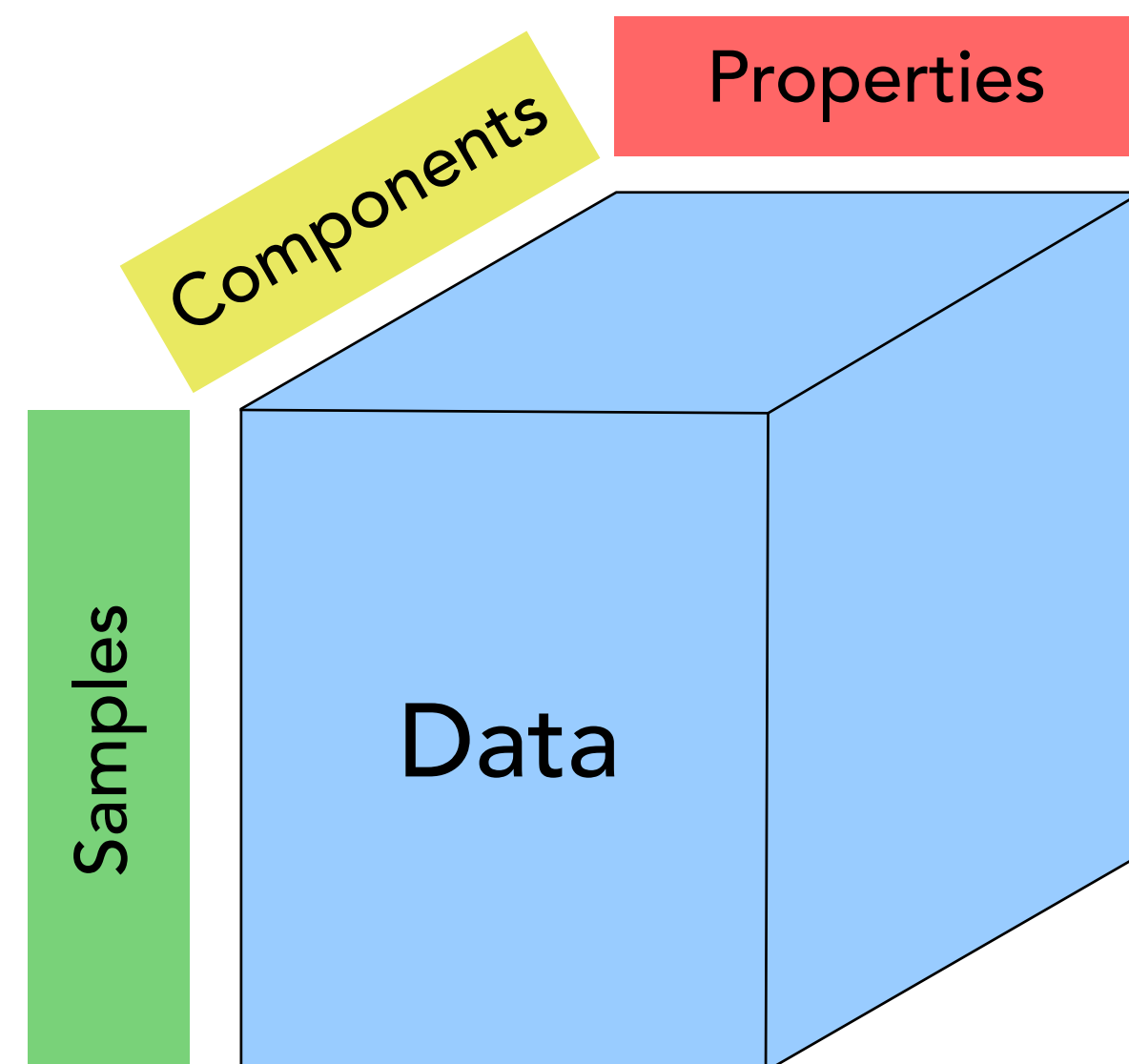


NumPy



julia

- They need to be able to share data with each other!
- Generic over how data is stored: CPU, GPU, Python, ...



Labels (samples/components/properties) are owned and manipulated by the C library

Data storage is provided by the user and opaque to metatensor

Working with metatensor in practice



Only the data structures, with full control over the data arrays



PyTorch

Data structures and manipulation functions

Working with metatensor in practice



Only the data structures, with full control over the data arrays



PyTorch

Data structures and manipulation functions

metatensor-operations

```
import metatensor as mts

desc = mts.join([soap, lode], axis="properties")
batch = mts.Labels("structure", [0, 2, 3, 4])

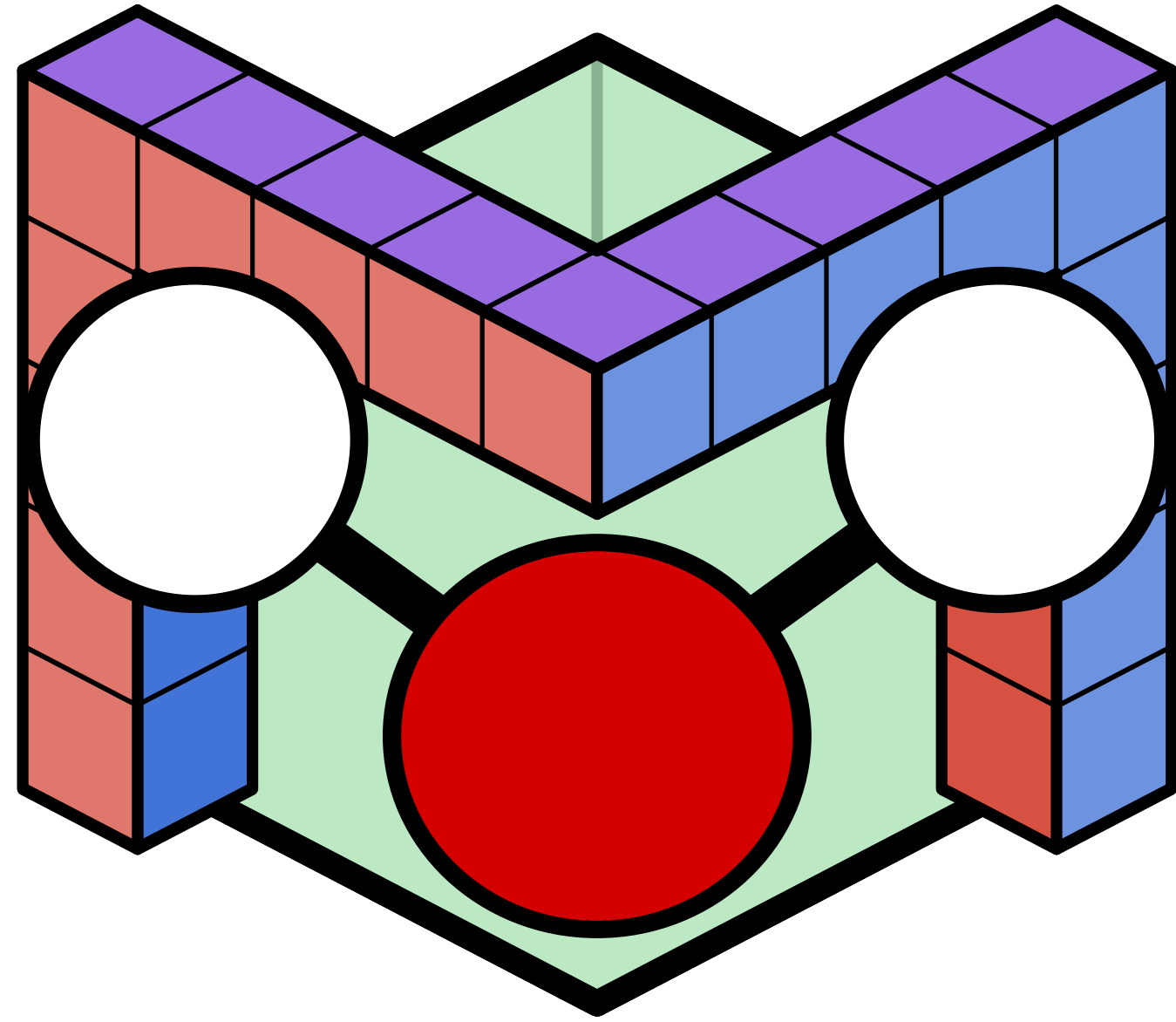
desc = mts.slice(desc, axis="samples", labels=batch)
```

metatensor-learn

```
import metatensor as mts
from metatensor.learn import nn as mts_nn
from metatensor.learn.data import Dataset

desc = mts.split(mts.load("saved.mts"))
dataset = Dataset(systems=[...], desc=desc)

class CustomModel(mts_nn.Module):
    def __init__(self):
        self.linear = mts_nn.Linear(...)
```



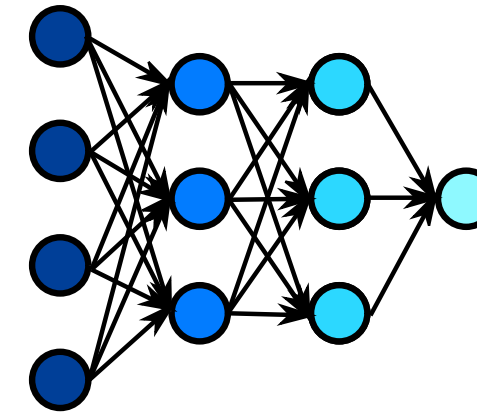
Metatonic: sharing models

What are we trying to do?

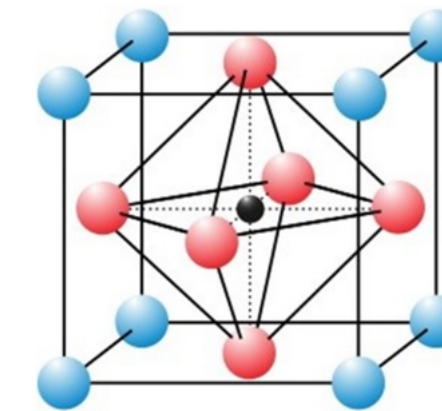
- Share atomistic machine learning models between developers and users



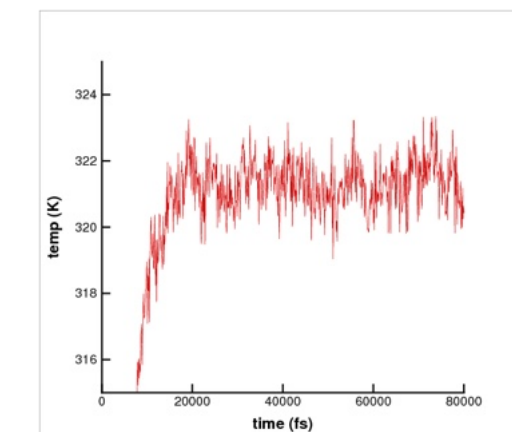
"I want to create new ML models"



"I want to use ML for research on my system"



"I want to use ML together with my new fancy MD simulation tool"



Single tool to exchange models

What are we trying to do?

- Share atomistic machine learning models between developers and users
- Reduce the amount of work to interface ML models with simulation engines

GAP

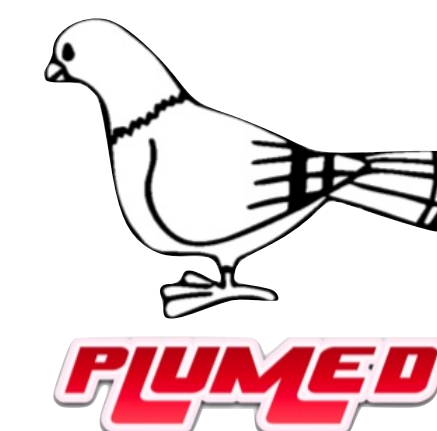
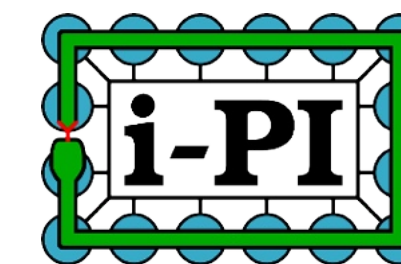
PET

ANI

MACE

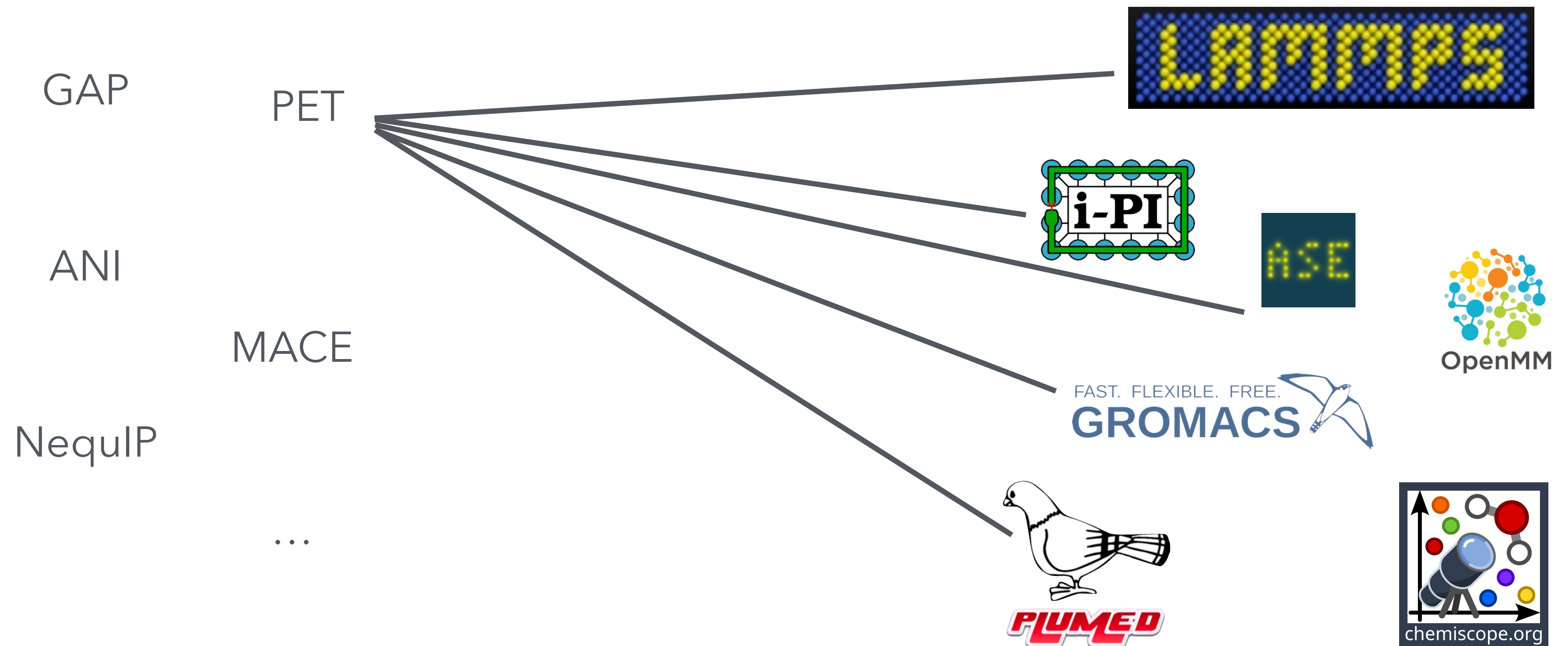
NequIP

...



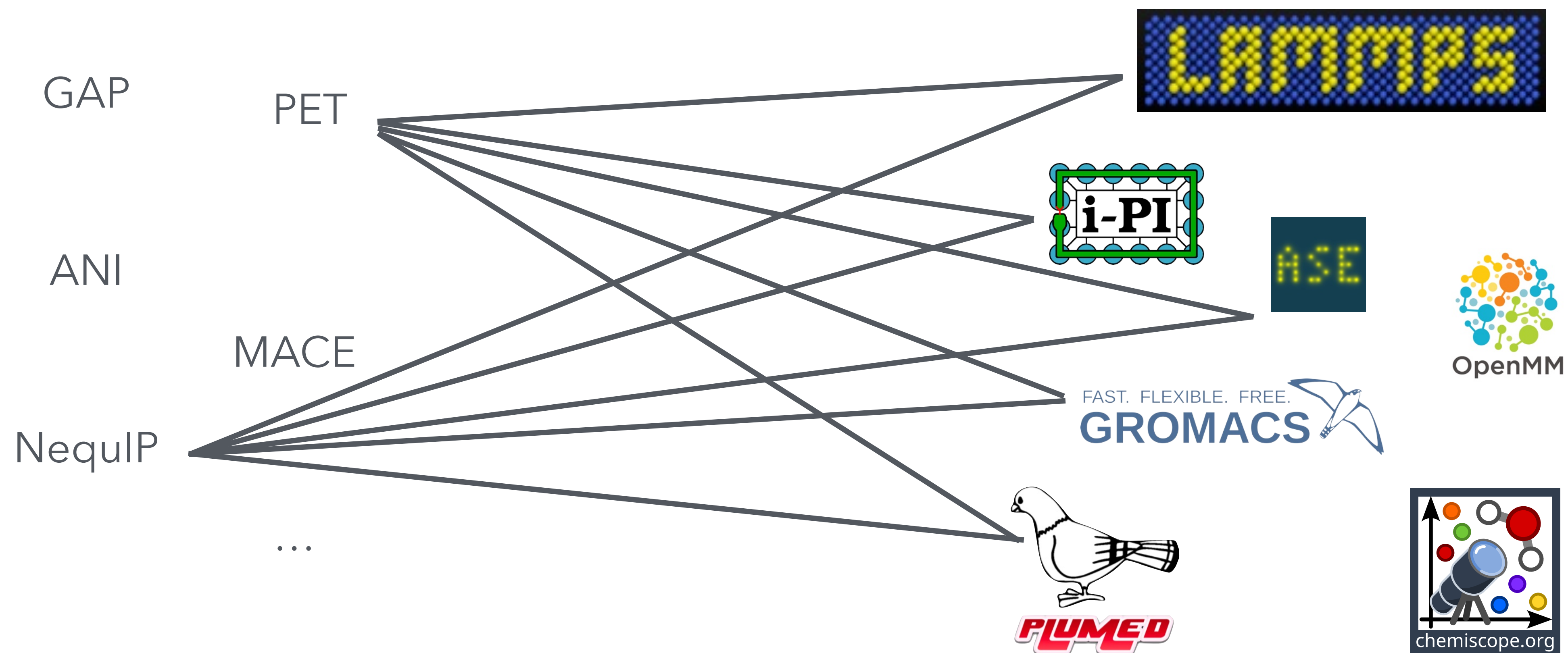
What are we trying to do?

- Share atomistic machine learning models between developers and users
- Reduce the amount of work to interface ML models with simulation engines



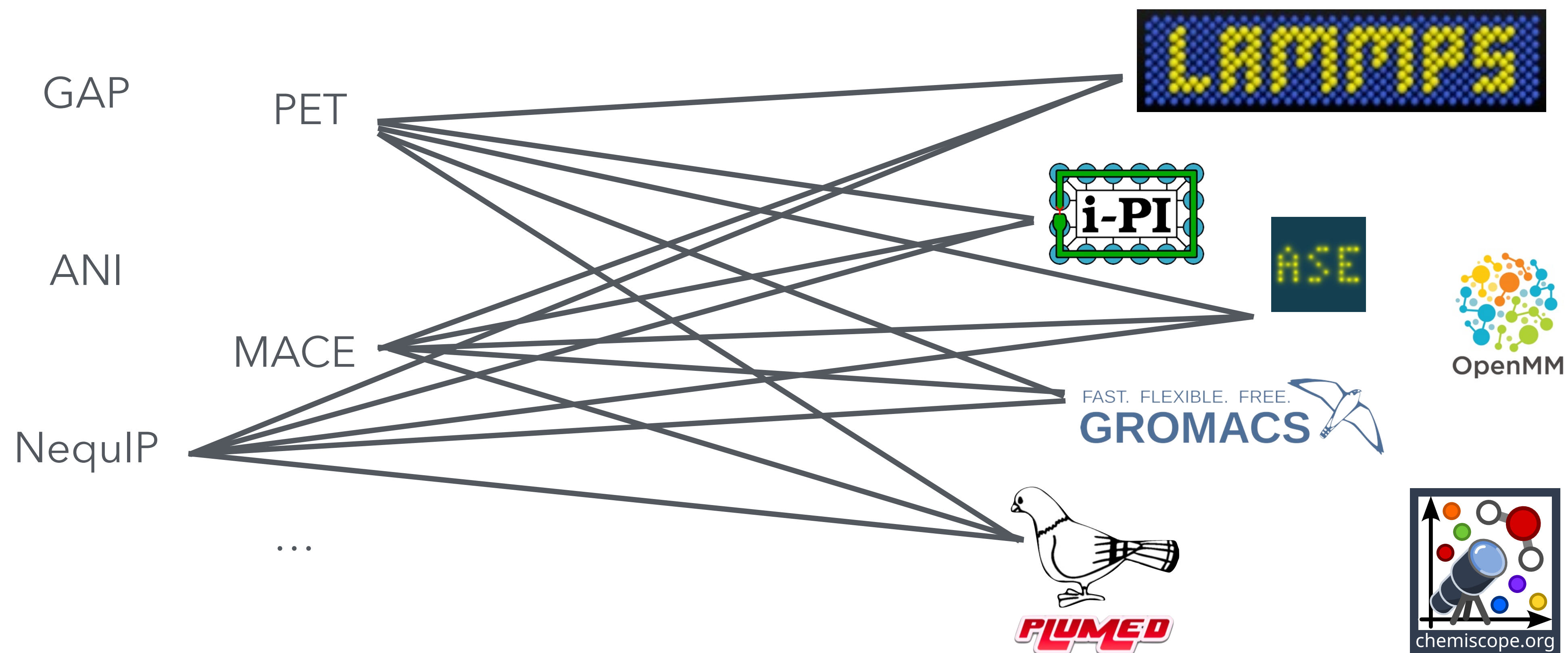
What are we trying to do?

- Share atomistic machine learning models between developers and users
- Reduce the amount of work to interface ML models with simulation engines



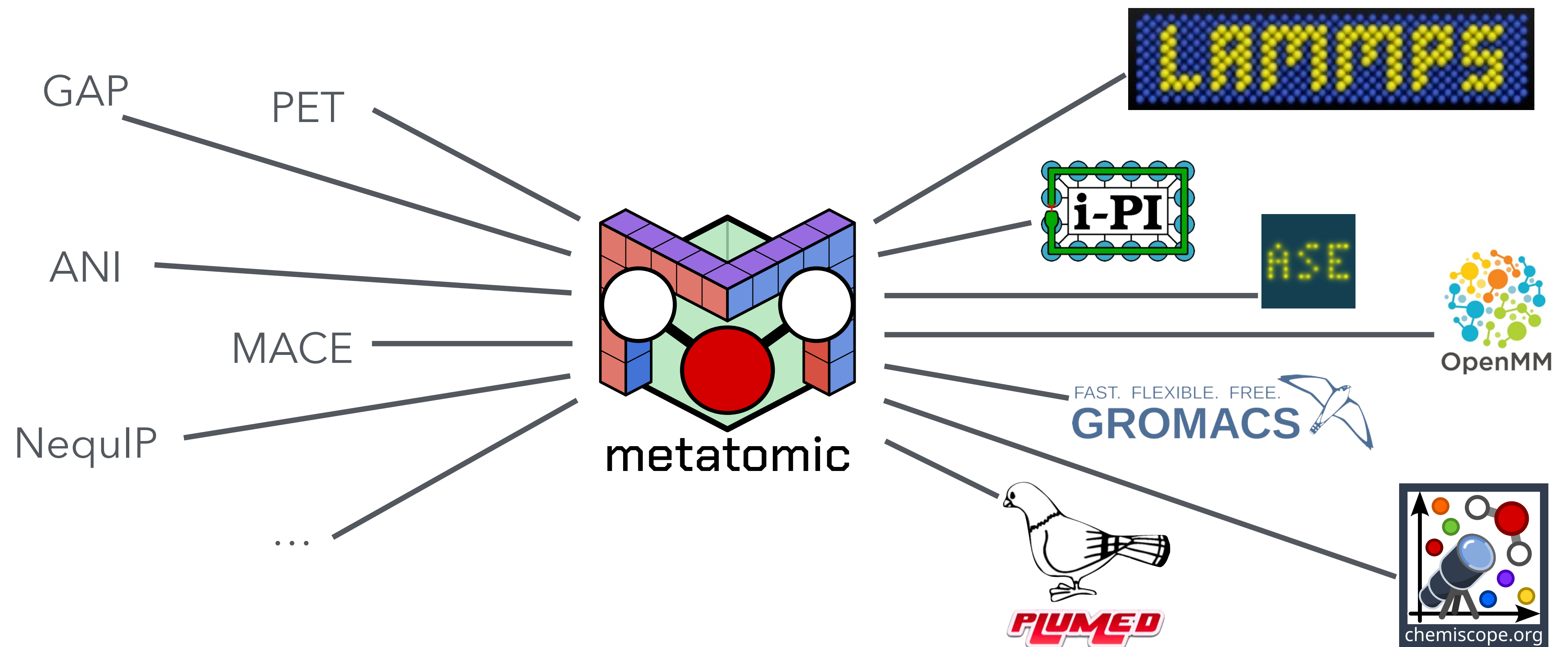
What are we trying to do?

- Share atomistic machine learning models between developers and users
- Reduce the amount of work to interface ML models with simulation engines



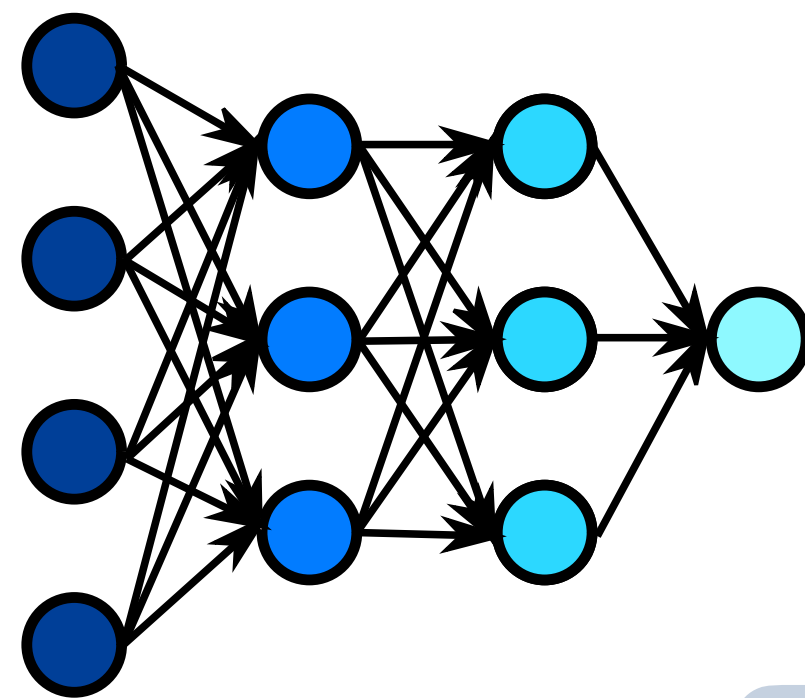
What are we trying to do?

- Share atomistic machine learning models between developers and users
- Reduce the amount of work to interface ML models with simulation engines



What are we trying to do?

- Share atomistic machine learning models between developers and users
- Reduce the amount of work to interface ML models with simulation engines
- Allow using ML models for every kind of tasks at the atomic scale



Run large-scale MD simulations

Quantify uncertainty

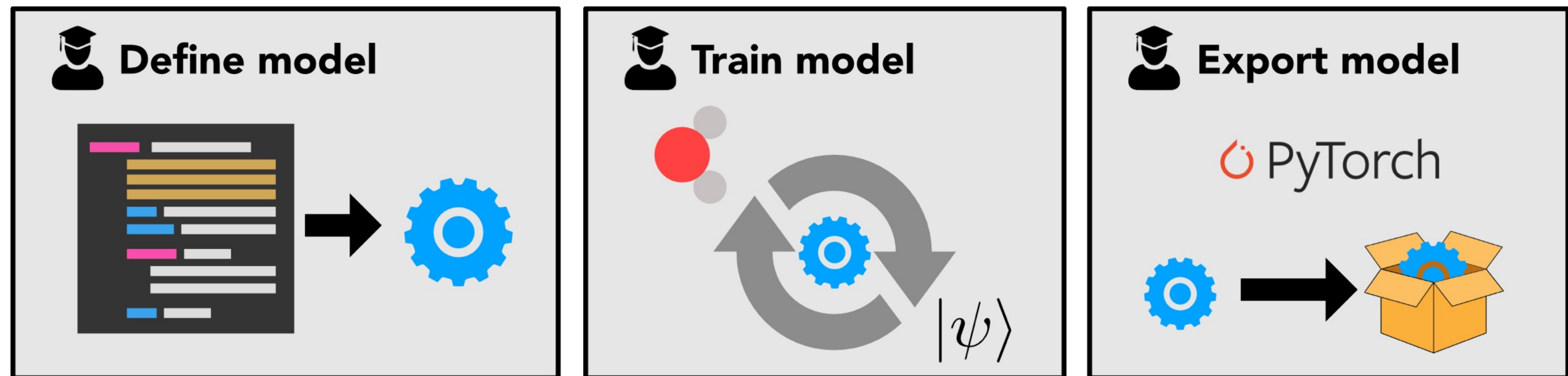
Predict atomic properties

Use ML-learned collective variables

Integrate inside *ab initio* calculations

How it works?

- Format to store trained models, based on TorchScript (**code** + data), which can be executed without requiring a Python setup.



- Load the model in a simulation engines, communicate informations, execute the model

Communication between the model and engine

Capabilities

Outputs

Devices (CPU, GPU, ...)

Data type

Interaction range

Supported atom types

Required inputs

Positions

Cell

Atomic types

Neighbor lists

Metadata

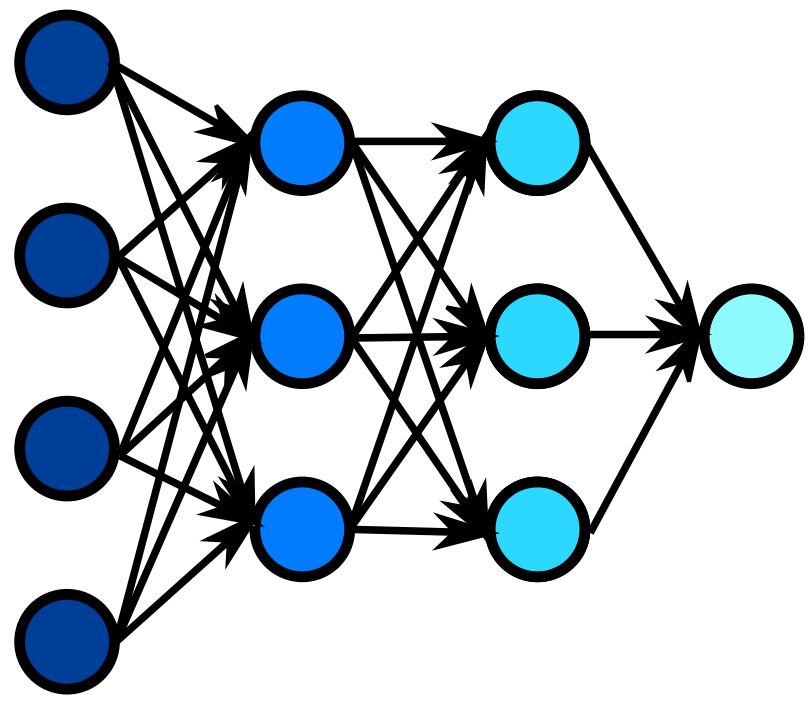
Model name

Description

Model authors

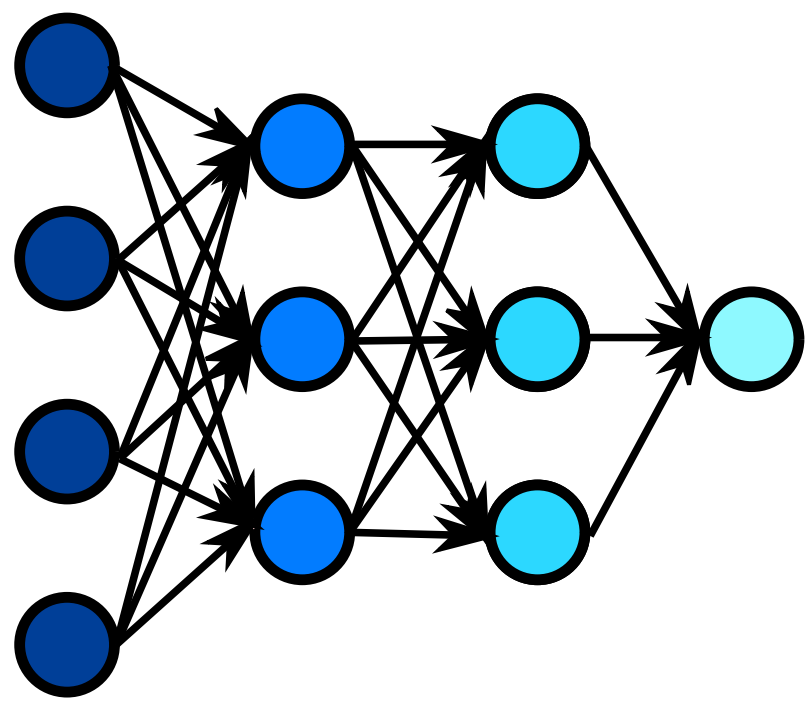
Paper references

One model, many outputs

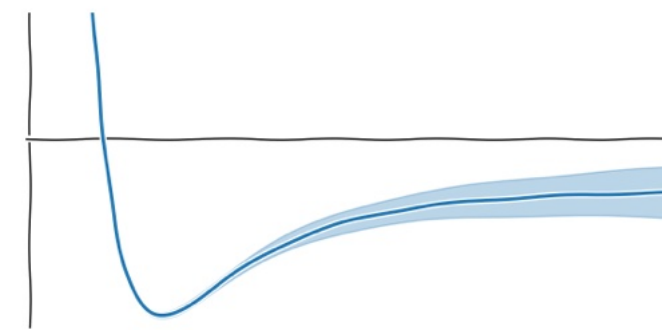


Single ML model

One model, many outputs

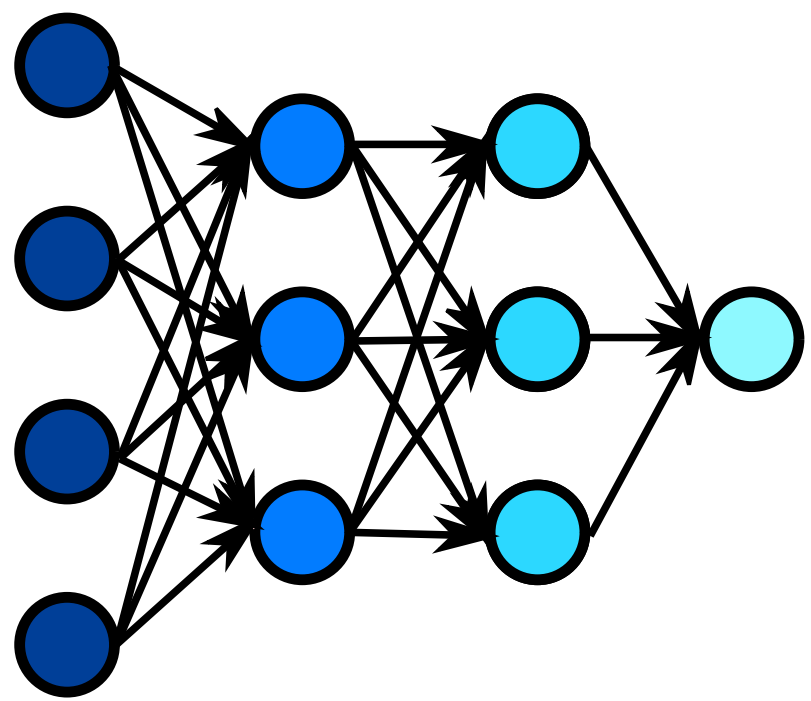


Single ML model

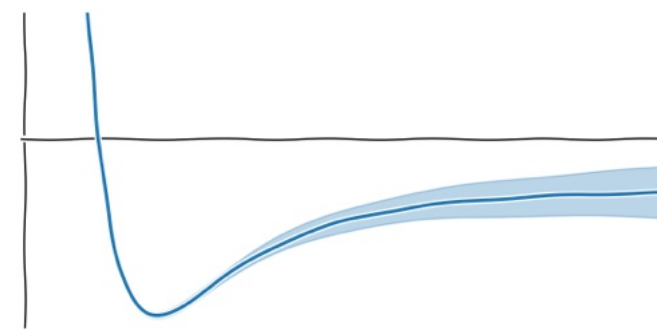


energy & forces

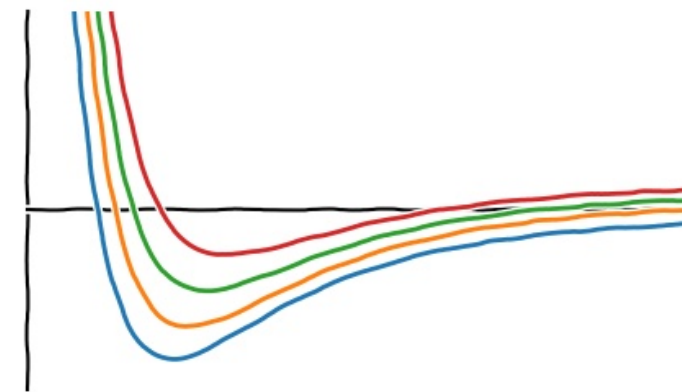
One model, many outputs



Single ML model

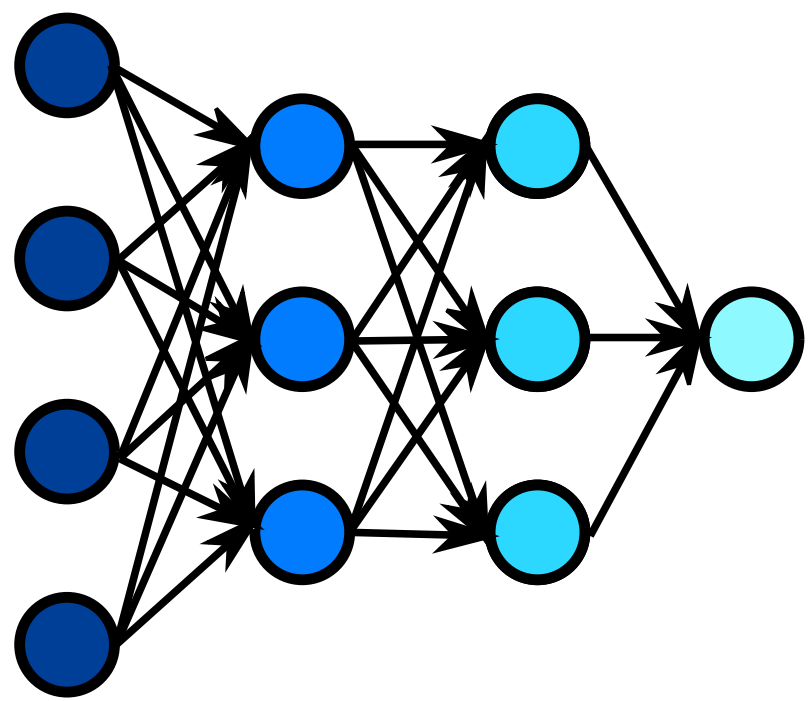


energy & forces

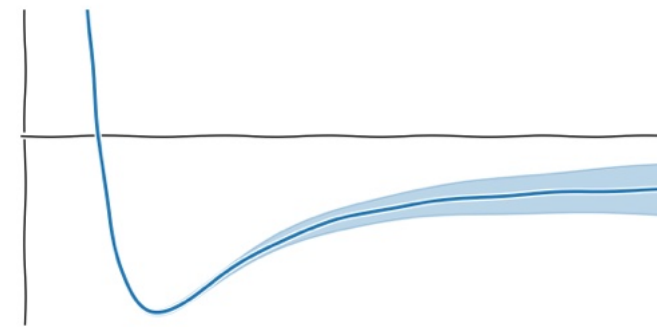


energy uncertainty

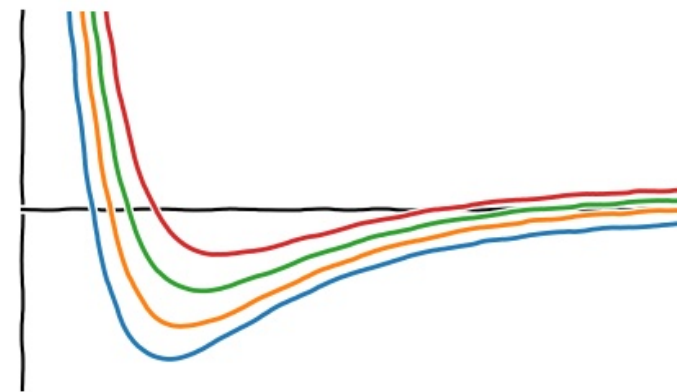
One model, many outputs



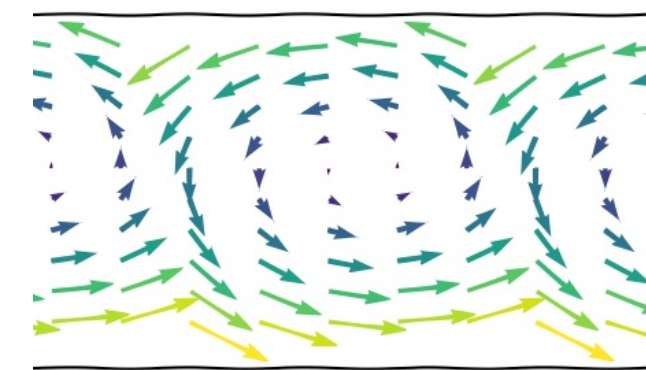
Single ML model



energy & forces

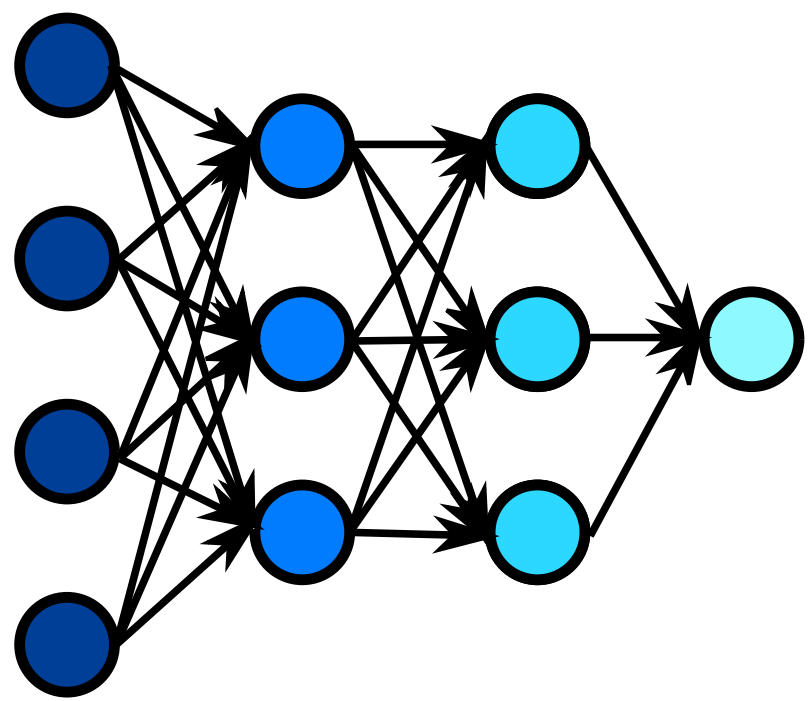


energy uncertainty

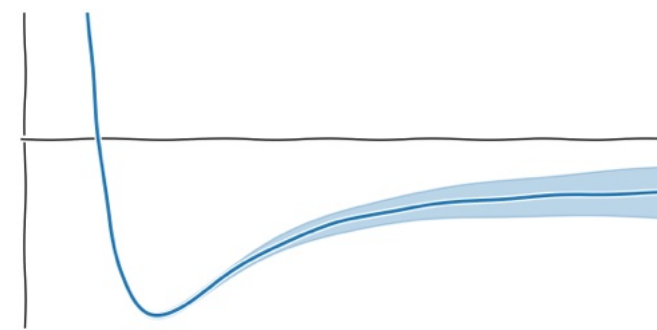


non-conservative

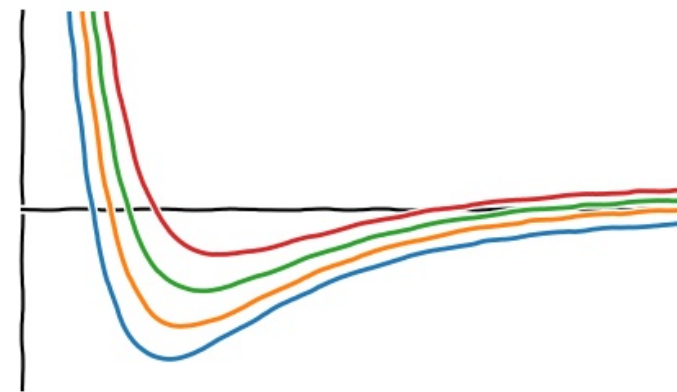
One model, many outputs



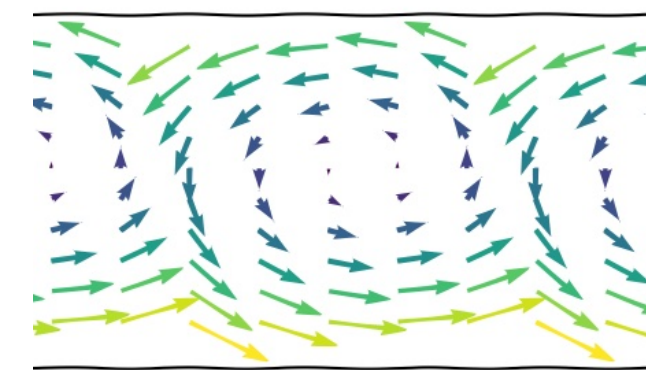
Single ML model



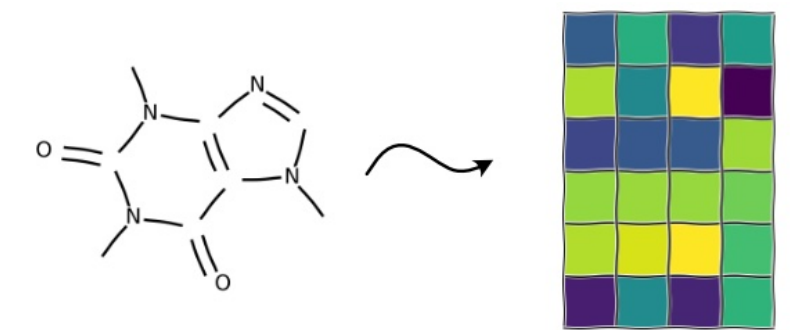
energy & forces



energy uncertainty

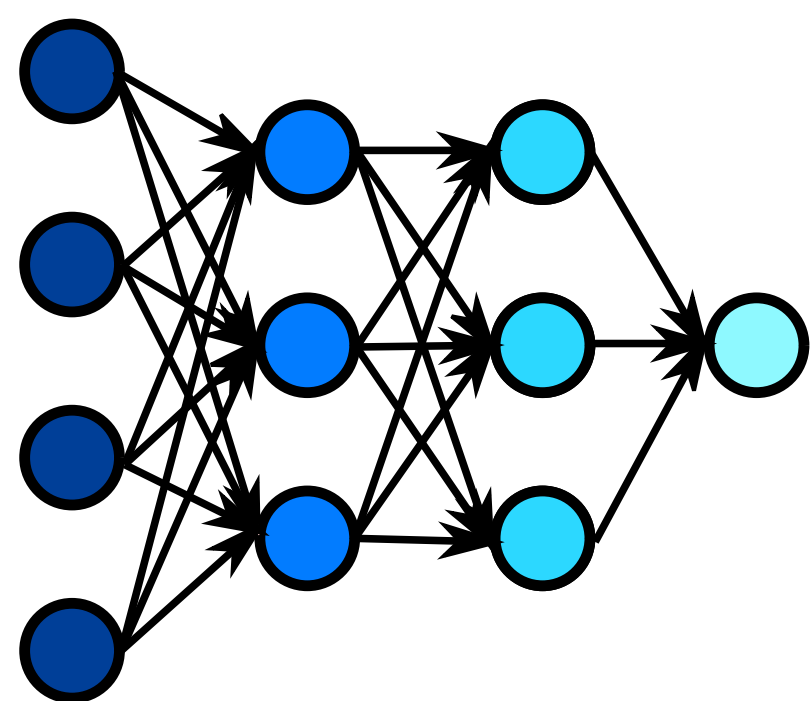


non-conservative

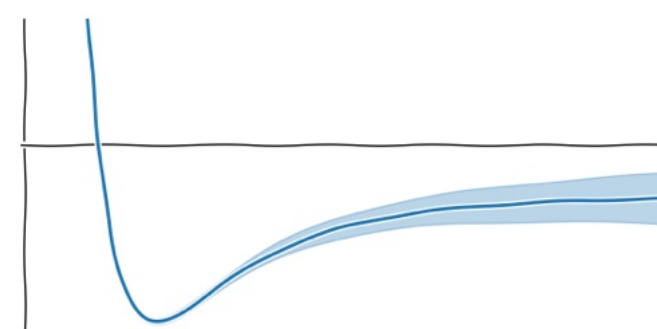


collective variables

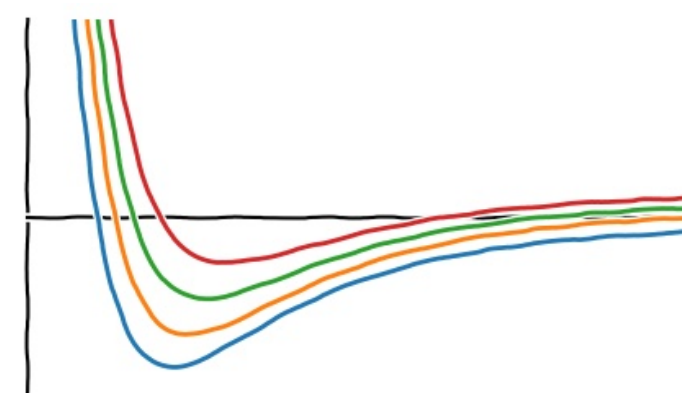
One model, many outputs



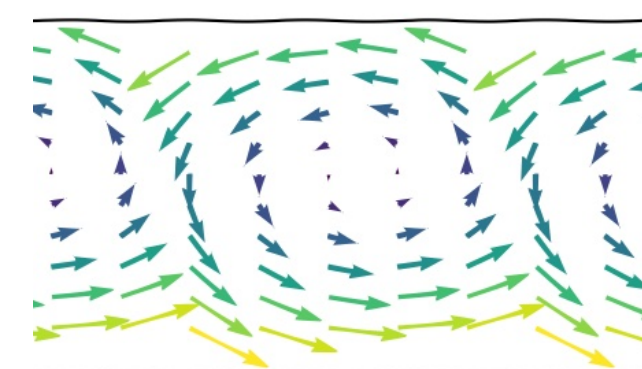
Single ML model



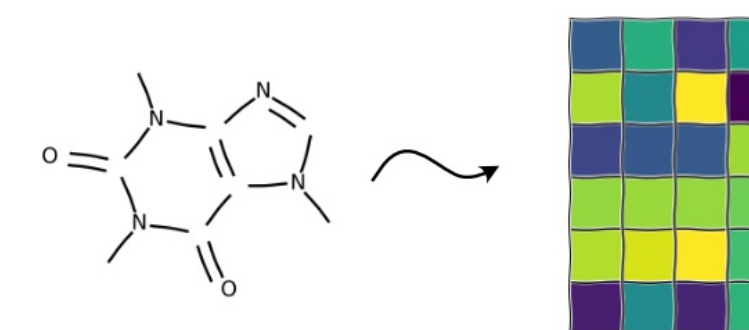
energy & forces



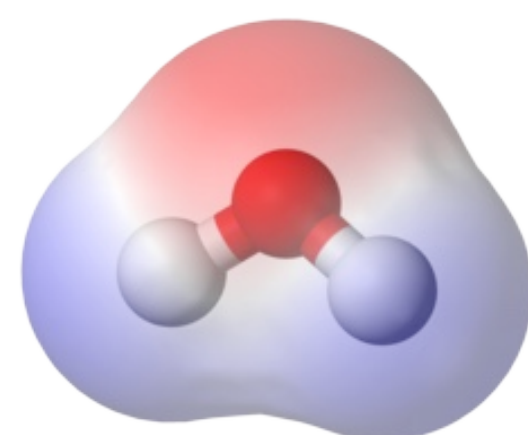
energy uncertainty



non-conservative

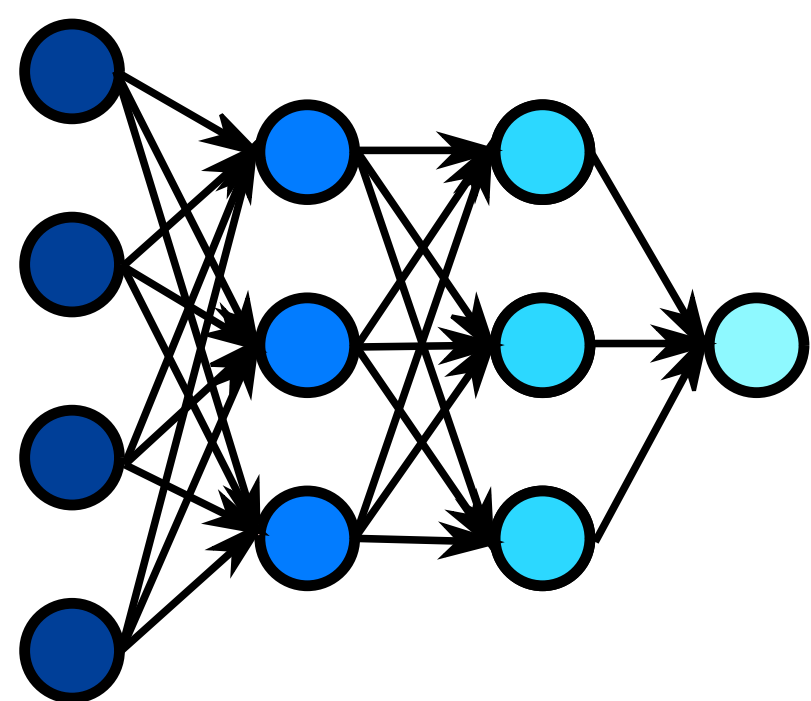


collective variables

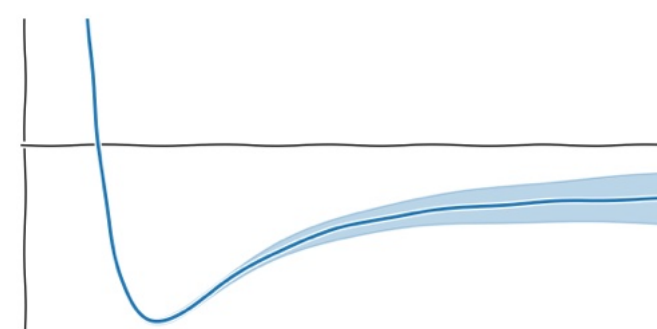


dipole, polarisation

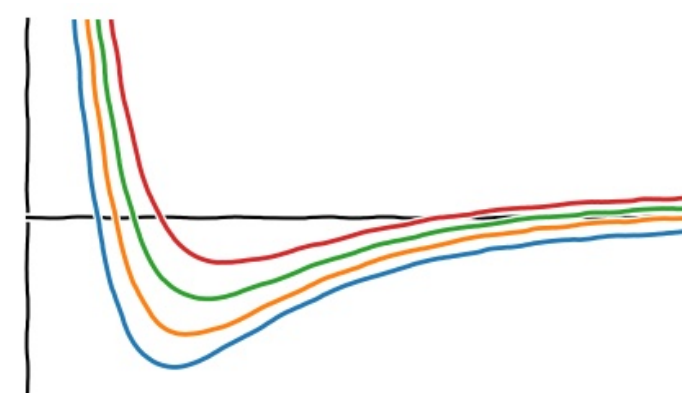
One model, many outputs



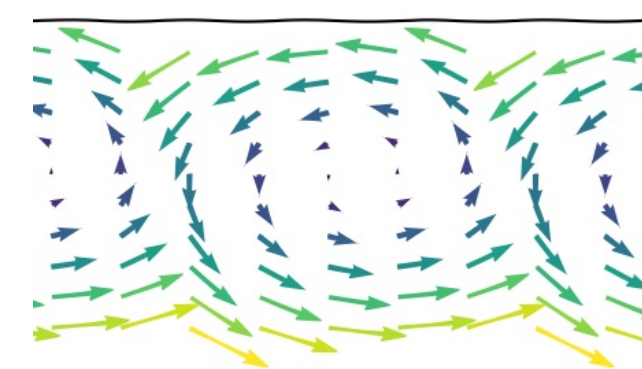
Single ML model



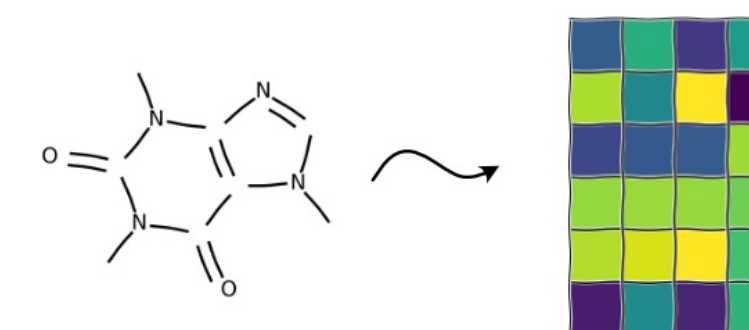
energy & forces



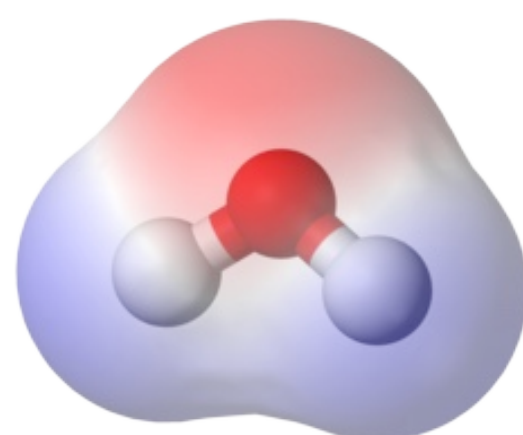
energy uncertainty



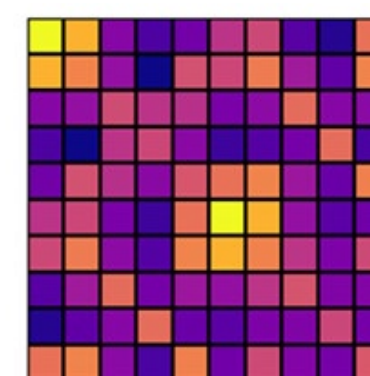
non-conservative



collective variables

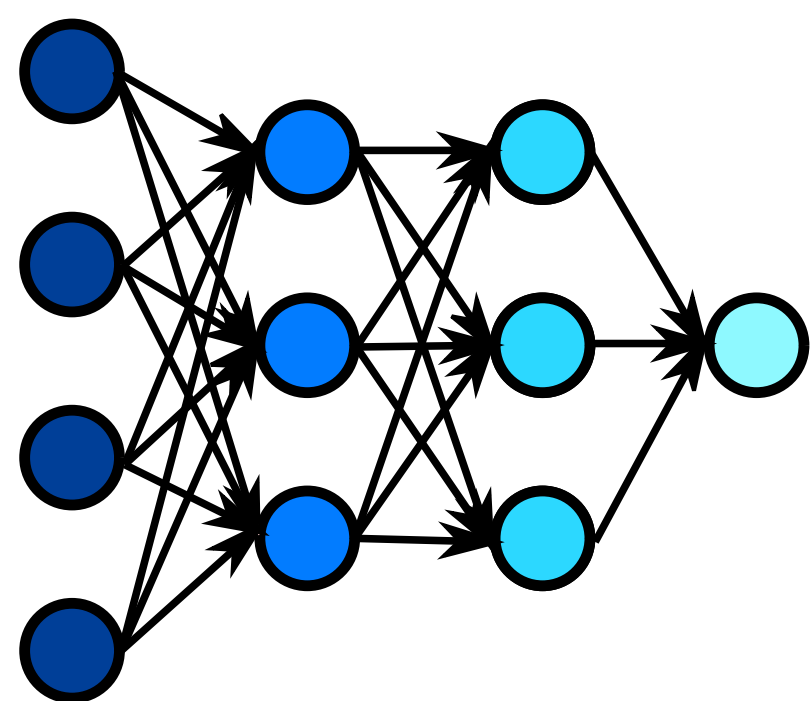


dipole, polarisation

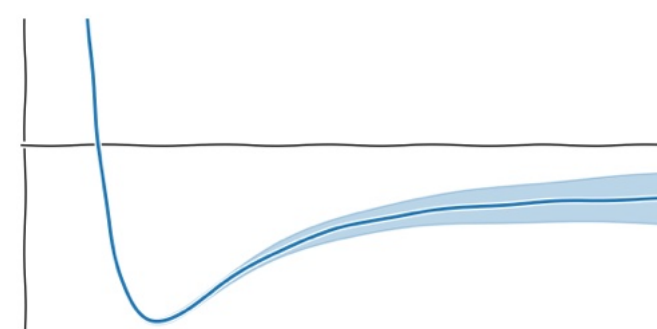


Hamiltonian

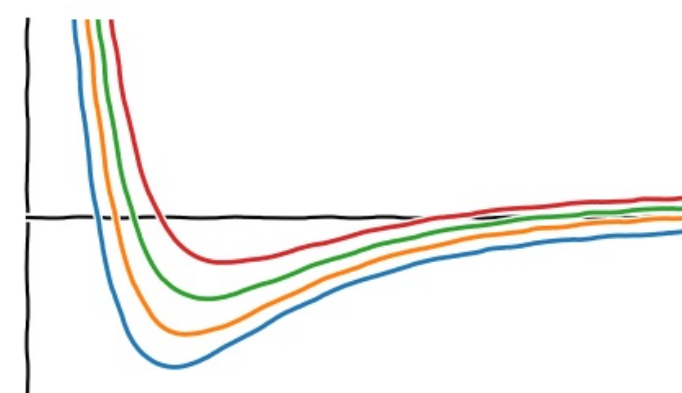
One model, many outputs



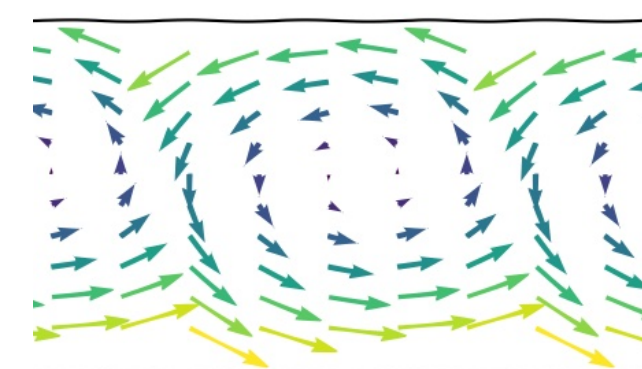
Single ML model



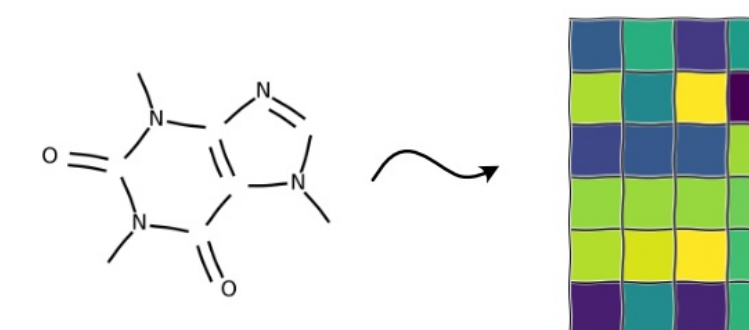
energy & forces



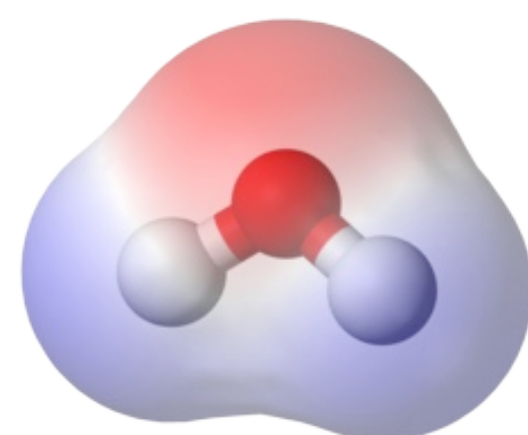
energy uncertainty



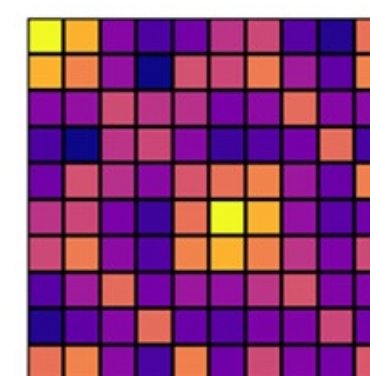
non-conservative



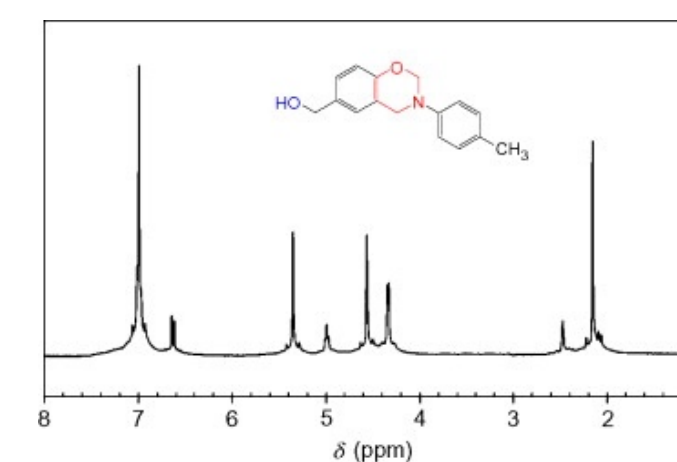
collective variables



dipole, polarisation

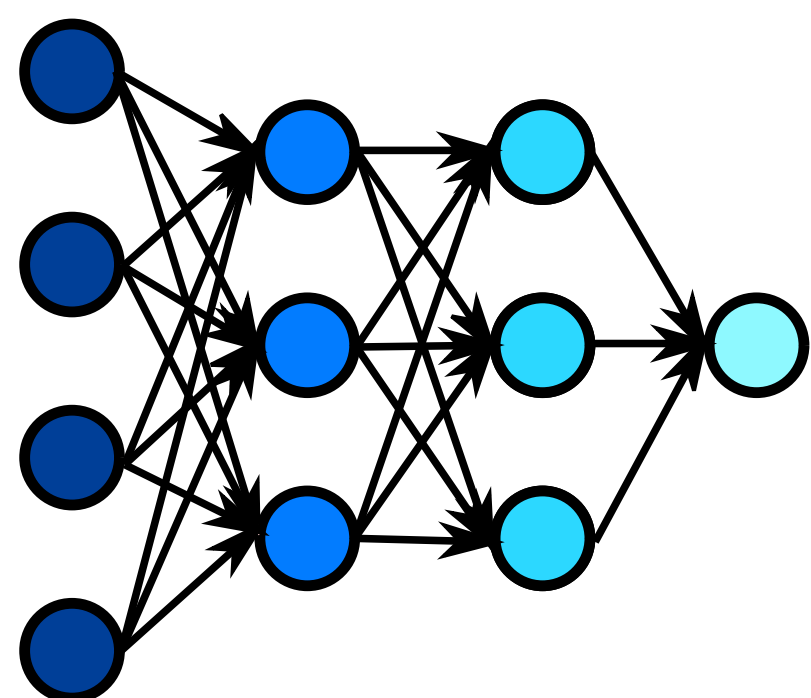


Hamiltonian

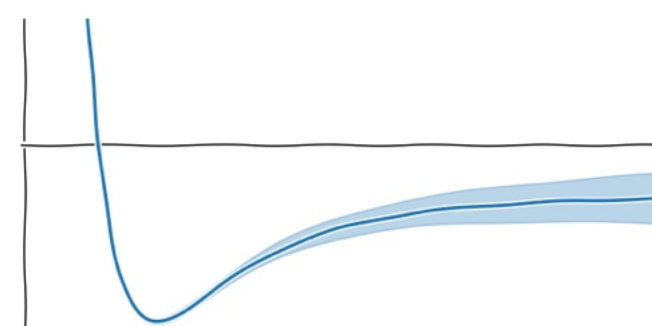


NMR shielding

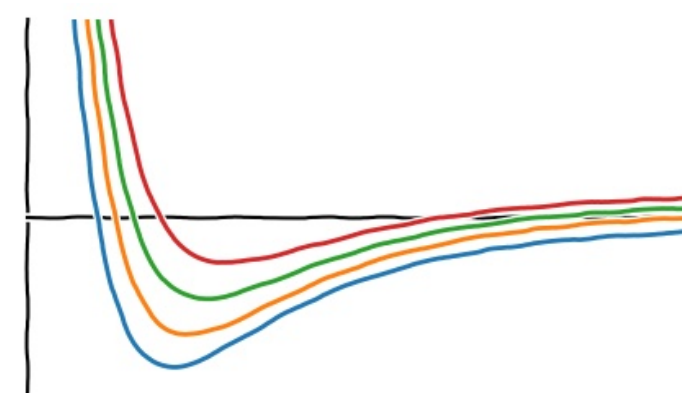
One model, many outputs



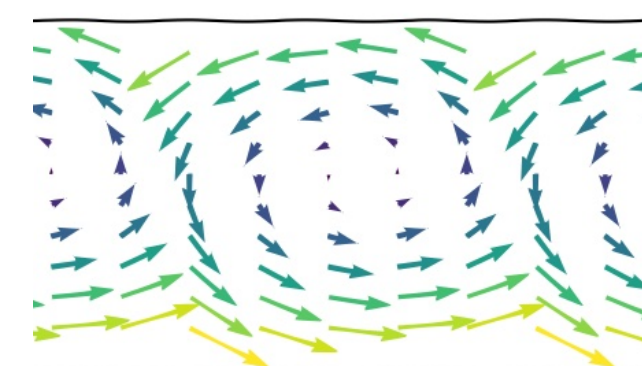
Single ML model



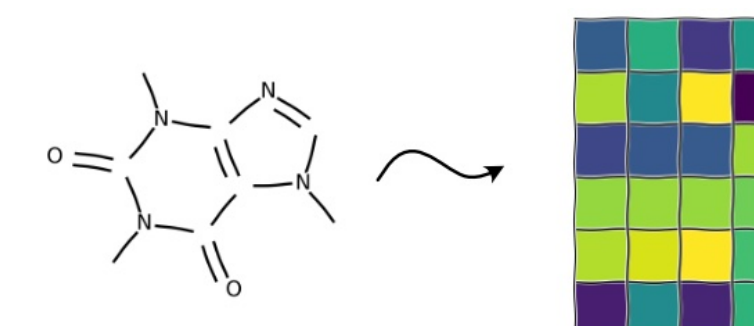
energy & forces



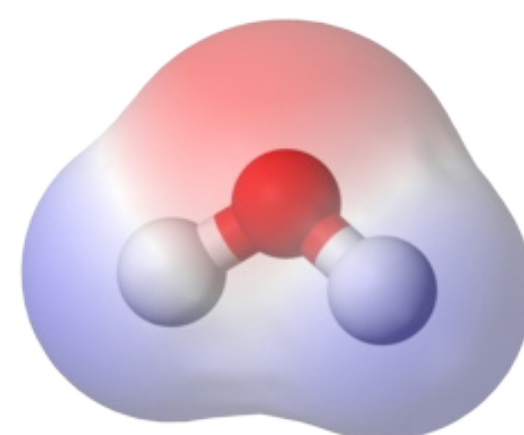
energy uncertainty



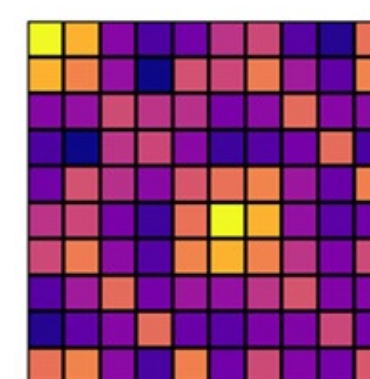
non-conservative



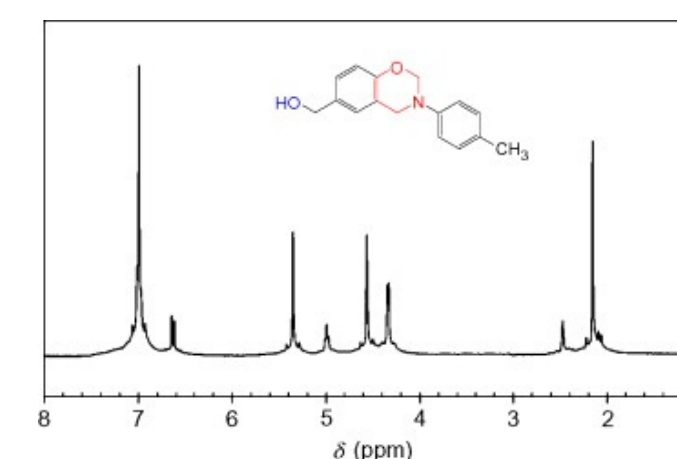
collective variables



dipole, polarisation



Hamiltonian

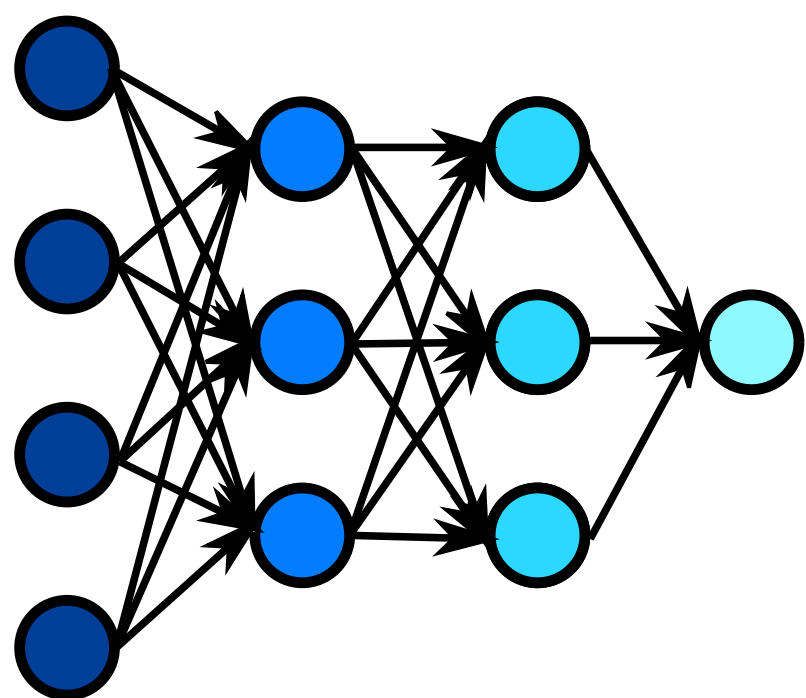


NMR shielding

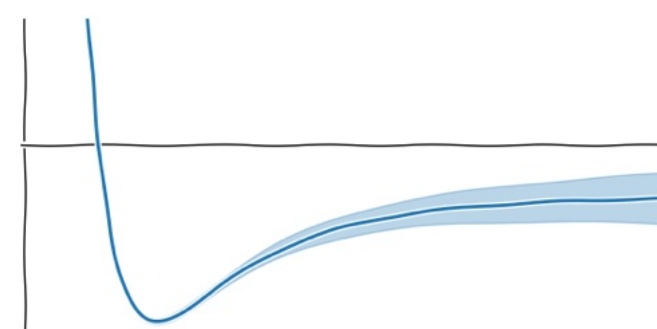
Density of states
Electron density
Positions prediction

...

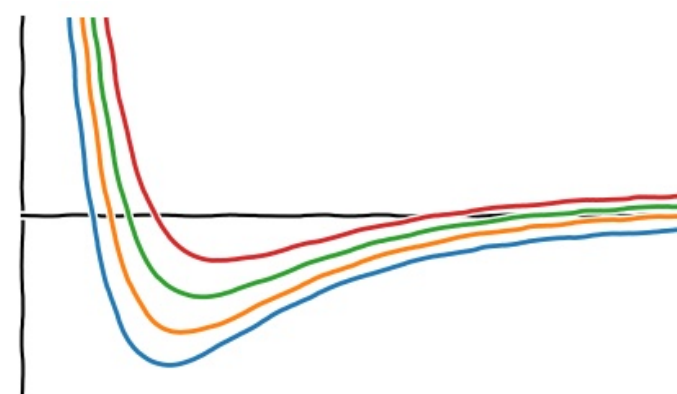
One model, many outputs



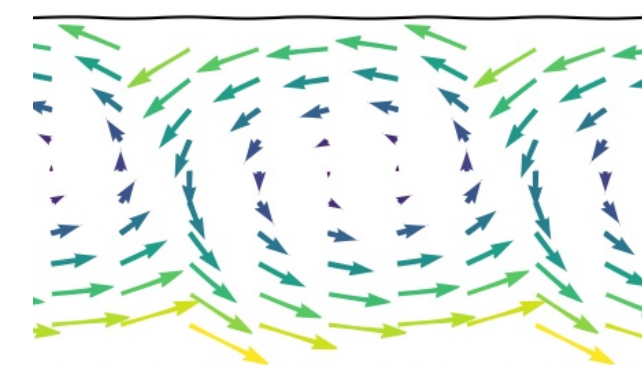
Single ML model



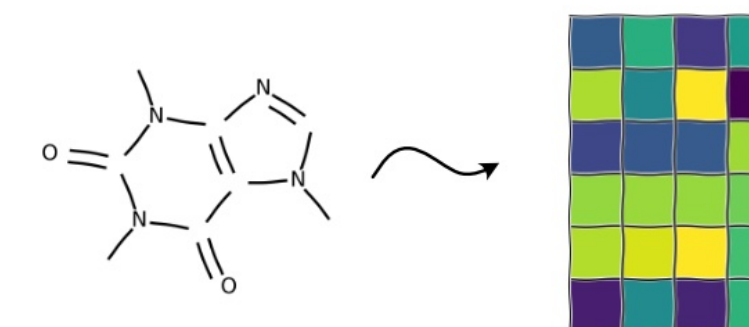
energy & forces



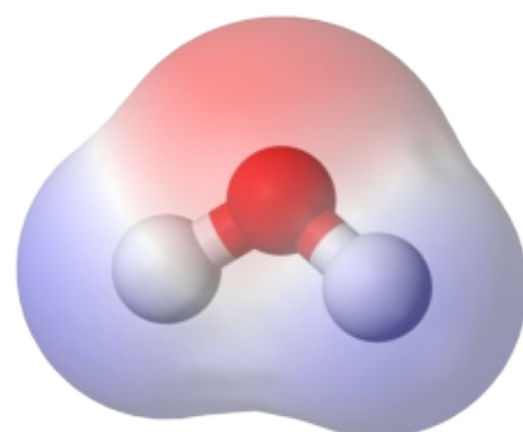
energy uncertainty



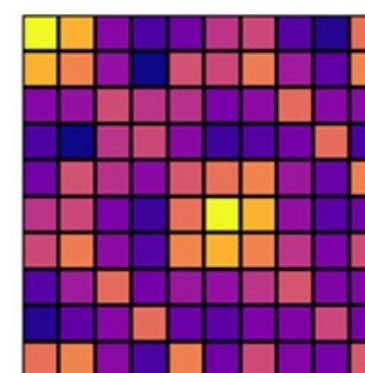
non-conservative



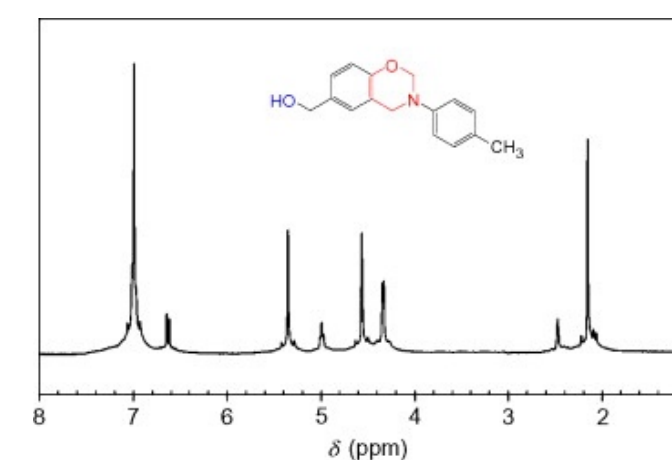
collective variables



dipole, polarisation



Hamiltonian

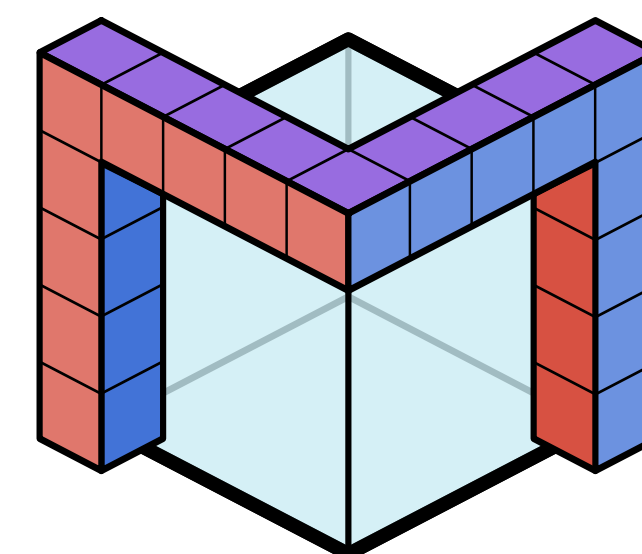


NMR shielding

Density of states
Electron density
Positions prediction

...

Store and exchange all kind of
output data with metatensor

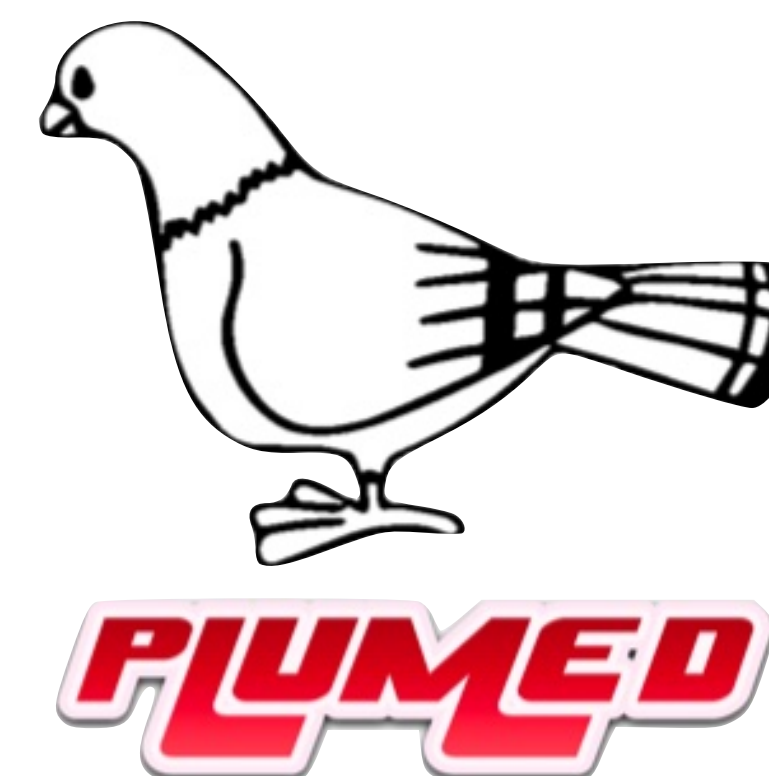


Existing interfaces to simulation tools



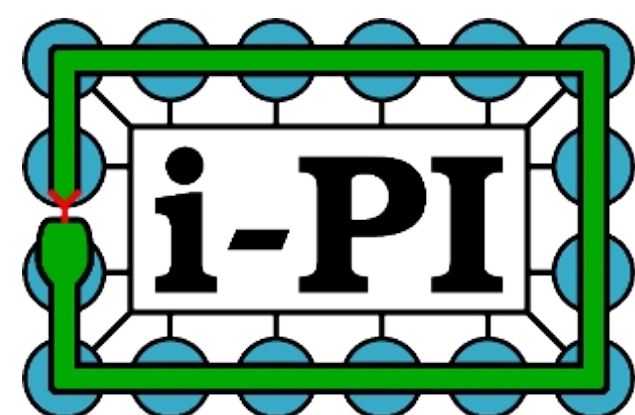
Large scale simulations
GPU acceleration with kokkos

Non-conservative forces for faster simulations,
corrected with multiple time stepping



Enhanced sampling

Custom CV with
Python code



Advanced time integrations,
including nuclear quantum effects

Propagate uncertainty from energy
to macroscopic observables



Ease of use from Python,
can evaluate all outputs



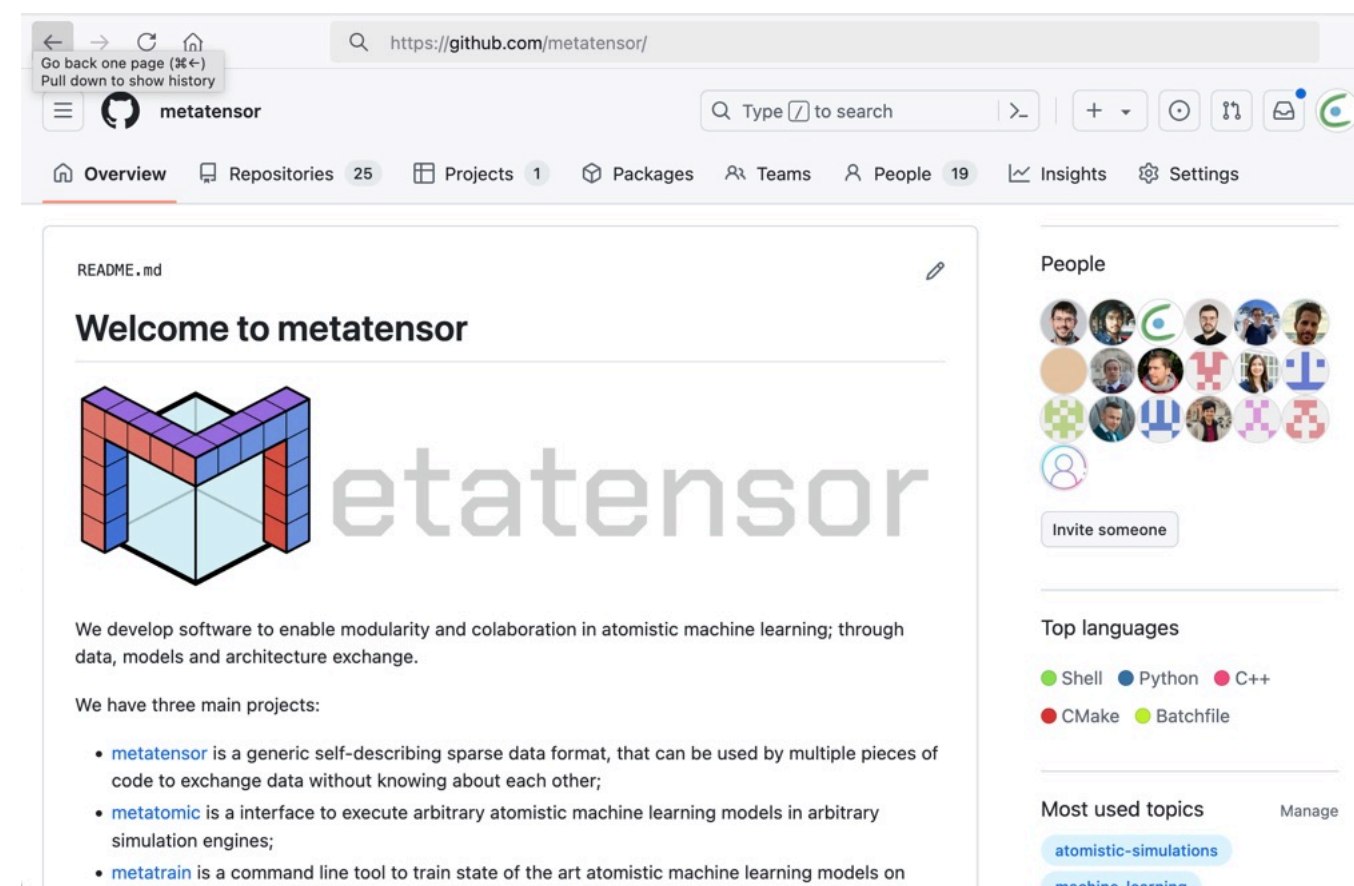
Interactive data exploration
Interactive CV creation

What's possible with all of this?

See all the upcoming presentations!

Building high quality code

Build in the open

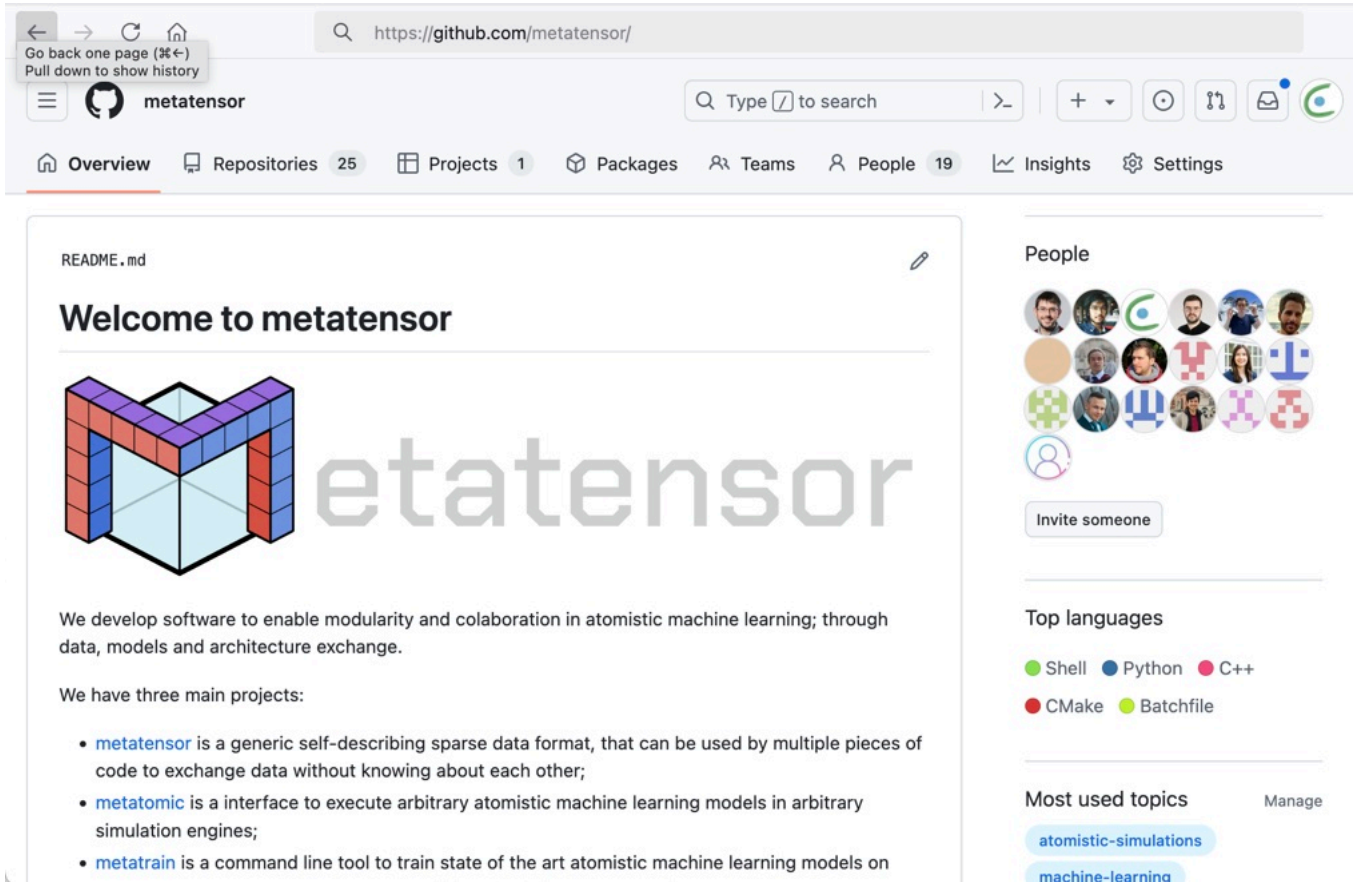


<https://github.com/metatensor>

Building high quality code

Build in the open

Comprehensive testing



All checks have passed

69 successful checks

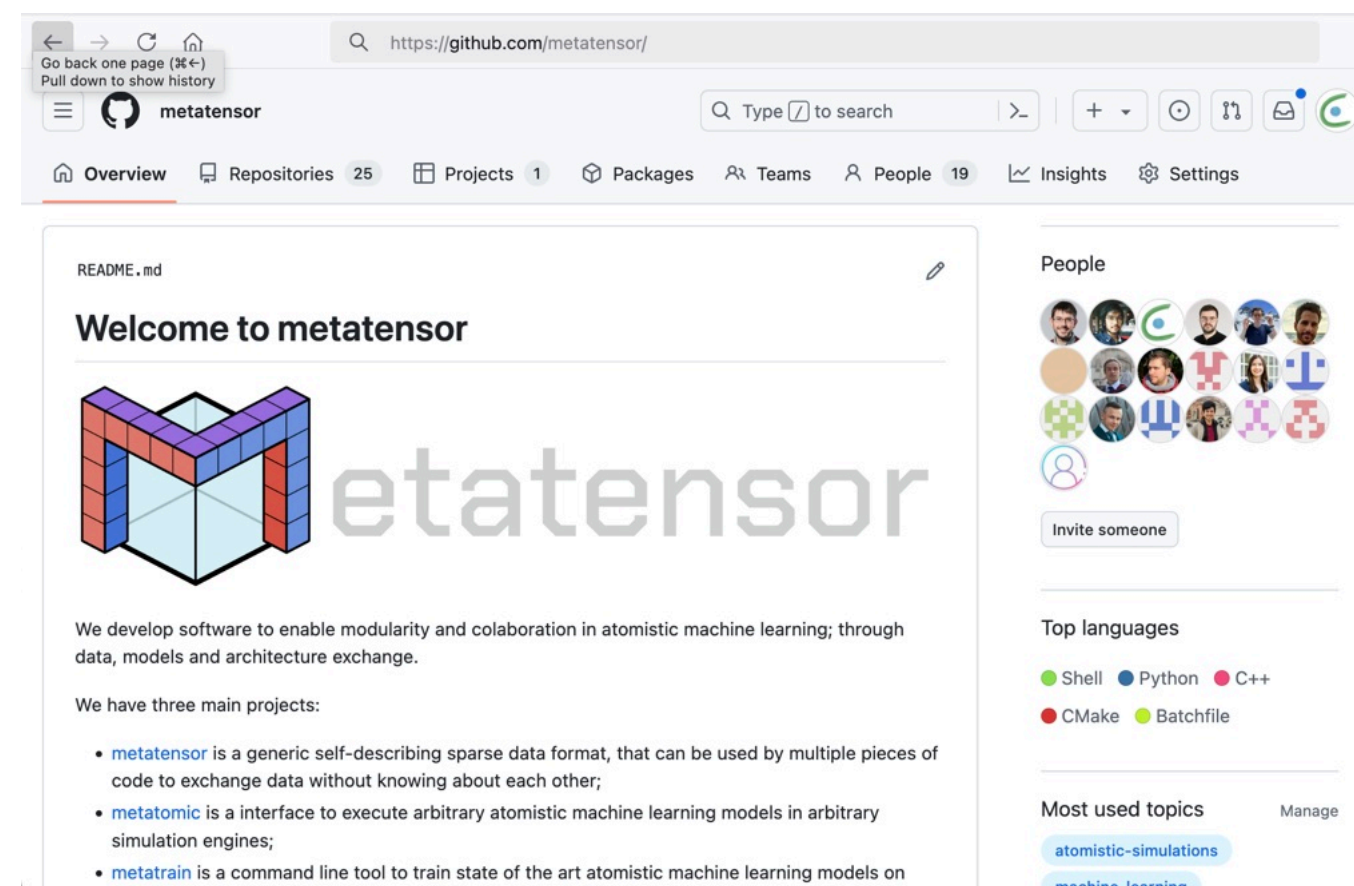
✓		Build Python wheels / x86_64 Linux (pull_request) ...	Details
✓		Coverage / collect code coverage (pull_request) S...	Details
✓		Documentation / build-and-publish (pull_request) ...	Details

<https://github.com/metatensor>

Building high quality code

Build in the open

Comprehensive testing



All checks have passed

69 successful checks

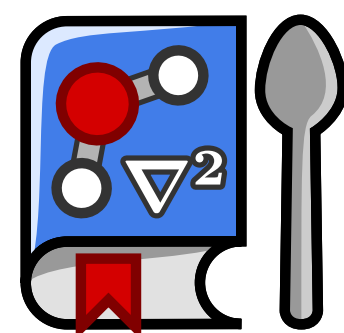
✓		Build Python wheels / x86_64 Linux (pull_request) ...	Details
✓		Coverage / collect code coverage (pull_request) S...	Details
✓		Documentation / build-and-publish (pull_request) ...	Details

<https://github.com/metatensor>

Docs and tutorials

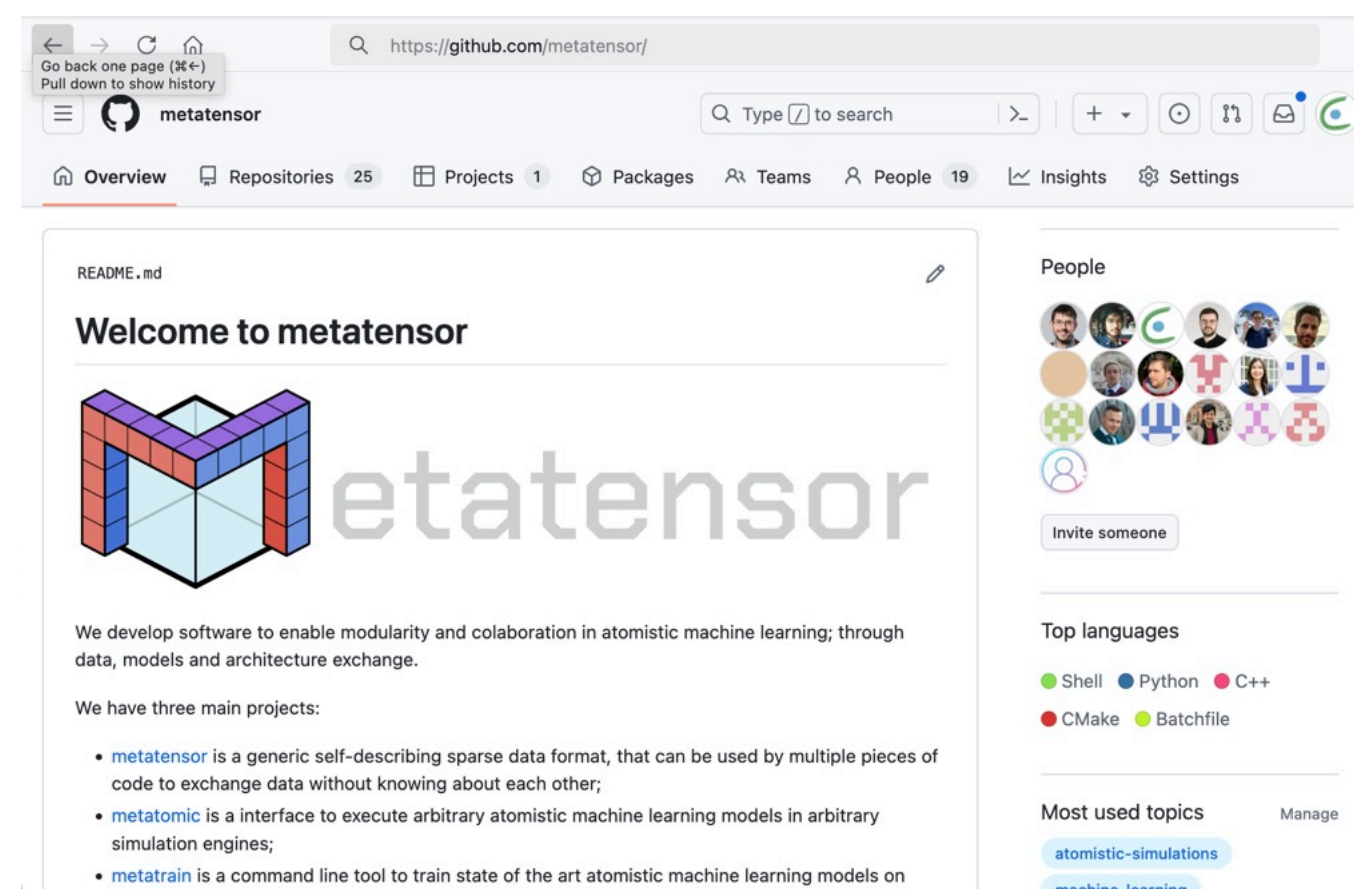
<https://docs.metatensor.org>

<https://atomistic-cookbook.org>



Building high quality code

Build in the open

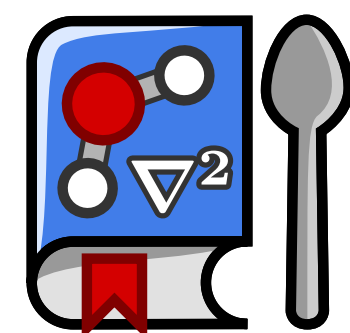


<https://github.com/metatensor>

Docs and tutorials

<https://docs.metatensor.org>

<https://atomistic-cookbook.org>



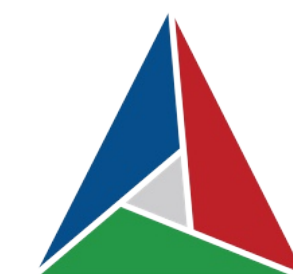
Comprehensive testing

All checks have passed

69 successful checks

✓		Build Python wheels / x86_64 Linux (pull_request) ...	Details
✓		Coverage / collect code coverage (pull_request) S...	Details
✓		Documentation / build-and-publish (pull_request) ...	Details

Packaging and installation



CMake



PyPI



conda



crates.io

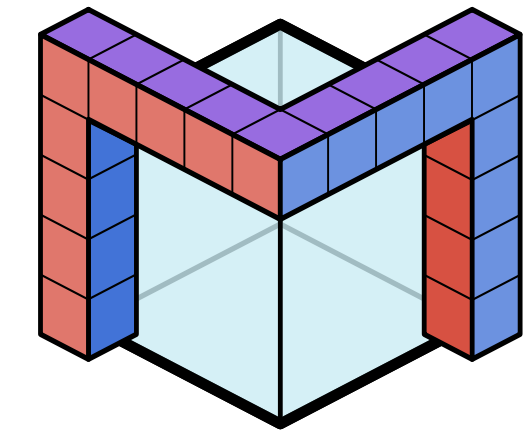


spack

What's next?

Better manipulation function, focus on usability

Integrate with more ecosystems (JAX, Julia, Fortran, ...)

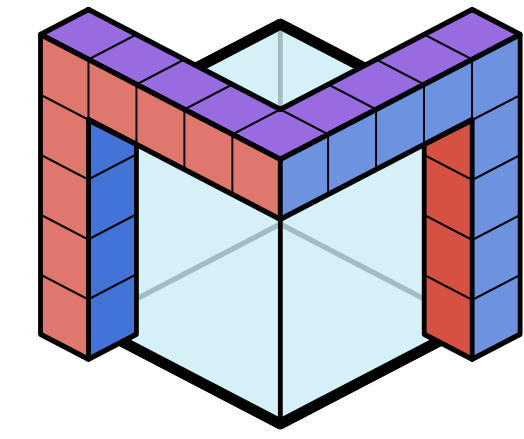


metatensor

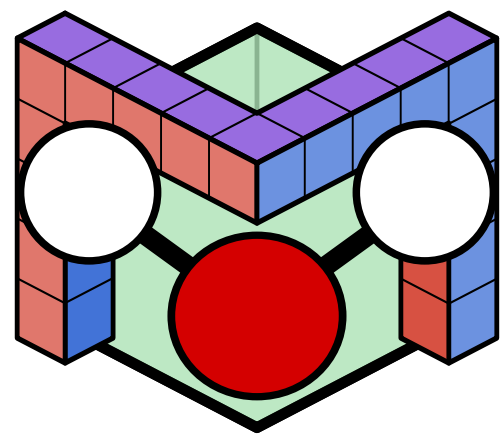
What's next?

Better manipulation function, focus on usability

Integrate with more ecosystems (JAX, Julia, Fortran, ...)



metatensor



metatomic

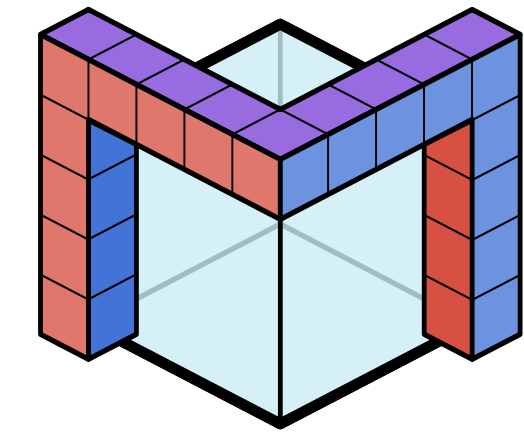
Decouple from TorchScript: use Julia model in LAMMPS! Python model in CP2K!

Integrate with more simulation engines (GROMACS, ESPREesSo, OpenMM, ...)

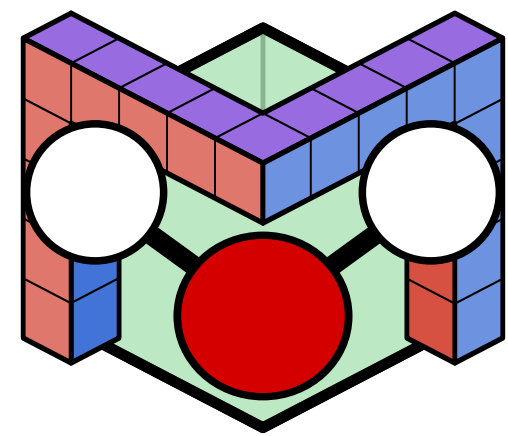
What's next?

Better manipulation function, focus on usability

Integrate with more ecosystems (JAX, Julia, Fortran, ...)



metatensor



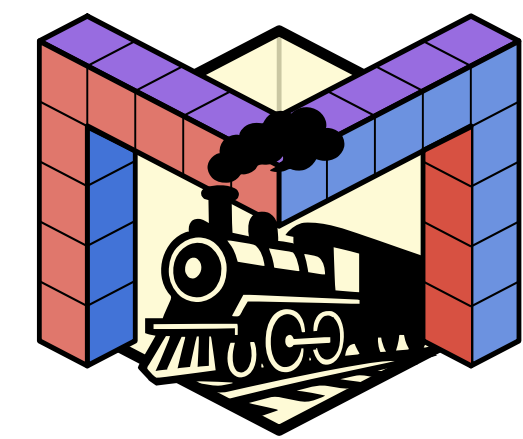
metatomic

Decouple from TorchScript: use Julia model in LAMMPS! Python model in CP2K!

Integrate with more simulation engines (GROMACS, ESPREesSo, OpenMM, ...)

Train on complex targets, access to more varied architectures

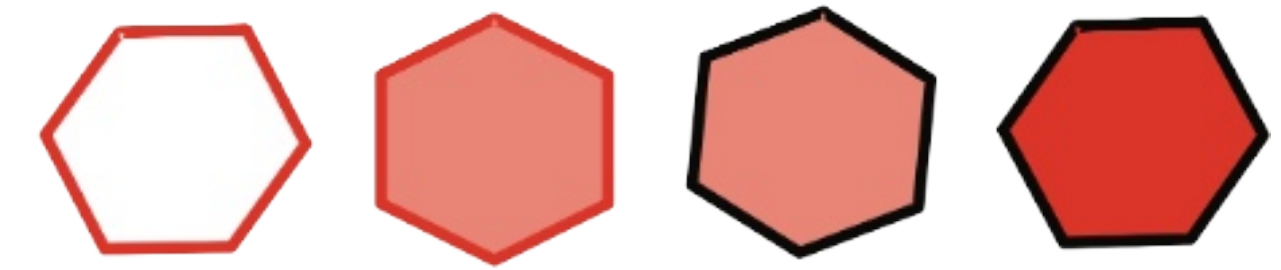
Integrate with workflow engines for active learning, dataset generation



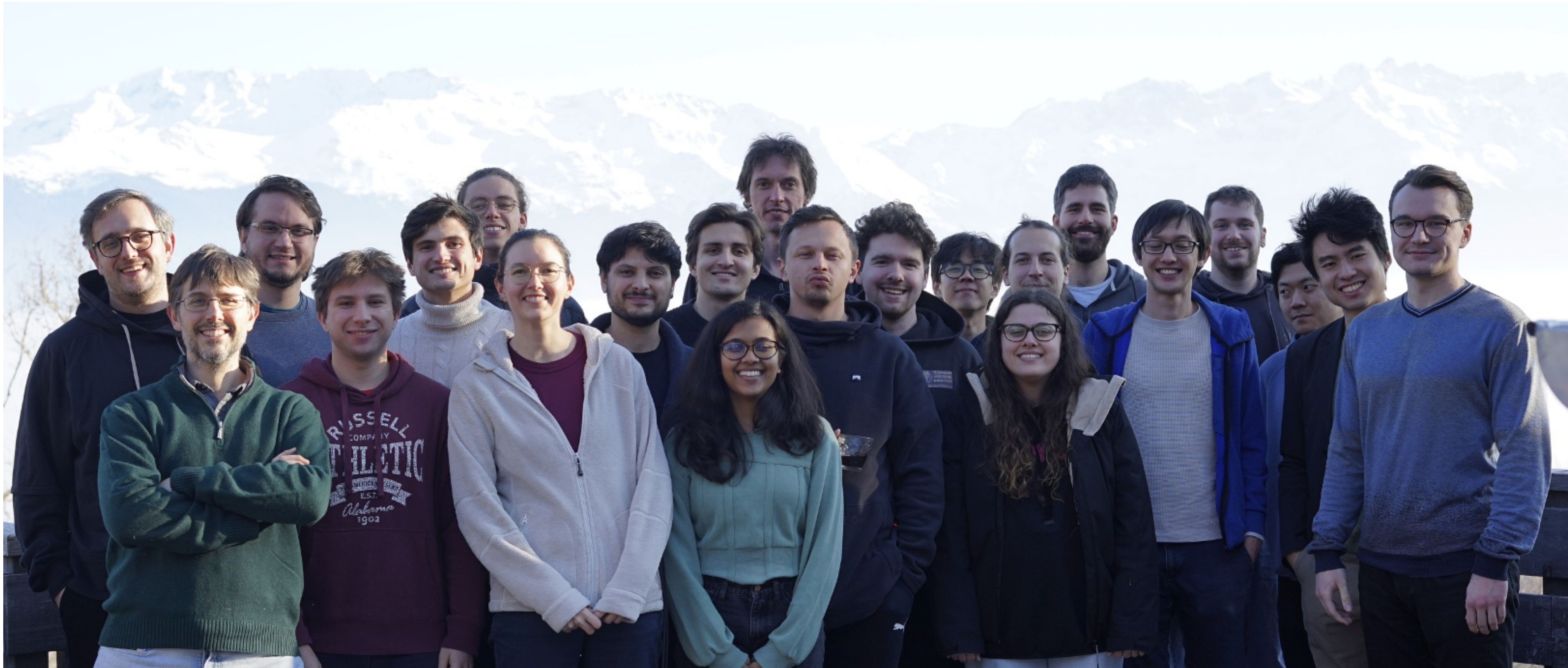
metatraining

Thanks everyone!

MARVEL



NATIONAL CENTRE OF COMPETENCE IN RESEARCH



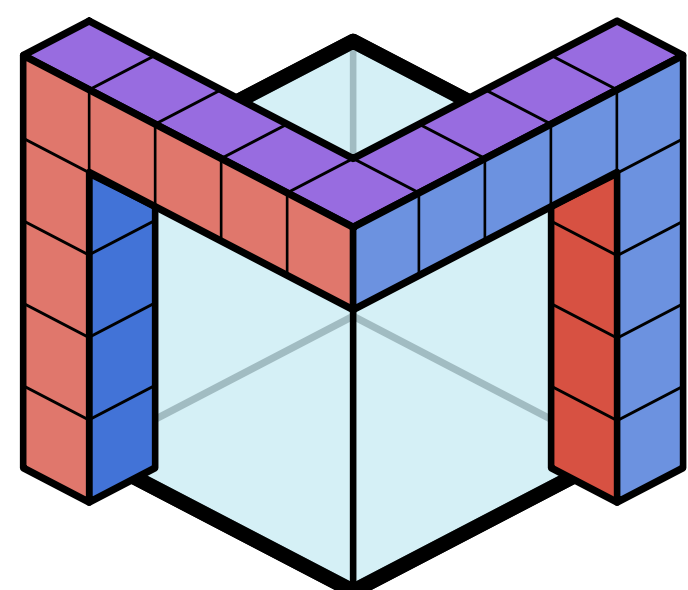
Alex Goscinsky



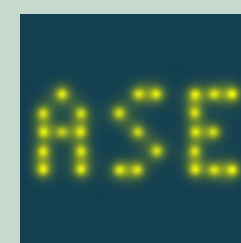
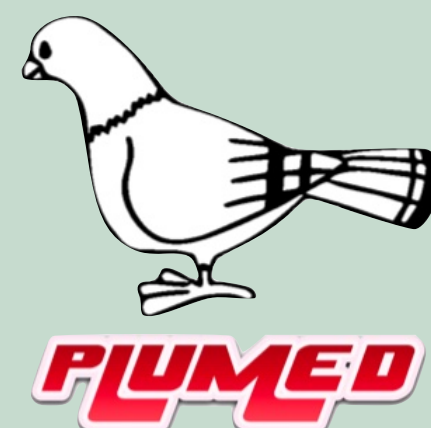
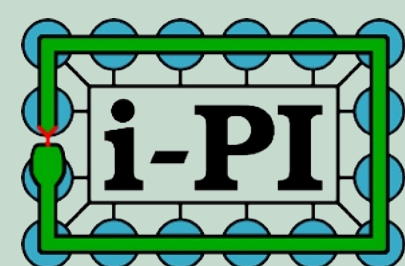
Jigyasa Nigam



Gareth Tribello



metatensor



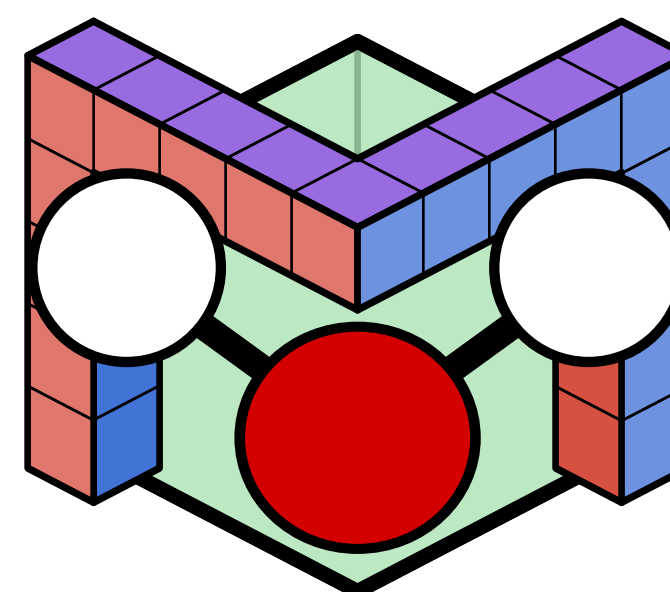
Torch-sim

ShiftML

PET-MAD

mts-operations

mts-learn



metatomic

SOAP-BPNN

LLPR

PET

GAP

NanoPET

SALTED

featonic

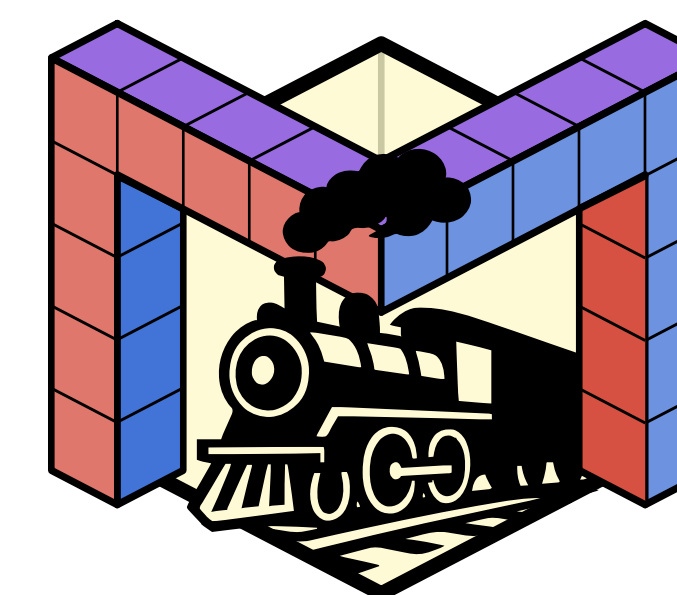
AniSOAP

vesin

spex



Q-Stack



metatrain